# CDC Compiler User Guide

Clock Domain Crossing Analysis

Release 2.3

Part Number 224246

May 10, 2005




**Copyright © Mentor Graphics Corporation 2005**
**All rights reserved.**

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

**RESTRICTED RIGHTS LEGEND 03/97**

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202-3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

**Contractor/manufacturer is**:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

**TRADEMARKS**: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: www.mentor.com/terms_conditions/trademarks.cfm.

# CONTENTS


About this Manual . . . . . 10
0-In Functional Verification Manuals . . . . . 10
Notational Conventions . . . . . 11
Syntax Conventions . . . . . 11
Directive Conventions . . . . . 12

**CHAPTER 1. CLOCK DOMAIN CROSSINGS 13**

Clock Domains . . . . . 15
Metastability . . . . . 16
Synchronizers . . . . . 17
Structured Synchronizers . . . . . 18
CDC Control Signal Synchronizers . . . . . 18
CDC Data Bus Synchronizers . . . . . 19
Ad Hoc Synchronizers . . . . . 21
Ratioed Synchronous Clocks . . . . . 21
Reconvergence . . . . . 23
Local Reconvergence . . . . . 23
Multicycle Reconvergence . . . . . 23
Multibit Reconvergence . . . . . 24
Global Reconvergence . . . . . 24
Verifying Reconvergence . . . . . 25
Netlist Analysis . . . . . 27
Assertion-Based Verification . . . . . 28
Specifying Design Intent . . . . . 29
CDC Checkers in Simulation . . . . . 30
Formal Verification with CDC Checkers . . . . . 31
Design Example . . . . . 31

Running the CDC Compiler . . . . . 36
Recommended Methodology . . . . . 37
cdc Command Invocation . . . . . 37
Working Directory . . . . . 39
cdc Input Files . . . . . 39
cdc Output Files . . . . . 41
CDC Schemes . . . . . 43
Attributes of CDC Schemes . . . . . 43
Severity . . . . . 43
Promotion . . . . . 44
CDC Synchronization Schemes . . . . . 45
Signal Synchronization . . . . . 45
Data Synchronization . . . . . 46
Signal or Data Synchronization . . . . . 46
Schemes of Potential CDC Problems . . . . . 48
Adjusting CDC Analysis . . . . . 49
Specifying Clocks and Clock Groups . . . . . 49
Clocks . . . . . 49
Clock Groups . . . . . 50
set_cdc_clock . . . . . 50
Assigning Ports to Clock Domains . . . . . 51
set_cdc_port_domain . . . . . 51
Setting Port Values to Constants . . . . . 52
set_constant . . . . . 52
Specifying Non-standard Synchronizer Types . . . . . 53
set_cdc_synchronizer . . . . . 54
Classifying Clock Domain Crossings . . . . . 55
set_cdc_signal . . . . . 56
Changing CDC Reporting and Promotion . . . . . 58
Severity Levels . . . . . 58
Promoted Checks . . . . . 59
set_cdc_report . . . . . 59
Other Global Directives . . . . . 61
set_reset . . . . . 61
exclude_hier . . . . . 61
set_black_box . . . . . 61

set_cdc_reconvergence_depth . . . . . 62

**CHAPTER 3. CDC SCHEME DATA SHEETS 63**

async_reset . . . . . 65
async_reset_no_sync . . . . . 66
blackbox . . . . . 67
bus_dff_sync_gated_clk . . . . . 68
bus_four_latch . . . . . 70
bus_two_dff . . . . . 72
bus_two_dff_phase . . . . . 74
combo_logic . . . . . 75
custom_sync . . . . . 77
dff_sync_gated_clk . . . . . 79
dmux . . . . . 81
fanin_different_clks . . . . . 82
four_latch . . . . . 84
memory_sync . . . . . 85
multi_bits . . . . . 87
multi_sync_mux_select . . . . . 89
no_sync . . . . . 90
pulse_sync . . . . . 92
reconvergence . . . . . 93
redundant . . . . . 95
shift_reg . . . . . 96
two_dff . . . . . 97
two_dff_phase . . . . . 99

**CHAPTER 4. CDC ANALYSIS 101**

Running 0-In View . . . . . 102
CDC Check Firings . . . . . 102
CDC Check Entries . . . . . 103
Verdi/Debussy Debug Environment Integration . . . . . 105
Analyzing Reports . . . . . 106
CDC Design Report . . . . . 106
Clock Group Summary . . . . . 106
User-Specified Clock Groups . . . . . 107

Inferred Clock Groups . . . . . 107
General Design Information . . . . . 108
Detailed Design Information . . . . . 108
Port Domain Information . . . . . 109
Clock Domain Crossing Report . . . . . 110
CDC Summary . . . . . 110
CDC Violations . . . . . 111
CDC Unknowns . . . . . 112
CDC Evaluations . . . . . 113
Memory Synchronization . . . . . 113
Blackbox Crossing . . . . . 114
Custom Synchronization Modules . . . . . 115
Asynchronous Reset Detection . . . . . 115
Transmitting Signals . . . . . 117
Hierarchical CDC Verification . . . . . 118
Verifying the Parent Module . . . . . 118
0in_cdc_hier_ctrl_*module*.v . . . . . 119
CDC Promotion . . . . . 122
Data Preparation . . . . . 122
CDC Checker Types . . . . . 123
Running Assertions in Simulation . . . . . 125
Running 0-In View . . . . . 125

**CHAPTER 5. CDC-FX 129**

Metastability Effects . . . . . 130
Metastable Registers . . . . . 130
Metastable Registers in Hardware . . . . . 130
Metastable Registers in Normal Simulation . . . . . 131
Modeling Metastable Registers . . . . . 132
Metastability Injection . . . . . 134
Common Methods . . . . . 134
Pseudorandom Delay Insertion . . . . . 134
Clock Jitter . . . . . 135
CDC-FX Metastability-injected Simulation . . . . . 135
cdc_fx Checker . . . . . 137
Corner Cases . . . . . 140
Statistics . . . . . 140

Specifying Metastability Windows . . . . . 141
Setup and Hold Periods . . . . . 141
Metastability Windows . . . . . 141
Negative Stop Value . . . . . 142
Metastability Windows Usage . . . . . 143
clks_aligned[1:0] . . . . . 143
set_cdc_fx_metastability_window . . . . . 145
Running cdc -fx . . . . . 147
set_cdc_fx_check . . . . . 148
0in_cdc_fx_sim_ctrl.v . . . . . 148
Running Metastability-Injected Simulation . . . . . 150
Design Style Support . . . . . 150
Suppressing Metastability Injection . . . . . 151
+0in_suppress_fx_name+*name* . . . . . *151*
+0in_disable_fx_force . . . . . 151
CDC-FX PLI Functions . . . . . 151
$0in_always_invert_metastable_signals () ; . . . . . 151
$0in_never_invert_metastable_signals () ; . . . . . 151
$0in_randomly_invert_metastable_signals () ; . . . . . 152
Analyzing Simulation Results . . . . . 153
Assertion Violations . . . . . 154
cdc_fx Checker . . . . . 155
Coverage . . . . . 156
cdc_fx Check Firings . . . . . 156
cdc_fx Checker Pins . . . . . 158
inject_metastability . . . . . 158
rx_q . . . . . 159

# INTRODUCTION

Welcome to the 0-In Functional Verification suite from Mentor Graphics. The 0-In Functional Verification suite is a collection of functional analysis tools and the CheckerWare® library of assertion checkers and monitors. The 0-In Functional Verification suite along with your standard HDL simulator and waveform analysis tools provides a complete and integrated functional verification environment for assertion-based verification of your complex HDL designs.

The 0-In Functional Verification suite has the following verification tools and libraries:

- ❏ Checklist tool for RTL coding checking and analysis.
- ❏ CDC Compiler for clock domain crossing analysis and CDC-FX metastability effects analysis.
- ❏ CheckerWare Compiler for running assertions in simulation.
- ❏ 0-In View graphical verification database browser.
- ❏ Formal Verification tools: Prove static formal property checker, Confirm static formal verification tool and Search dynamic formal verification tool.
- ❏ CheckerWare library of assertion checkers, CDC transfer protocol checkers and standard interface protocol monitors.

## ABOUT THIS MANUAL

This manual is for integrated circuit (IC) designers, IC verification engineers and IC design consultants. It assumes you are proficient with a hardware description language (HDL)—currently, 0-In Functional Verification products support Verilog. It also assumes you are familiar with the UNIX operating system, UNIX commands and the operation of graphical utilities.

## 0-IN FUNCTIONAL VERIFICATION MANUALS

The manual set for the 0-In Functional Verification suite has the following documents:

- ❏ *Tutorials & Examples* work book contains tutorials on using the 0-In tools.
- ❏ *Checklist User Guide* describes the operation of the Checklist coding checks and analysis system.
- ❏ *CDC Compiler User Guide* describes the operation of the CDC compiler and analysis system and the CDC-FX metastability analysis system.
- ❏ *Assertions in Simulation User Guide* describes the CheckerWare compiler and assertions in simulation.
- ❏ *Formal Verification User Guide* describes the operation of the 0-In formal verification tools: Prove, Confirm and Search.
- ❏ *CheckerWare Data Book* describes the CheckerWare library of assertion checkers.
- ❏ *CheckerWare Monitors* data book describes the CheckerWare library of interface assertion monitors.
- ❏ *0-In Functional Verification Suite Quick Reference* is a booklet of the syntaxes for the 0-In Functional Verification suite products.
- ❏ *0-In Functional Verification Suite Frequently Asked Questions* is a booklet containing answers to frequently asked questions and a glossary.
- ❏ The *0-In Functional Verification Suite Messages* is a booklet containing descriptions of error/warning messages.

# NOTATIONAL CONVENTIONS

The following textual conventions are used in this manual:

| | |
|---|---|
| *emphasis* | Italics in plain text are used for two purposes: (1) titles of manual chapters and appendixes, and (2) terminology used inside defining sentences. |
| *variable* | Italics in sans-serif text indicate a meta-variable. You must replace the meta-variable with a literal value when you use the associated statement. |
| literal | Regular sans-serif text indicates literal words used in syntax statements or in output. |
| <variable> | Angle brackets indicate a meta-variable. This notation is used in usage messages from the software. |

## Syntax Conventions

Syntax statements appear in sans-serif typeface as shown here. In syntax statements, words in italics are meta-variables. You must replace them with relevant literal values. Words in regular (non-italic) sans-serif type are literals. Type them as they appear.

Except for the following meta-characters, regular characters in syntax statements are literals. The following meta-characters have the given syntactical meanings. **You do not type these characters.**

| | |
|---|---|
| . . . | Ellipses indicate a repeatable entry. |
| [ ] | Square brackets indicate an optional entry. |
| { } | Braces indicate a required entry. |
| \| | Or-bars separate choices in [ ] and { } entries. |

The following examples demonstrate these syntax conventions.

| | |
|---|---|
| *signal*. . . | One or more occurrences of *signal*. |
| [ –var *req_signal* ] | Zero or one occurrences of –var *req_signal*. |
| –var *req_signal...* | –var followed by one or more occurrences of *req_signal*. |
| [ *std_option* ]... | Zero or more occurrences of *std_option*. |
| { *arg1 arg2* }... | One or more occurrences of *arg1 arg2*. |
| { *var1* \| *var2* \| *var3* } | One occurrence of *var1*, *var2* or *var3*. |
| { *constant* \| *val1 val2* } | One occurrence of *constant* or *val1 val2*. |
| [ *var1* \| *var2* \| *var3* ]... | Zero or more occurrences of any of *var1*, *var2*, and *var3*. |

## Directive Conventions

The following meta-variables in directive syntax statements represent the shown object types.

| | |
|---|---|
| *signal* | Single-bit register or wire. |
| *variable* | Expression that can change value at any time. |
| *constant* | Expression that evaluates to a statically constant value. |
| *“string”* | String enclosed in double-quotes. |

When specifying a directive from a directive syntax, you substitute for each meta-variable an HDL identifier, or an expression enclosed in parentheses, that evaluates to an object of the corresponding type.

# CHAPTER 1: CLOCK DOMAIN CROSSINGS

Most complex designs have more than one clock. For these designs, the logic clocked by each clock forms the *clock domain* for the clock. Logic entirely inside a clock domain can be verified with the same approach as that for a single-clock design.

A singular problem arises however, from signals that connect logic in different clock domains. Signals that cross clock domain "boundaries" must be properly synchronized and they must obey all relevant transfer protocols. The process of verifying these requirements is called clock domain crossing (CDC) analysis.

This chapter describes the 0-In method of verifying CDC signals, including the 0-In CDC compiler. This method combines netlist analysis, inference of design intent, assertions in an assertion library, simulation and formal verification for a complete CDC verification solution.

# 1 CLOCK DOMAIN CROSSINGS

CDC verification ensures all appropriate CDC signals have correct synchronization logic. But, CDC verification really addresses the larger question:

*Does my CDC synchronization logic prevent corruption across clock domains?*

Even for a design that has correct synchronizers on all signals, you must consider questions such as:

- ❏ What happens if the CDC signals are changing when the handshake signal indicates they are stable?
- ❏ Does the gray-code logic on a multiple bit bus using 2FF synchronization have a bug?
- ❏ Does the input to a data synchronizer change in two successive clock cycles of the receiving domain?

Problems such as these usually do not cause simulations to fail—instead, they are commonly manifested as intermittent post-silicon failures. To protect against these types of failures—and ensure CDC problems are addressed early in the design process—you can use the 0-In CDC verification methodology, which consists of a two-pronged approach:

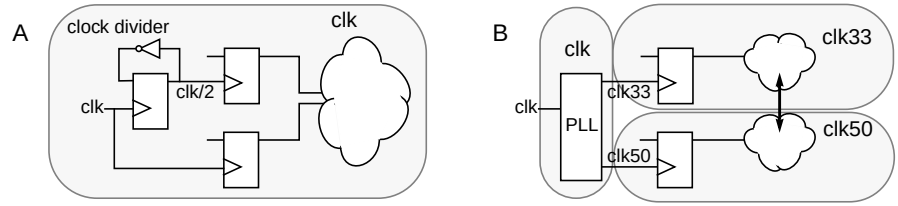
- ❏ 0-In CDC netlist analysis.
- ❏ Assertion-based verification with 0-In CDC protocol checkers

This section gives you an overview of these procedures—after the section defines the basic CDC terminology.

# CLOCK DOMAINS

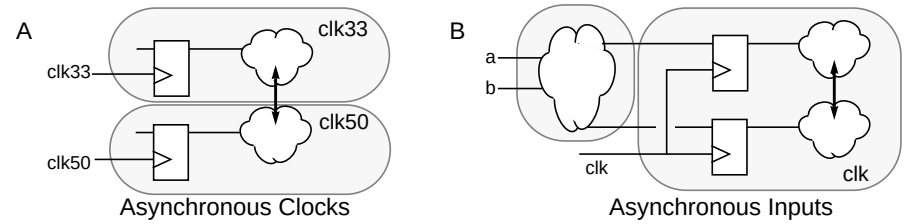
A *clock domain* is a section of a design that has a clock asynchronous to—or which has a variable phase relationship to—another clock in the design. For example, suppose one clock is derived from another clock through a clock divider. These two clocks have a constant phase relationship, so the two sections of the design that use these clocks are really part of the same clock domain (Figure 1A). However, suppose two clocks have frequencies 50 MHz and 33 MHz. These clocks' phase relationship changes over time, so they clock two different clock domains (Figure 1B).

**Figure 1: Clock domains and clock dividers**




If the primary inputs to a circuit include multiple clocks, these *asynchronous clocks* determine separate clock domains (Figure 2A). If the inputs to a circuit are asynchronous to the circuit itself, these *asynchronous inputs* are in separate clock domains (Figure 2B).

**Figure 2: Asynchronous clock domains**



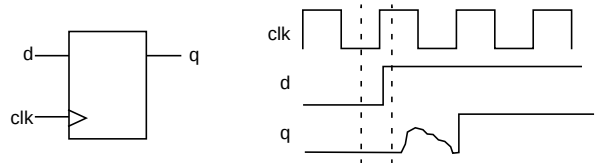

## Clock Groups

All the clocks that are part of the same clock domain constitute a *clock group*. Hence, clock groups partition all of the clocks in the design. The clock groups identify the various clock domains in the design.

# METASTABILITY

If the data input of a register changes within the setup and hold time of the register's clock edge, the register output value can become *metastable*. The electrical value is neither a logical 0 nor logical 1. Eventually, the output value will settle to a value of 0 or 1, but that value might not be the same as the input value (Figure 3). If downstream logic uses a metastable output value, the logic will likely produce an incorrect result—unless the logic is specifically designed to deal with the situation.

**Figure 3: Metastable flip-flop**




Traditional RTL simulation cannot detect many problems related to metastability because standard simulators do not model such fine-grained electrical effects. Gate-level timing simulations cannot verify all possible signal-to-clock relationships. So, potential setup/hold violations (metastability) can be missed along with the effects they have on the overall operation of the circuit. Static timing analysis is able to detect potential setup/hold violations, but only in designs with multiple clocks having fixed phase relationships. For designs with multiple asynchronous clocks, the potential for metastability will always exist. So, most metastability problems are detected in post-silicon verification as intermittent design problems.

The following mean-time-between-failure (MTBF) formula predicts how often metastability occurs:

$$MTBF = \frac{1}{f_{clk} \times f_{in} \times t_d}$$

$f_{clk}$ = clock frequency

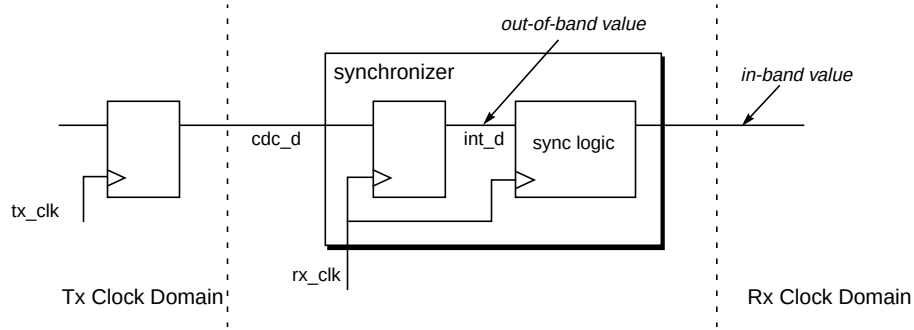
$f_{in}$ = asynchronous signal frequency

$t_d$ = duration of the setup and hold window

# SYNCHRONIZERS

Designers generally assume signals are *in-band*: they have a value of either logic 0 or logic 1. Metastable signals can have values that are neither 0 or 1, so they are considered *out-of-band* signals. Out-of-band signals have unexpected effects and propagate unpredictably. So to handle CDC signals, designers *fence in* potentially metastable logic to ensure logic beyond the fence only needs to handle in-band signals. The logic inside the fence is called a *synchronizer* (Figure 4).

**Figure 4: Synchronizer**




Metastability is manifested as variable delays in transitions of CDC signals (also called *CDC jitter*). Relative to normal simulation, transitions are randomly delayed or advanced. This affects *every* CDC signal. Even if a CDC signal or data bus has a synchronizer, the output of the synchronizer is subject to variable delays. Logic outside the fence in the receiving domain might not interpret receive data correctly in the presence of variable delays. Such intolerance of metastability effects can lead to functional errors in hardware, even when normal simulation cannot detect the problem.

Designers implement various types of synchronizers as appropriate for particular situations and design styles. Logic for each synchronizer type assumes a set of preconditions about the logic connecting the synchronizer and about the function of the circuit during simulation (Figure 5). Rules for the synchronizer's connections can be checked during compilation before simulation. *Transfer protocols* can only be checked as the circuit operates in simulation. A synchronizer type along with its connection rules and transfer protocol is called a *synchronization scheme*.

**Figure 5: Synchronization scheme example**

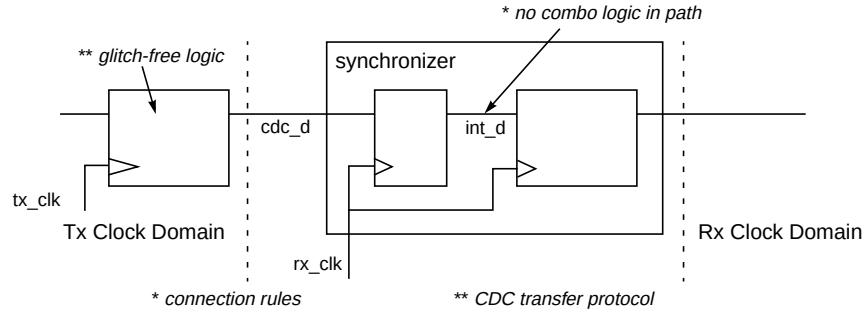
Synchronizer logic is based on two types of assumptions:

- Connection rules

  Assumptions that can be checked statically.

- CDC transfer protocol

  Assumptions that must be checked during simulation.




## STRUCTURED SYNCHRONIZERS

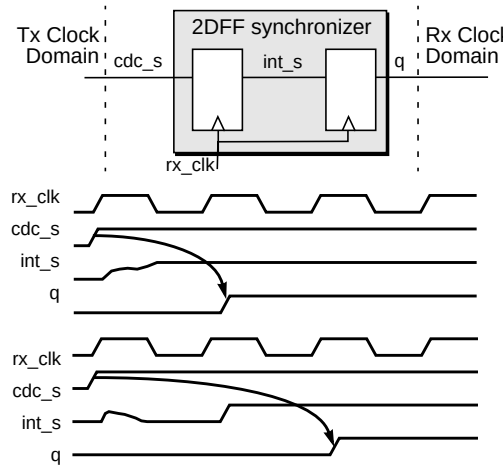
Most CDC implementations use one or more synchronizers from a set of popular, well-characterized synchronization schemes. These *structured synchronizers* must follow well-defined connection rules and obey specific transfer protocols. 0-In CDC identifies clock domains, CDC signals and structured synchronizers. CDC then verifies that the structured synchronizers follow their connection rules and promotes their transfer protocols to CheckerWare CDC protocol checkers for use with assertion-based simulation and formal verification.

### CDC Control Signal Synchronizers

A typical 2DFF 1-bit synchronizer is shown in Figure 6. In most technologies, if the first register (`R1`) becomes metastable, it almost always settles to 0/1 before the second register (`R2`) samples its output (`q1`). The 2DFF synchronizer is the most commonly used synchronizer for single-bit CDC connections such as individual control signals.

Other structured synchronizers are available, such as the 4-latch synchronizer (similar to 2DFF) and the pulse synchronizer (2DFF with a pulse).

**Figure 6: 2DFF synchronizer**




Connection Rules:

- Logic in cdc_s path must be glitch free
- No combinational logic is allowed in int_s path

CDC Transfer Protocol

- Transmit clock domain logic must hold cdc_s stable for at least:
  $period_{rx_clk} + t_{setup} + t_{hold} + t_{max_skew}$

## CDC Data Bus Synchronizers

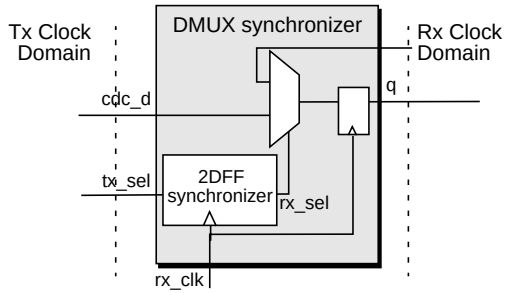
The 2DFF synchronizer is used for CDC control signals. It does not synchronize the bits of a data bus. Figure 7 shows some popular CDC *data bus synchronizers*.

- ❏ The DMUX synchronizer uses a select control signal from the transmit domain, synchronized by a 2DFF synchronizer to enable a MUX when the transmit data is available.
- ❏ The FIFO synchronizer uses a multiclock, multiaccess FIFO to queue up transmitted data.
- ❏ The multibit data synchronizer uses a 2DFF synchronizer for each bit, but only one bit at a time can change during any receive clock cycle.
- ❏ The custom synchronizer is a special-purpose synchronizer designed, specified and implemented by the user.

0-In CDC reports all structured synchronizers and promotes CheckerWare checkers to verify their CDC transfer protocols in simulation and formal verification.

**Figure 7: CDC data bus synchronizers**

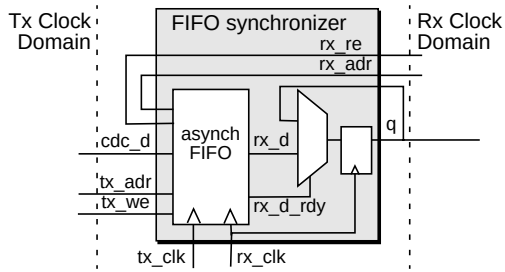
**DMUX Synchronizer**




CDC Transfer Protocol

- 2DFF synchronizer must obey CDC transfer protocol for tx_sel.
- Transmit clock domain logic must hold cdc_d stable while tx_sel or rx_sel are asserting.

**Asynchronous FIFO Synchronizer**

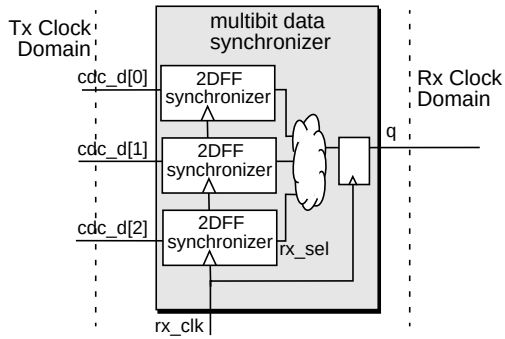



CDC Transfer Protocol

- FIFO must not overflow or underflow.
- Transitions of tx_adr must have Hamming distance of 1.
- Interval from write to read of any FIFO location must not be less than:

  $t_{setup} + t_{max_propagation_time}$

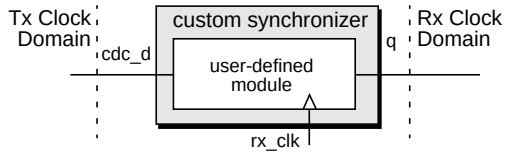
**Multibit Data Synchronizer**




CDC Transfer Protocol

- Transitions of cdc_d must have Hamming distance of 1.
- 2DFF synchronizers must obey CDC transfer protocol for each cdc_d bit.

**Custom Synchronizer**




CDC Transfer Protocol

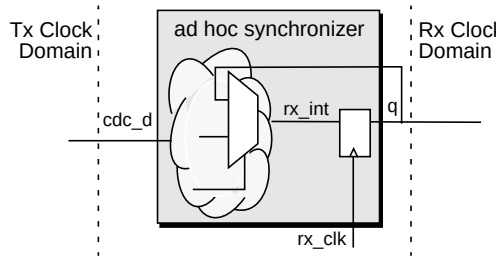
- User-defined protocol

## AD HOC SYNCHRONIZERS

Any clock domain crossing that does not have a structured synchronizer must be synchronized by custom logic or software. These *ad hoc synchronizers* prevent receive registers from sampling CDC signal values when they are changing, so the receive register outputs can never go metastable. For example, an ad hoc synchronizer might use custom logic to control its receive register's load enable signal. Or, software might control the loading of a circuit's configuration registers.

0-In CDC reports CDC signals without structured synchronizers and promotes appropriate CDC protocol checkers to verify ad hoc synchronization in simulation and formal verification (Figure 8).

**Figure 8: Example ad hoc synchronizer**




CDC Transfer Protocol

When a changing rx_int is sampled by rx_clk:

- rx_int must be stable from $t_{setup}$ before rx_clk edge to $t_{hold}$ after rx_clk edge.

## RATIOED SYNCHRONOUS CLOCKS

Transmit and receive clocks that are synchronized are called *ratioed synchronous clocks* (RSC). Implementations that use ratioed synchronous clocks do not necessarily require synchronizers. Metastability is avoided altogether because the circuit is implemented in a way that ensures data is never transmitted or received on a "wrong" clock edge.

For example, transmit and receive registers could be enabled by a single master enable signal (that allows only transmit-enable or receive-enable). Or, the receive enable can be triggered by a canonical sequential delay after the transmit signal.

RSC synchronization does need to obey a CDC transfer protocol. 0-In CDC checks each RSC crossing to ensure:

- ❏ Enables are correctly delayed.
- ❏ Enables are used correctly:
  - If the receive enable is deasserted, the path from the transmit register to the receive register must be cut off.
  - If the transmit enable is deasserted, the transmit register must reload the current data.

# RECONVERGENCE

*Reconvergence* is the process of using separately-propagated correlated information. The singular problem with reconvergence is: can you assure that all of the correlated information used at the destination is consistent with the information that is transmitted from the source? A primary example of reconvergence is information crossing clock domain boundaries that is recombined in the receiving logic.

Because they can become metastable, CDC signals are always subject to variable delays—even when a proper synchronizer is used. Variable delays can cause reconverging information to be inconsistent. One example is a data bus that is split and transmitted across different synchronizers. If the synchronized parts are recombined, the received value will not match the transmitted value in some cycles.

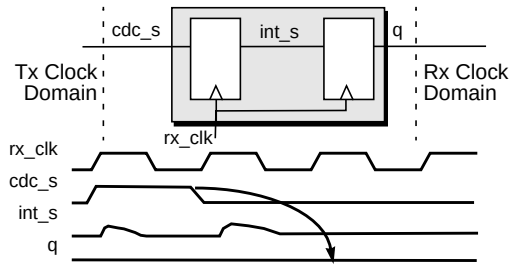
# LOCAL RECONVERGENCE

*Local reconvergence* is the process of propagating a single item of information and reconstituting the information at the destination. An example of local reconvergence of CDC signals is the multicycle reconvergence of a CDC control signal. An example of local reconvergence of CDC data buses is multibit reconvergence.

## Multicycle Reconvergence

*Multicycle reconvergence* is the process of transmitting sequential information to a destination. A shown in Figure 9, transmit logic must avoid transitioning data too quickly or the receive logic must tolerate loss of information in consecutive cycles.

**Figure 9: Multicycle reconvergence example**

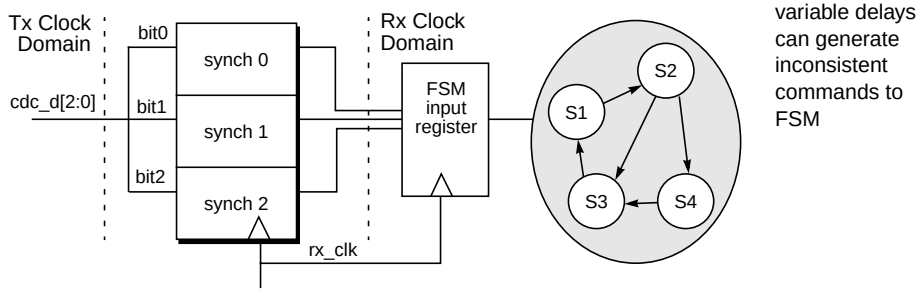



Variable delays can disturb correlations between consecutive transitions. In this example, the cdc_s pulse is not propagated to the receive logic.

### Multibit Reconvergence

*Multibit reconvergence* is the process of splitting a unit of data into pieces and recombining the data unit at the destination. Figure 10 shows an example of a data word whose individual bits are synchronized using parallel single-bit synchronizers (considered as a single compound synchronizer). In this case, the output word to an FSM can represent a command that is not consistent with the current state.

**Figure 10: Multibit reconvergence example**

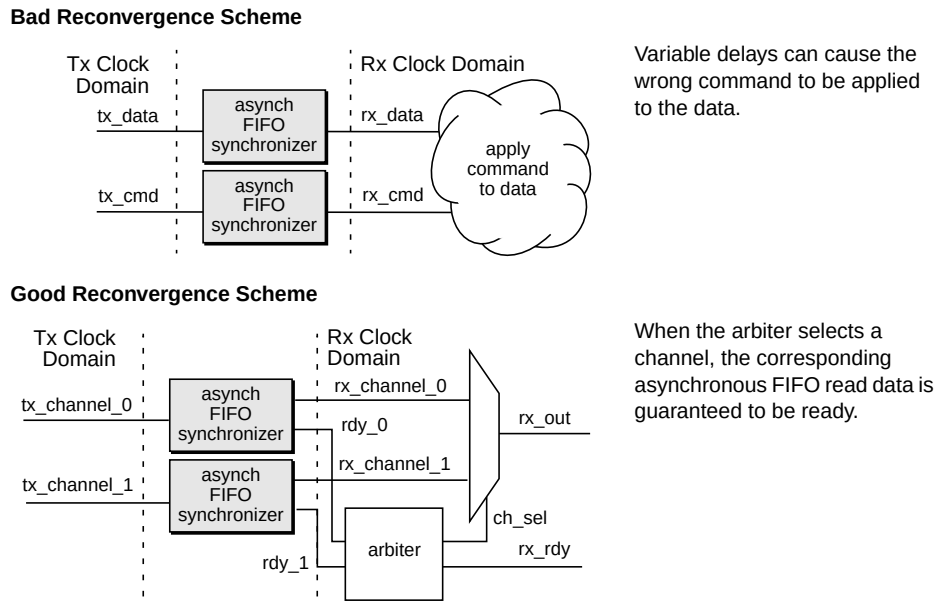



## GLOBAL RECONVERGENCE

*Global reconvergence* is the process of propagating multiple items of information and integrating the information at the destination. An example of global reconvergence is a set of data items transmitted across a clock domain boundary through different synchronizers that is combined in the receive clock domain. Another example is transmitting multiple items of information across a clock domain boundary at different times using the same synchronizer.

Schemes to implement global reconvergence can be bad or good. A bad scheme results in unrecoverable errors either in simulation (bad) or in the hardware implementation of the circuit (worse). A good scheme ensures the information is always consistent. Either variable delay data cannot be sampled in the receive domain or the receive domain logic can recover from variable delayed values (Figure 11).

**Figure 11: Global reconvergence examples**




## VERIFYING RECONVERGENCE

Global and local reconvergence problems in CDC circuits are avoided by designing the circuits using proper synchronizers and good global reconvergence schemes. Simulation alone is not sufficient to verify that a circuit's hardware implementation tolerates metastability effects and will operate correctly without global reconvergence issues. Normal simulation does not model metastability robustly and completely misses behavior that can manifest in the circuit's hardware implementation. A multifaceted approach to CDC verification is necessary for a high degree of confidence that a particular global reconvergence scheme is a good one:

- Static reconvergence analysis.

  0-In CDC creates a static reconvergence report showing reconvergence points organized by signature. This report can uncover reconvergence scenarios that might be overlooked.

- ❏ CDC transfer protocol checking.

  0-In CDC identifies structured synchronizers and promotes their CDC transfer protocols to CheckerWare checkers for use in simulation with assertions and formal verification.

- ❏ Metastability-injected verification.

  0-In CDC-FX identifies registers that can become metastable. For each such register, CDC generates the following directive:

  - `cdc_fx` checker directive for metastability-injected simulation

    Creates a checker with metastability injection and analysis logic. During simulation, the `cdc_fx` checker assumes control of the register's output and modifies the output to inject variable delays on bits that transition when transmit and receive clocks are almost aligned (i.e, some outputs are potentially metastable).

# NETLIST ANALYSIS

0-In CDC *netlist analysis* performs the following critical functions for CDC verification:

- Verifies synchronizers.

  Netlist analysis examines the design RTL and uses the user-specified and inferred clock groups to find the clock trees that determine the clock domains in the design. It then assigns each register to a clock domain. Netlist analysis identifies the CDC signals by finding registers that drive registers from other clock domains. Then, netlist analysis verifies that correct synchronizers are present on all CDC signals.

- Identifies CDC schemes.

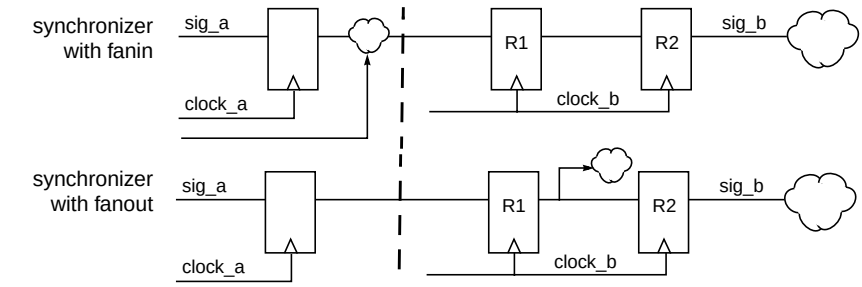
  Netlist analysis categorizes each CDC logic pattern as one of a set of predefined CDC schemes. For example, one scheme contains all single-bit CDC signals that are synchronized by two-register data synchronizers. Another scheme contains all multiple-bit CDC signals that are not synchronized.

  Some schemes are identified as violations. For example, the examples shown in Figure 12 have single-bit CDC signals with fanin or fanout within the synchronizers. You can redefine the statuses used for the different schemes; for example, you can make latch synchronization a violation.

- Promotes CDC protocol checkers.

  Netlist analysis generates a file of suggested CDC protocol checkers for all CDC signals based on their transmit clocks, transmit signal names, receive clocks, receive signal names, and their CDC scheme (protocol). You can then use any of the suggested checkers in simulation and formal verification to ensure correct functional behavior.

**Figure 12: Invalid synchronizers**



## ASSERTION-BASED VERIFICATION

An *assertion* is a user-provided specification of design intent. Assertions are assumptions about the legal behavior of the design. They can be checked during simulation with assertions. They can act as targets and constraints for formal verification. Assertion-based verification (ABV) is a functional verification methodology based on assertions about a design.

A design's CDC signals assume a set of CDC protocols. The 0-In CDC verification methodology combines the 0-In CDC verification IP and 0-In methods to give an effective means for verifying the design's CDC protocols.

To illustrate, consider a CDC signal synchronized by a two-register data synchronizer. This synchronization scheme has the following implied CDC protocol:

> *The transmit signal remains stable until the synchronizer's output register has sampled the previous value.*

You can check this protocol with the following assertion:

> *The transmit signal must not change for at least N consecutive transmit-clock cycles.*

*N* is a function of the transmit-clock and receive-clock frequencies. The above assertion can be checked using both simulation with assertions and formal verification.

If this assertion is violated, a possible counterexample exists in which metastability on the CDC signal can cause an incorrect result. In this case, the transmitted data will be missed by the receiver logic. In regular simulation—which is not subject to metastability unless it is a catastrophic violation of the protocol—the signal reaches its goal and the problem is not detected. However when using simulation with assertions, an assertion will fire. You can then examine the conditions under which the failure occurs and correct the design problem.

## SPECIFYING DESIGN INTENT

A *checker* is a conglomeration of assertions about the legal behavior of a particular design object or interface protocol. A *CDC protocol checker* is an interface protocol checker for a particular CDC synchronization scheme. Furthermore, a CDC checker's assertions completely characterize its associated synchronization scheme. If a design is not capable of violating any of the checker's assertions, then the design obeys the associated CDC synchronization scheme.

The CheckerWare assertion library includes the set of CDC checkers described in the CheckerWare Data Book. CheckerWare CDC checkers are available for the common CDC synchronization schemes.

0-In CDC netlist analysis identifies the synchronization schemes used for the CDC signals in the design. Based on the analysis, 0-In CDC "suggests" instances of CheckerWare CDC checkers that you can use to validate CDC behavior with assertion-based verification. However, netlist analysis cannot always infer completely the design intent—some additional information often must be supplied to codify the actual design intent.

User input is an important part of the CDC verification process. 0-In CDC netlist analysis reports the CDC signals and their categories. At this point, you might decide that certain detected CDC schemes are errors, correct them and rerun 0-In CDC.

After you determine that all reported CDC synchronization schemes are intended, you typically examine the list of suggested CDC checkers. You might consider some of these irrelevant and discard them. To instantiate the rest, you add any additional information that netlist analysis cannot infer (see Figure 13) and include the CDC checkers in simulation with assertions and formal verification sessions.

**Figure 13: CDC checkers example**

```
/* 0in cdc_sync
    –tx_var cpu_if1.pci_req –tx_clock cpu_clk –tx_reset 1'b0
    –tx_min 'P_cpu_clk_pci_clk_tx_min –areset 1'b0
    –name cdc_sync_0 */
/* 0in cdc_dsel
    –tx_data cpu_if1.fifo1.deq_addr –tx_data_select 1'b0
    –tx_min 'P_cpu_clk_pci_clk_tx_min
    –rx_data_select data_sel
    –tx_clock cpu_if1.fifo1.clk –tx_reset 1'b0
    –rx_clock mtr.clk –rx_reset 1'b0
    –areset 1'b0 –name cdc_dsel_0 */
```

## CDC CHECKERS IN SIMULATION

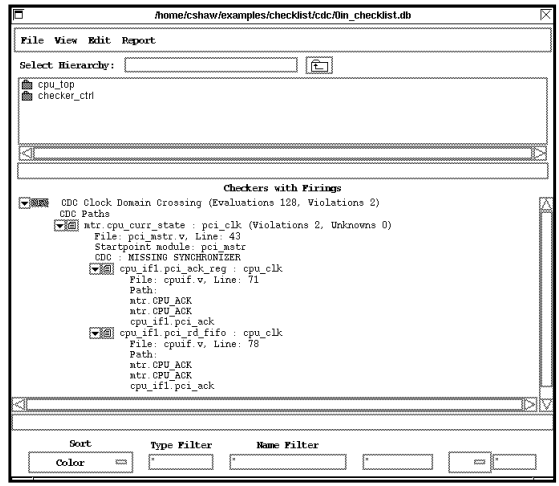
To simulate a design with CDC checkers, you use assertions in simulation and your standard HDL simulator. During assertions in simulation, the CDC checkers verify the functionality of the CDC protocols. A violation of a protocol assertion makes the associated CDC checker "fire"—whether or not the violation causes a functional error.

**Note:** Simulating the effects of metastability is unnecessary with this methodology.

To debug assertion firings, you can use the 0-In View assertion database browser and your standard waveform viewer or debug environment. From 0-In View, you can browse the data for individual assertion firings (Figure 14). Also from 0-In View, you can run your waveform viewer or debug environment with the assertion firing waveform from simulation automatically loaded. A major advantage of using CDC checkers is that very little additional effort is required. 0-In assertions in simulation work seamlessly with existing simulation environments. 0-In View supports quick and easy debugging of assertion firings.

In addition to performing functional checks, CDC checkers collect simulation statistics. By examining CDC statistics, you can readily determine how thoroughly simulation tests cover corner cases. You can focus further verification with the 0-In formal tools around unstressed corner cases.

**Figure 14: 0-In View showing CDC violations**



## FORMAL VERIFICATION WITH CDC CHECKERS

CDC checkers are compatible with the 0-In formal tools—Prove, Search and Confirm. Only a small incremental effort is needed as the 0-In formal tools use the same configuration files and assertions as assertions in simulation. Formal verification has a singular advantage over assertions in simulation: formal verification automatically analyzes complex corner cases that are difficult to simulate.

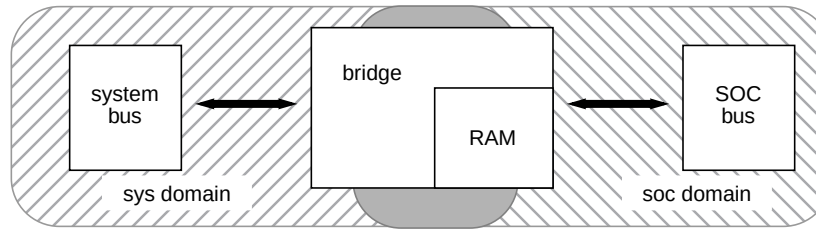
A design's individual assertions—including CDC checker assertions—are called *formal targets* when they are used a goals the formal tools try to prove or disprove. Other assertions can be used as guides for the formal tools to ensure they do not attempt to test illegal behavior. These assertions are called *formal constraints*. The CDC checkers' assertions are always used as targets, not constraints.

Prove is a theorem prover that tries to prove targets cannot be violated. Targets with inconclusive results are passed to Search and Confirm to attempt to disprove them. These tools disprove a target assertion by finding a counterexample to the assertion. A *counterexample* is a legal stimulus sequence applied to the design that eventually makes the assertion fail—that is, fire. Counterexamples are also called assertion firings.

## DESIGN EXAMPLE

To illustrate the 0-In CDC verification methodology, consider the multiclock design shown in Figure 15. This design serves as a bridge between an external system interface and an internal proprietary SoC bus. It supports both master and slave operation modes and allows transactions to originate on both the external system interface and the internal bus. The design is implemented in about 75K gates

**Figure 15: Bus bridge design example**



The 0-In CDC verification methodology for the bus bridge design involved the following steps:

1. **Run 0-In CDC.**

   0-In CDC revealed the following about the design:

   - 2 clock domains
   - 276 CDC signals
   - 35 errors
     – 9 missing synchronizers on single-bit signals
     – 26 synchronizers with combinational logic in front of receiving registers

2. **Fix problems and rerun 0-In CDC.**

   Iterating through cycles of running CDC and correcting the HDL code, each error was either eliminated or marked for CDC to ignore.

3. **Examine the instances of suggested CDC checkers.**

   CDC generated a control file with instances of all suggested CDC checkers. The user edited this file down to 16 CDC checkers that verify critical logic. Data that CDC could not infer was added manually by the user.

4. **Run assertions in simulation with the CDC checkers.**

   The 16 CDC checkers were included with existing simulation tests that use assertions in simulation. Simulation did not find any assertion violations. However, the statistics and coverage information collected by the checkers indicated that the simulations did not cover several significant corner cases. Additional simulation tests or formal verification were needed to analyze these corner cases.

5. **Run Prove, targeting the CDC checkers' assertions.**

   Prove performed formal analysis on the design looking for ways to prove the targeted assertions. As a result, one CDC checker was proven valid. Under all potential conditions, none of the checker's assertions could possibly fire.

6. **Run Search, targeting the CDC checkers' assertions.**

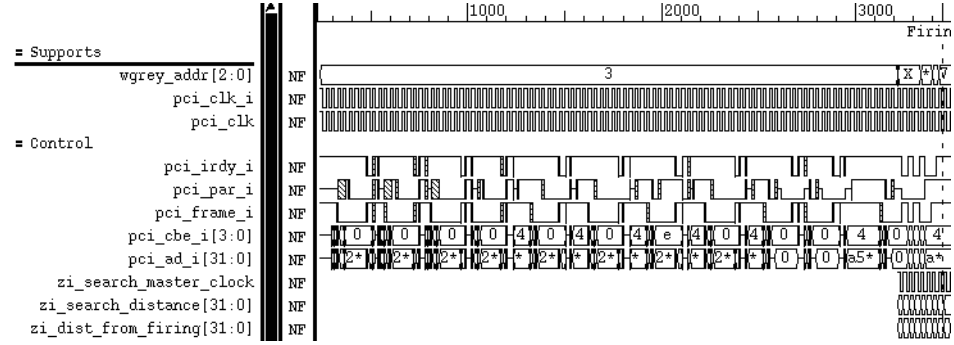
   Since testbenches already exist for the design, the user ran search. Search is designed to perform formal analysis on designs using many starting points (taken from the simulations) taken to shallow depths. Search analysis detected two counterexamples that fired assertions in two CDC checkers.

**7. Run 0-In View and analyze checker firings.**

The user ran 0-In View to analyze the checker firings. The user learned that both firings were caused by a specific sequence of commands and data transactions on the external bus, followed by the external bus becoming unavailable at the same time as the end-of-frame for the ongoing transaction.

This specific sequence was never encountered in the simulation tests and writing a test to create this behavior would have been difficult. Search produced the violations by starting from a simulated state and using a trace 11 cycles long which changed 4 separate inputs from their simulated values.

**Figure 16: CDC protocol violation found by Search**



# 1 CLOCK DOMAIN CROSSINGS

# CHAPTER 2: CDC COMPILER

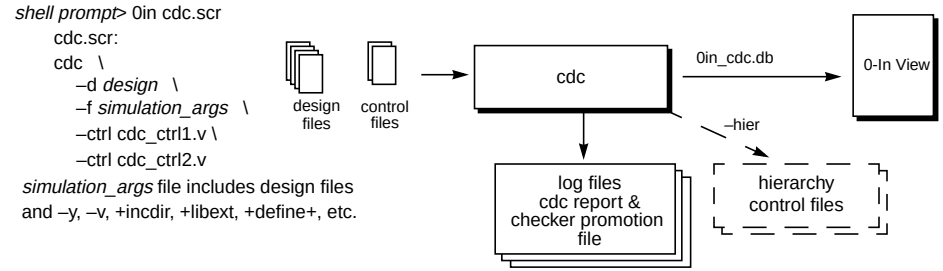
CDC analysis checks the design, infers clock trees, identifies signals that cross clock domain boundaries and detects a variety of synchronization patterns. The analysis also suggests CDC checkers for possible inclusion in the design instrumentation.

# RUNNING THE CDC COMPILER

To check a target design with the CDC compiler you run the cdc utility—cdc is one of the 0-In shell commands. The standard arguments you use to simulate the target design also can be used as arguments to the cdc command. You identify the design to analyze with a –d *target_design* argument and specify control files with –ctrl *control_file* arguments.

Figure 17 shows a sample 0-In shell script called cdc.scr that runs the cdc command. The command generates a 0-In View database (0in_cdc.db), the CDC design report (0in_cdc_design.rpt) and the CDC report (0in_cdc.rpt). The 0-In shell also outputs the 0-In shell log file (0in.log) and the detailed 0-In shell log file (0in_detail.log).

**Figure 17: Running cdc**




The following example shows a cdc invocation using the *script_file* option of the 0in shell.

```
shell prompt> more verilog_run.f
pci_mstr.v pci_slv.v tb_bus_to_pci.v
shell prompt> more cdc.scr
cdc –f verilog_run.f –d pci_slv
shell prompt> 0in cdc.scr
```

You also can specify the cdc command directly to the 0in shell as shown in the following example:

```
shell prompt> 0in –cmd cdc –f verilog_run.f –d pci_slv
```

## RECOMMENDED METHODOLOGY

The following steps outline the recommended methodology for performing CDC analysis:

1. Run cdc.

   *shell prompt*> 0in cdc.scr

2. Examine the report files.

   Ensure clocks are traced correctly. If they are not, use set_cdc_clock (page 50) and set_constant (page 52) as needed and rerun cdc.

3. Add any other directives (see "Adjusting CDC Analysis" on page 49).

4. Rerun cdc, analyze results and repeat as required.

## CDC COMMAND INVOCATION

Table 1 shows the syntax for the cdc command.

*Important!* You cannot include UNIX environment variables in a **0in** shell script or within an interactive **0in** shell session. You can include UNIX environment variables in invocations from the **system** shell prompt or from a **system** shell script.

**Table 1: cdc command syntax**

```
cdc {HDL_file |–f filelist }... [ –d design ] \
    [ –ctrl control_file ]... [ –ctrl_list control_file_list...[–]][ –wd working_directory ] \
    [ –oy search_path ] [ –ignore_translate_off ] [ +incdir{+include_dir}... ] \
    [ –y library_dir ]... [ –v library_file ]... [ –qy library_dir ]... [ –qv library_file ]... \
    [ +skip_modules{+module }...] [ +skip_syscalls{+systemcall }...] \
    [ –skip_protected_code ] [ –skip_protected_modules ] \
    [ +libext{+extension }... ] [ +define{+macro[=value}... ] \
    [ +nowarn{+messageID}...] [ +error{+messageID}...] [ –cr report_file] \
    [ –effort unlimited ] [ –v2k ] [ –hier ] [ –fx ] [ +other_arg ]...
```

| | |
|---|---|
| *HDL file* | Source HDL files. |
| –f *filelist* | File containing additional arguments or source filenames. |
| –d *design* | HDL module to be verified. Default: all top-level modules. |
| –ctrl *control_file* | Control file. Can appear multiple times. |
| –ctrl_list *control_filelist...* [–] | Checker control file list. The dash ( – ) is used to terminate a list of file names, if needed to separate the names from HDL filenames. |

| | |
|---|---|
| –wd *working_directory* | 0in working directory. Default: same as the 0in shell working directory. |
| –oy *search_path* | Translate off override search path. |
| –ignore_translate_off | Includes logic in translate_off/translate_on blocks. |
| +incdir+ *include_dir* | Input include directory. |
| –v *library_file* | Library file. |
| –y *library_dir* | Directory containing library files. |
| –qv *library_file* | Library file to skip. |
| –qy *library_dir* | Directory containing library files to skip. |
| +skip_modules {+ *module* }... | List of modules to skip. For example, you can use this option to skip a testbench module containing behavioral Verilog code. |
| +skip_syscalls {+ *systemcall* }... | List of system calls to skip. |
| –skip_protected_code | Skips encrypted code. |
| –skip_protected_modules | Skips encrypted modules. |
| +libext[+ *extension* ]... | File extension(s) to use when searching for library files. For example, +libext+.v |
| +define+ *macro*[ = *value* ] | Text macro specification. |
| +nowarn{+ *messageID* }... | Turns off reporting of the specified warning messages. Message ID is a category and number (e.g., parser-47). Default: all warning messages reported. |
| +error{+ *messageID* }... | Turns the specified warning messages into errors. Message ID is a category and number (e.g., parser-47). |
| –cr *report_file* | Name to give the CDC report. Default: 0in_cdc.rpt. |
| –effort unlimited | Uses formal analysis (with increased memory consumption and run time) to improve results. Caution: some large designs (for example those with large case statements) can run out of memory or have long run times. |
| –v2k | Allows analysis of V2K constructs. |
| –hier | Generates CDC hierarchy control files. |
| –fx | Generates 0in_cdc_fx_sim_ctrl.v control file with directives for promoted cdc_fx checkers. |
| + *other_arg*... | Other + arguments are ignored. |

## Working Directory

The cdc command uses a 0in working directory for storing internal data and for saving the checker logic. If a 0in working directory already exists, cdc assumes it is a valid 0in directory and has data from a previous cdc run.

The cdc working directory overrides any setting used for the 0-In shell.

### Specifying a Working Directory

By default, the 0in working directory is 0in_cache in the current run directory. You can specify the 0in working directory using the –wd option. For example:

```
–wd ../test/0in_cache
```

## cdc Input Files

When running cdc you need the following input data (see Figure 18 and Table 2):

- ❏ HDL source files—design files, include files, and library files—defining the target design.
- ❏ Optional checker control files containing global directives.

**Figure 18: Input to cdc**

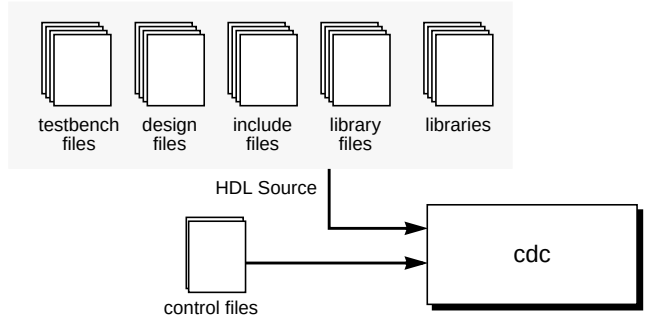



**Table 2: cdc input files**

| | |
|---|---|
| design files | HDL files describing the target design. |
| control files | Contain global directives. |

## Target Design

A *target design* is a high-level module that you want to verify. For the CDC compiler, the target design is the design for which you want to run static CDC analysis.

By default, the target design is the top-level module (or collection of modules) found in the design filelist. You can specify a particular module with the –d *design* option to the cdc command.

*Important!* The –d option lets cdc focus on the module you are interested in. Using this option is preferred as this option allows the tool to skip unrelated modules in the testbench.

## Control Files

In addition to the HDL source files, the input data to the cdc command can include control files. Control files contain global directives used to control the operation of the CDC compiler, assertions in simulation and the formal tools. The relevant global directives for the CDC compiler are as follows:

- set_cdc_clock — associates the specified clock signals as a single clock domain and specifies their properties.
- set_cdc_port_domain — associates specified ports to a clock domain.
- set_cdc_synchronizer — specifies a non-standard synchronizer type.
- set_cdc_signal — changes the classification of selected clock domain crossings.
- set_cdc_report — changes CDC reporting and promotion.

The following global directives apply to both the CDC compiler and other 0-In tools:

- set_constant — ties the specified signals to a given constant value.
- set_reset — identifies a signal as a reset.
- exclude_hier — excludes a module and its hierarchy from verification.
- set_black_box — sets the specified modules as black boxes.

To specify the control file for the cdc command, use the –ctrl option. For example:

```
–ctrl cdc_ctrl.v
```

Figure 19 shows an example of a control file for CDC.

**Figure 19: Sample control file for cdc**

```
//  ctrl_cdc.v: Sample Control File (CDC compiler)
//
module ctrl_cdc;
    // 0in set_cdc_clock pci_clk arb_clk –group PCI
    // 0in set_cdc_clock cpu_clk –group CPU
    // 0in set_constant cpu.test_en 1'b1
    // 0in set_cdc_report –scheme reconvergence –severity off
endmodule
```

## cdc Output Files

As the cdc command analyzes the design, it generates a series of status messages. These messages are saved in a detailed log file. In addition, errors and status messages are printed to STDOUT and are logged in the 0in.log file. Fatal errors cause cdc to terminate immediately.

The cdc command generates both concise (0in.log) and verbose (0in_detail.log) log files (Figure 20 and Table 3). The results of CDC analysis are shown in the CDC report (0in_cdc.rpt). The cdc command also creates a 0-In View database file (0in_cdc.db) showing check errors and statistics.

**Figure 20: Output from cdc**

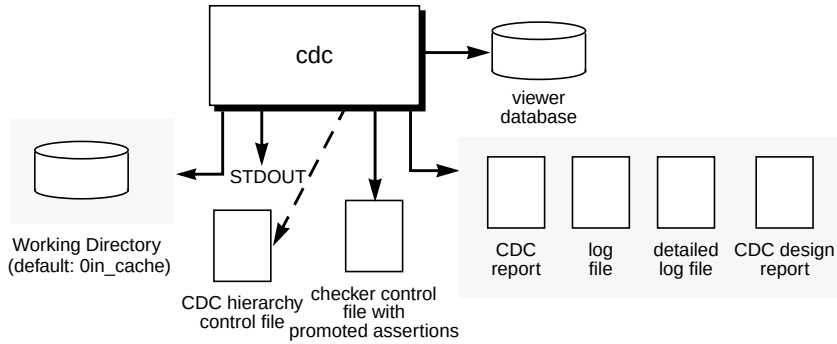


**Table 3: cdc output files**

| | |
|---|---|
| 0in.log | Summary log file for the session. This is a copy of the standard output. |
| 0in_detail.log | Detailed log file for the CDC run. |
| 0in_cdc.db | 0-In View database showing CDC results and statistics. |
| 0in_cdc_design.rpt | CDC design report.. |
| 0in_cdc.rpt | Clock domain crossing report. |
| 0in_cdc_ctrl.v | Clock domain crossing checker control file containing suggested CDC protocol checker directives for signals crossing clock domains (for use with simulation and the formal tools) |
| 0in_cdc_hier_ctrl_*module*.v | CDC hierarchy control file for *module*. Created using the –hier option to cdc. |

# CDC SCHEMES

The CDC compiler analyzes the design netlist and identifies logic involving CDC signals that matches various pattern templates. Each occurence of logic that matches a template is called a *CDC scheme*. For example, you might have a 1-bit CDC signal sig from clock domain A being synchronized by two in-phase D flip-flops in clock domain B. The CDC compiler will identify this particular clock domain crossing and its synchronization logic.

Classes of CDC schemes with the same functionality are called *CDC scheme types* (to distinguish them from individual CDC schemes). For example, the above CDC path with its synchronization logic is a CDC scheme of the two_dff type.

*Chapter 3, CDC Scheme Data Sheets* has the detailed data sheets for the CDC schemes.

## ATTRIBUTES OF CDC SCHEMES

Each CDC scheme detected by the CDC compiler has two attributes: severity and promotion. By default, the severity and promotion attributes are taken from the default attributes of the CDC scheme type. You can adjust these attributes for individual CDC schemes, groups of schemes and for the entire set of CDC schemes of a particular scheme type, using the set_cdc_report directive (see page 59).

Part of the initial CDC compiler runs typically is spent adjusting the attributes of CDC schemes to fine-tune the CDC reporting for the target design characteristics. For example, a combo_logic scheme in one part of the design might be acceptable, but it might be a serious concern in another part.

### Severity

Severity is an indicator of the seriousness of the particular logic pattern (CDC scheme). It also reflects the action the user is expected to take.

#### Violation

A violation is considered a must-fix problem. For example, an unsynchronized CDC signal from clock domain A to clock domain B might be a concern, so you might want to assign severity violation to all of these CDC schemes.

Unknown

An unknown is considered a valid static scheme, but it has an associated CDC transfer protocol that must be checked when the design is simulated. Most unknowns have designated CDC protocol checkers that the CDC compiler configures automatically. All or any combination of these chekcers can be included in assertions in simulation sessions and in formal verification (as “promoted checkers”)

Evaluation

An evaluation is considered a valid CDC scheme that does not require CDC transfer protocol checking. CDC data for evaluations are, however, included in the 0-In View database.

Reporting Off

If a CDC scheme is not of interest, you can indicate it should not be reported. For example, test logic or configuration data might result in a combo_logic scheme being reported. But, the combinational logic can be considered constant during regular operation. In this case, you might want to turn the reporting of this particular CDC scheme off.

## Promotion

Whenever possible, CDC creates and configures a CDC transfer protocol checker for a scheme. Most synchronizers have corresponding protocol checkers. The process of identifying a CDC scheme and creating its protocol checker is called *checker promotion*. Here, a CDC scheme is deemed OK from a static perspective. The compiler does not have enough information to determine whether or not a synchronization problem will occur. The answer depends on the operating environment of the design.

Promoted checkers can be used during assertions in simulation to monitor CDC activity and verify proper synchronization of CDC signals. They are also used in static and dynamic formal verification.

You can use the set_cdc_report directive to turn off CDC checker promotion for specific CDC schemes.

# CDC SYNCHRONIZATION SCHEMES

Most CDC scheme types refer to synchronization logic. Some schemes apply to single-bit CDC signal synchronization; some apply to multiple-bit data bus synchronization; and some apply to both.

## Signal Synchronization

Signal synchronization schemes identify single-bit CDC signals with various synchronizers. For example, a `two_dff` scheme uses two in-phase D flip-flops clocked in the receive domain as a synchronizer (Figure 21).

**Figure 21: two_dff signal synchronization scheme**

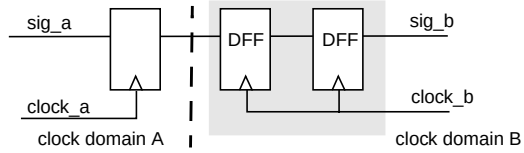



Table 4 shows the signal synchronization scheme types.

**Table 4: Signal synchronization schemes**

| Scheme Type | Synchronizer | Message | Default Severity |
|---|---|---|---|
| two_dff | Two or more in-phase flip-flops. | TWO DFF SYNCHRONIZER | evaluation |
| two_dff_phase | Two out-of-phase flip-flops. | TWO DFF SYNCHRONIZER OPPOSITE POLARITY | violation |
| four_latch | Four or more cascaded latches. | FOUR LATCH SYNCHRONIZER | evaluation |
| pulse_sync | Pulse. | PULSE SYNC | evaluation |
| shift_reg | Shift register. | SHIFT REG | evaluation |
| dff_sync_gated_clk | Two flip-flops with gated clock. | DFF GATED SYNC | violation |
| async_reset | Asynchronous reset scheme. | ASYNC RESET | evaluation |
| async_reset_no_sync | —none— | ASYNC RESET NO SYNC | violation |
| no_sync | —none— | MISSING SYNCHRONIZER | violation |

## Data Synchronization

Data synchronization schemes identify multiple-bit CDC data buses with various synchronizers. For example, a `bus_two_dff` scheme uses two in-phase D flip-flops clocked in the receive domain as a synchronizer (Figure 22). Such a synchronizer is typically used for single-bit signals. When used to synchronize data buses this synchronization scheme should only be used to synchronize data buses that are grey coded (i.e., at most one bit changes per clock cycle).

**Figure 22: bus_two_dff signal synchronization scheme**

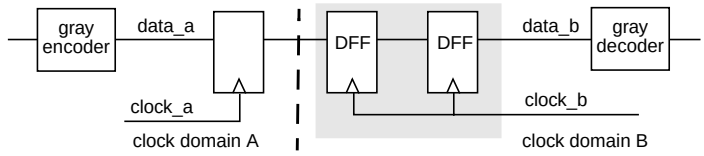



Table 5 shows the data synchronization scheme types.

**Table 5: Data synchronization schemes**

| Scheme Type | Synchronizer | Message | Default Severity |
|---|---|---|---|
| bus_two_dff | Two or more in-phase flip-flops. | BUS SYNC | unknown |
| bus_two_dff_phase | Two out-of-phase flip-flops. | BUS SYNC | unknown |
| bus_four_latch | Four or more cascaded latches. | BUS SYNC | unknown |
| bus_dff_sync_gated_clk | Two flip-flops with gated clock. | BUS DFF GATED SYNC | violation |
| multi_bits | —none— | MULTIPLE BITS | unknown |

## Signal or Data Synchronization

Signal/data synchronization schemes identify CDC signals and data buses with various synchronizers. For example, a `dmux` scheme uses two in-phase D flip-flops clocked in the receive domain to synchronize the select signal to a MUX that selects the CDC data (Figure 23). The CDC data can be either a single-bit signal or a multiple-bit data bus.

**Figure 23: dmux signal/data synchronization scheme**

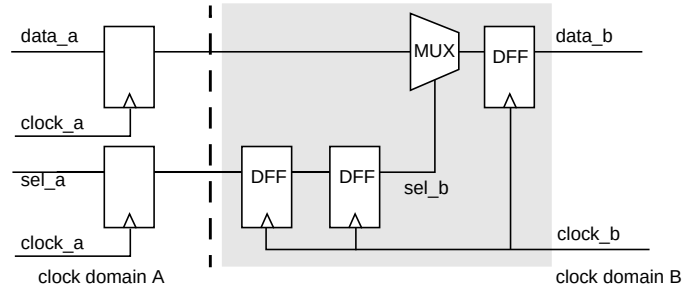



Table 6 shows the signal/data synchronization scheme types.

**Table 6: Signal and data synchronization schemes**

| Scheme Type | Synchronizer | Message | Default Severity |
|---|---|---|---|
| dmux | MUX enable. | DMUX | unknown |
| multi_sync_mux_select | Multiple MUXs. | MULTIPLE SYNCHRONIZERS AT DMUX | unknown |
| memory_sync | 2D array. | MEMORY SYNC | evaluation |
| custom_sync | Custom. | CUSTOM | evaluation |

## SCHEMES OF POTENTIAL CDC PROBLEMS

In addition to identifying CDC synchronization schemes, the CDC compiler identifies CDC schemes that exhibit the potential for error. For example, combinational logic in a CDC path could cause a valid synchronizer to malfunction (Figure 24).

**Figure 24: combo_logic synchronization scheme**

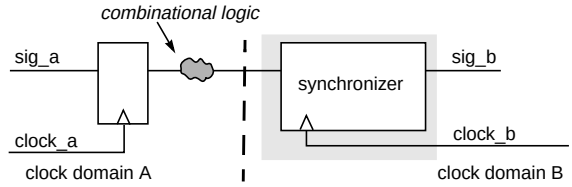



Table 7 shows the potential CDC problem scheme types.

**Table 7: Potential CDC problem schemes**

| Scheme Type | Problem | Message | Default Severity |
|---|---|---|---|
| combo_logic | Combinational logic on a synchronization path. | COMBO LOGIC | violation |
| reconvergence | Reconvergent CDC signals. | RECONVERGENCE | violation |
| redundant | CDC signal drives multiple synchronizers. | REDUNDANT | violation |
| fanin_different_clks | Fanin logic from multiple clock domains. | FANIN LOGIC FROM DIFFERENT CLOCKS | violation |
| blackbox | Black box in fan-out of a synchronized signal. | BLACKBOX | evaluation |

# ADJUSTING CDC ANALYSIS

CDC analysis makes certain assumptions about the design. After running the CDC compiler, the results from CDC design analysis using the default assumptions are reported in the CDC design report (`0in_cdc_design.rpt`). Examine these reports to see if the default assumptions are correct. If they are not, use the information presented in this section to tailor the assumptions to match the style used for your particular design.

To specify global directives, add them to a checker control file and specify the `–ctrl` *control_file* option to the CDC compiler. You also can add the CDC global directives to the HDL source modules.

## SPECIFYING CLOCKS AND CLOCK GROUPS

CDC analysis determines the boundaries between clock domains. To do this, it infers unspecified clocks and performs clock tree analysis to determine the clock domains. By default, each primary clock port that drives synchronous logic in the design defines a separate clock domain. The CDC compiler can also create a clock domain internal to the design on the output of clock gating or muxing logic (i.e., where it cannot determine the domain of the resulting clock). When internal clocks are created (such as when two or more clocks feed into the muxing logic) the equation used to derive this clock also is reported.

After CDC analysis, the CDC compiler reports information about clocks and clock domains in the CDC design report (`0in_cdc_design.v`). You can use the information in this report to help you set port constants, resolve the clocks in the equation(shown in figure xxx) and create appropriate clock groups.

### Clocks

CDC analysis can infer clocks as described above. However, you can also identify specific clocks and specify their characteristics. This methodology is recommended because:

- Clock characteristics are used to calculate parameter values used to configure directives for promoted checkers. (The clock period is used to calculate this information.)

- Clock characteristics must be specified for formal analysis. (Active clock edge and waveform information is used with the period to configure formal analysis.) Identifying these characteristics for CDC simplifies clock specification for formal verification.

## Clock Groups

By default, CDC analysis aggressively partitions the design logic into clock domains. However for most designs, logic clocked by multiple clocks often is considered part of the same clock domain. To get CDC analysis to treat various clocks as clocking logic in the same clock domain, you can create a clock group comprised of the clocks. For such a user-defined clock group, you can (but are not required to) name the clock group. Naming user-defined clock groups clarifies the reporting of clock paths in reports and the viewer databases.

## set_cdc_clock

The set_cdc_clock global directives specify clocks with their characteristics and redefine the clock domains. This directive has the following syntax:

```
set_cdc_clock clock_signal...
    [-posedge] [-negedge] [-waveform value value]
    [-period value] [-group group] [-module module]
```

You must specify one or more clocks. Each *clock_signal* can be a primary port, a hierarchical path to a clock signal or, if you specify –module, a local clock in that *module*. When you specify –module, corresponding clocks in all instances of *module* are identified as belonging to the same clock group and therefore there clocked logic in part of the same clock domain.

No two set_cdc_clock directives can specify the same clock.

The set_cdc_clock directive has the following options:

- [–posedge] [–negedge] [–waveform *value value*]

  Arguments that characterize the specified clocks for formal analysis. They are the same as those used for set_clock. These options are not used by CDC analysis.

- [–period *value*]

  Clock period in time units. This value is used to calculate directive parameters for promoted checkers.

- [–group *group*]

  Name to give the clock group. If no –group name is provided, the clock signals determine a single unnamed user-specified clock domain. You can have multiple set_cdc_clock directives that specify the same –group name. All specified clocks are part of the same clock domain corresponding to the group. You can use this capability to create a clock domain for clocks with different characteristics (in particular, for clocks with differing frequencies).

The following example control file module creates two clock domains:

```
module cdc_ctrl;
    // 0in set_cdc_clock U0.clk U1.clk –period 100 –group A
    // 0in set_cdc_clock U0.clk2 U1.clk2 –period 50 –group A
endmodule
```

## ASSIGNING PORTS TO CLOCK DOMAINS

By default, each primary port is assigned to the same clock domain as:

- A register driven by the port (for an input port), or
- The port that drives the register (for an output port).

The default clock domains assigned to the ports are reported in the Port Domain table of the CDC design report (0in_cdc_design.rpt).

### set_cdc_port_domain

The set_cdc_port_domain global directive indicates one or more specified ports belong to a given clock domain—or indicates they are asynchronous to all identified clock domains. Use this directive to override the inferred clock domain assignments of ports. This directive has the following syntax:

```
set_cdc_port_domain
        {port... | {–input | –output | –inout}[port...]}
        {–clock clock_signal | –async } [–module module]
```

You must specify one or both of the following arguments:

- *port...*

  One or more ports (wildcards are allowed).

- –input, –output or –inout

  A collection of ports: all input ports, all output ports or all inout ports.

You also must specify one of the following arguments:

- –clock *clock_signal*

  Identifies the clock domain. You can specify a top clock name or a clock signal local to the specified modules.

- –async

  Marks the ports as asynchronous to all clock domains.

The directive applies to ports on all instances of the module specified by *module*. Wildcards are allowed in –module, in which case the directive applies to all instances of the matching modules. If –module is omitted, the directive applies to instances of the current module (when the directive is placed in the HDL for the module) or the –d module (when the directive is placed in a control file).

Each set_cdc_port_domain directive with a –clock argument assigns all matching ports to the same clock domain (i.e., the directive does not assign to multiple clock domains).

For example:

```
module cdc_ctrl;
    // 0in set_cdc_port_domain a b c –clock U1.clk_A
    // 0in set_cdc_port_domain d –clock U2.clk_B
endmodule
```

## SETTING PORT VALUES TO CONSTANTS

CDC analysis assumes that ports in the design can change—potentially leading to additional CDC violations. Many designs have inputs (for example configuration ports and test pins) that CDC analysis should assume to be constant.

### set_constant

The set_constant global directive sets ports to constant values. This directive has the following syntax:

```
set_constant port constant
```

CDC analysis assumes the specified *port* is held at the specified *constant* value. For example:

```
module cdc_ctrl;
    // 0in set_constant U0.rst 1'b1
endmodule
```

## SPECIFYING NON-STANDARD SYNCHRONIZER TYPES

CDC analysis identifies a standard set of synchronizers, however some designs can use specialized non-standard synchronizers. Schemes that use these synchronizers provide proper synchronization, but they still need transfer protocol checking. Also, the design must use these specialized synchronizers only where appropriate.

You can configure the CDC compiler to recognize, support and verify the following types of non-standard synchronizers (Figure 25):

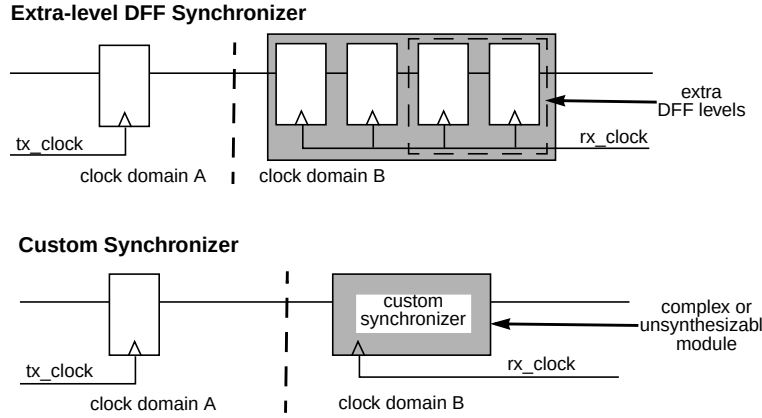
- ❏ Extra-level DFF synchronizers.

  Standard 2DFF synchronizer is extended by adding extra levels of flip-flops.

- ❏ Custom synchronizers.

  Synchronizer implemented with custom complex or unsynthesizable logic.

**Figure 25: Non-standard synchronizer types**



## set_cdc_synchronizer

The set_cdc_synchronizer global directive identifies non-standard synchronizer types and their corresponding properties. Use this directive to configure CDC analysis to handle specific extra-level DFF or custom synchronizers. This directive has the following syntax:

```
set_cdc_synchronizer
       { dff [-level level]
           [-tx_clock clock_signal] [-rx_clock clock_signal]
       | custom -module sync_module
           [-pins variable...] [-clock clock_signal]}
```

Each directive identifies a single synchronizer type, so you must specify either one of following sets of arguments, but not both.

dff

Identifies an extra-level DFF synchronizer type. The –level is the number of DFF instances in the synchronizer (default:2). Single-bit crossings synchronized by this type of synchronizer are considered properly synchronized–by default, they have unknown severity. Those single-bit crossings synchronized with other types of DFF synchronizers (for example, 2DFFs or other types of extra-level DFF synchronizers) are not considered valid—by default, they have violation severity.

You can restrict the assignment to DFF synchronization schemes on signals from a specified –tx_clock domain or to a specified –rx_clock domain or on signals between both.

custom

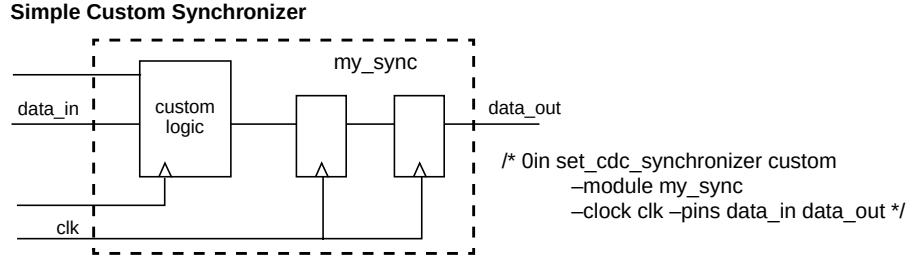
Identifies one or more custom synchronizer types. A custom synchronization scheme must be contained within its own module definition (Figure 25). You must identify the synchronizer module (i.e., the –module argument is required). You can use wildcards in the module identifier: the custom synchronizer specification applies to all matching modules.

Once a module has been defined as a custom synchronizer, CDC analysis will not report structural issues within the module, but it will continue to perform analysis on the synchronizer inputs to ensure they are used correctly. For example, if the domain of the signal feeding the input of a custom synchronizer does not match the domain of the associated pin, then CDC analysis performs combinatorial logic checking. The definition of custom synchronizers is particularly useful for complex synchronization schemes and for standard synchronization that includes testability logic.

The –clock argument identifies the local clock pin on the custom synchronizer module corresponding to the currently-defined synchronization scheme. Even if the module has only one clock input, you should explicitly identify this input (for sanity-checking purposes).

The –pins option specifies the local pins on the synchronizer module associated with the specified clock. If –pins is not specified, all pins of the module are associated with the specified clock.

**Figure 26: Custom synchronizer example**




## CLASSIFYING CLOCK DOMAIN CROSSINGS

For CDC analysis, the CDC compiler classifies the type of each CDC signal (control or data) in the design using predefined rules and templates. After each CDC signal's type is identified, the CDC compiler analyzes the signal to ensure that a synchronization scheme for that signal type is used. If it is not, the CDC compiler issues a warning and assigns the signal a corresponding severity (see "Severity Levels" on page 58). For some designs, default CDC analysis might incorrectly identify the signal type. The result depends on the type of incorrectly identified signal. The CDC compiler might report a false error or it might report a correct synchronization scheme—but one that is different from the intended synchronization scheme.

## set_cdc_signal

The set_cdc_signal global directive reclassifies the CDC signal type of specified CDC signals or adds properties to specified CDC signals. Use this directive to override the inferred classifications of CDC signals and to identify synchronization properties of particular CDC signals. This directive has the following syntax:

```
set_cdc_signal
        variable...
        {–control | –data | –split | –gray_coded | –stable}
        [–module module]
```

You must specify the following:

- *variable...*

  One or more CDC signals. Names can include wildcards. The classification or CDC property assigned by the directive is applied to all matching variables.

- –control, –data, –split, –gray_coded or –stable

  Classification or property to apply.

The directive applies to matching CDC signals in all instances of the module specified by *module*. Wildcards are allowed in –module, in which case the directive applies to all instances of the matching modules. If –module is omitted, the directive applies to the current module (when the directive is placed in the HDL for the module) or the –d module (when the directive is placed in a control file).

For example:

```
module cdc_ctrl;
    // 0in set_cdc_signal tx_status –control –module mtr
    // 0in set_cdc_signal write_addr –gray_coded –module pci_if
endmodule
```

The set_cdc_signal directive recognizes the following classifications and properties:

- –control

  Identifies the specified multi-bit *variables* as control signals for CDC analysis.

  By default, a multiple-bit CDC signal is assumed to be a data bus, which normally should not be synchronized by only dual flip-flops or 4 latches. CDC data signals that use these synchronization schemes are reported as BUS SYNC warnings. By reclassifying a multiple-bit CDC signal as a group of control signals, these synchronization schemes are considered valid.

- –data

  Identifies the specified single-bit *variables* as data signals for CDC analysis.

  By default, a single-bit CDC signal is assumed to be a control signal, which normally may be synchronized by only flip-flops or 4 latches. By reclassifying a single-bit CDC signal as a data signal, these synchronization schemes are considered invalid. CDC data signals that use these synchronization schemes are reported as BUS SYNC warnings.

- –split

  Treats bits of the specified multiple-bit *variables* as individual control signals for CDC analysis.

  By default, synchronization of a CDC data bus must be performed on the whole bus. If not, the bus is marked with a BUS SYNC warning. But, if a register has only one bit derived from a CDC signal, then synchronization of that bit should be allowed. For example, a status bit might be extracted from a state variable or a single multiple-bit register (for example, a status register) might store amalgamated control signals. Splitting a CDC data bus for CDC analysis eliminates the BUS SYNC warning.

  Treating bits individually is independent of the synchronization type: the bits can be synchronized with control synchronization (for example, 2DFF) or data synchronization (for example, DMUX). You might want to treat bits in a bus individually for any of the following reasons:

  - Only some bits of the bus cross a clock domain boundary.
  - Different bits in the bus go to different clock domains.
  - The bus is a collection of individual signals (such as status signals) that you want to consider as individual bits.

  Various bits of a split bus can be used together in the destination domain (for example, in a receiving state machine decoding). To check for reconvergence problems CDC analysis identifies these crossings as POTENTIAL RECONVERGENCE warnings.

- –gray_coded

  Identifies the specified *variables* as properly gray-coded data buses for CDC analysis.

  2DFF and 4latch synchronizations can be valid for a gray-coded data bus. So by default, checks for CDC data buses that have either of these types of synchronizers are promoted as a cdc_hamming_distance_one checkers to verify both gray-coding and synchronization. Identifying a particular variable

as gray-coded indicates the data bus is properly gray-coded (i.e., with gray coder/decoder logic), so gray-code checking is unnecessary. In this case, the crossing check is promoted as a cdc_sync checker.

- –stable

  Identifies the specified *variables* as stable signals for CDC analysis.

  By default, all CDC signals require synchronization. By identifying a particular CDC signal as stable, the signal is assumed to require no synchronization. CDC analysis treats the signal as standard non-CDC signal.

## CHANGING CDC REPORTING AND PROMOTION

After CDC analysis, the CDC compiler reports information about clock domain crossings in the CDC report (0in_cdc.rpt) and generates a promotion file (0in_cdc_ctrl.v) containing directives for promoted transfer protocol checks.

### Severity Levels

CDC information is grouped according to the severity level of the logic comprising each crossing paired with its synchronization scheme. Each severity type represents a different level of concern:

- A *violation* is considered a must-fix situation: all violations should be eliminated before proceeding to simulation or formal verification.
- An *unknown* is considered a potential problem: unknowns are usually promoted to CDC transfer protocol checks.
- An *evaluation* is considered a properly synchronized crossing scheme: evaluations are reported for informational purposes, but they are assumed to obey their transfer protocols.

In addition, some clock domain crossings might be extraneous (either in general or just for the current analysis). Reporting of these crossings can be turned off to simplify the CDC report. The CDC compiler assigns default CDC severities according to the synchronization scheme, however, note the following:

- All crossings are reported (i.e., none are turned off).
- Clock domain crossings with suspicious synchronization have unknown severity.
- Clock domain crossings with proper synchronization have evaluation severity.

## Promoted Checks

The promotion file generated by the CDC compiler contains checker directives for the transfer protocols of clock domain crossings of concern. By including the promotion file as a control file, CDC transfer protocols can be checked during static formal verification, simulation and dynamic formal verification.

A promotion file could contain a checker directive for each clock domain crossing, but in most cases that approach is too aggressive. You normally want to focus verification effort on potential problems. By default, the CDC compiler promotes CDC checks for unknowns.

## set_cdc_report

The set_cdc_report global directive sets the severity level and promotion property of matching clock domain crossings. Use this directive to override the default severity and promotion of clock domain crossing checks. This directive has the following syntax:

```
set_cdc_report
        { –severity {violation | unknown | evaluation | off}
        | –promotion {on | off}
        | –severity {violation | unknown | evaluation | off}
                –promotion {on | off} }
        [–scheme sync_scheme]
        [–from variable] [–to variable] [–through variable]
        [–tx_clock clock_signal] [–rx_clock clock_signal]
        [–from_signals variable variable...] [–module module]
```

You must specify one of the following (or both):

- –severity *level*

  Severity level: violation, unknown, evaluation or off. Specifying off removes the crossings from CDC analysis.

- –promotion on | off

  Whether or not to promote a transfer protocol checker for the crossing.

If you do not specify other arguments, the directive applies to all CDC crossings. Otherwise, you can specify any combination of the following arguments to identify one or more CDC crossings. The specified crossings are all of those that match all of the criteria:

- –scheme *sync_scheme*

  Crossings that are identified as conforming to the specified synchronization scheme.

- –from *variable* –to *variable* –through *variable*

  One or more paths that include CDC crossings. You can specify any combination of –from, –to and –through arguments and you can include wildcards in variables.

- –tx_clock *clock_signal*

  Crossings from transmission logic clocked by a particular clock.

- –rx_clock *clock_signal*

  Crossings to receive logic clocked by a particular clock.

- –from_signals *variable variable...*

  Two or more variables from synchronizers in reconverging CDC paths. This option only applies to reconvergence analysis, so you must specify –scheme reconvergence with this option. The variables identify registers or wires that contain data in one clock domain that are separately synchronized and then recombined in another clock domain.

The directive applies to matching CDC crossings in all instances of the module specified by *module*. Wildcards are allowed in –module, in which case the directive applies to all instances of the matching modules. If –module is omitted, the directive applies to instances of the current module (when the directive is placed in the HDL for the module) or the –d module (when the directive is placed in a control file).

For example:

```
module cdc_ctrl;
    /* 0in set_cdc_report –scheme blackbox
            –from out2a –to bb1.* –severity off  */
    // 0in set_cdc_report –scheme two_dff_phase –severity evaluation
endmodule
```

## OTHER GLOBAL DIRECTIVES

Other useful global directives that affect CDC are described below.

### set_reset

For asynchronous reset detection, the CDC compiler infers resets. However, you can explicitly specify resets using the set_reset directive. The set_reset directive has the following syntax:

```
set_reset reset_signal –module module [–posedge | –negedge]
      [–async | –sync]
```

The specified signal is identified as a reset signal with the specified characteristics (–posedge –async is the default).

### exclude_hier

By default, the entire design specified by the –d *module* argument (or all top-level modules) is analyzed by cdc. You can use the exclude_hier global directive to skip processing for a module and its lower levels of hierarchy. The exclude_hier directive has the following syntax:

```
exclude_hier –module module
```

The specified modules (wildcards are allowed) and all of their lower levels of hierarchy are excluded from processing by the CDC compiler. The exclude_hier global directive retains the modules' port definitions. If you use +skip_modules+*module* instead of exclude_hier, the port definitions are excluded, so the parent modules are black-boxed.

### set_black_box

If a submodule contains logic that the CDC compiler should not analyze, you can designate the module as a black box. If a CDC signal drives logic that includes a black box, it is reported as a blackbox CDC scheme.

The set_black_box global directive designates a module as a black box. This directive has the following syntax:

```
set_black_box black_box_module
```

## set_cdc_reconvergence_depth

The CDC compiler checks each CDC signal for a number of cycles after the value is received in the receive clock domain, to see if the transmitted data value reconverges with other transmitted data. This is the reconvergence scheme check. The maximum number of cycles analyzed is called the *reconvergence depth*. The CDC compiler only detects reconverging values if they reconverge within the reconvergence depth from the cycle the values are received.

The default reconvergence depth is 1 cycle, which corresponds to combinational reconvergence. The set_cdc_reconvergence_depth global directive changes the reconvergence depth. This directive has the following syntax:

```
set_cdc_reconvergence_depth number_of_cycles
```

# CHAPTER 3: CDC SCHEME DATA SHEETS

CDC analysis identifies logic patterns relevant to CDC verification. Groups of related patterns identify specific classes of situations called *CDC schemes*. Each CDC scheme represents a specific type of CDC synchronizer architecture or a potential CDC issue. This chapter consists of the data sheets for the various CDC schemes (Table 8). Each data sheet has the following information:

- ❏ General Information — 0-In View message, schematic representation, description and the following information:
- ❏ Crossing Type — signal, data bus, both or not applicable.
- ❏ Promoted Checker — CDC checker promoted to check the associated transfer protocol.
- ❏ Default Severity — evaluation, unknown or violation.
- ❏ Examples — examples of global directives that change the default behavior.

In addition, some data sheets show:

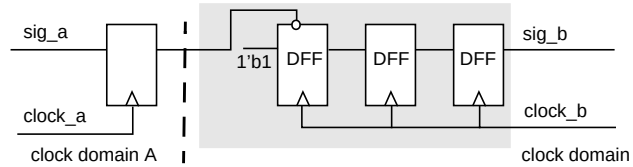
- ❏ Potential Problems — information about consequences you should consider.
- ❏ Notes — additional information relevant to the scheme.

**Table 8: CDC checking schemes**

| Scheme Type | Signal Synchronizer | Message | Default Severity |
|---|---|---|---|
| two_dff | Two or more in-phase flip-flops. | TWO DFF SYNCHRONIZER | evaluation |
| two_dff_phase | Two out-of-phase flip-flops. | TWO DFF SYNCHRONIZER OPPOSITE POLARITY | violation |
| four_latch | Four or more cascaded latches. | FOUR LATCH SYNCHRONIZER | evaluation |
| pulse_sync | Pulse. | PULSE SYNC | evaluation |
| shift_reg | Shift register. | SHIFT REG | evaluation |
| dff_sync_gated_clk | Two flip-flops with gated clock. | DFF GATED SYNC | violation |
| async_reset | Asynchronous reset scheme. | ASYNC RESET | evaluation |
| async_reset_no_sync | —none— | ASYNC RESET NO SYNC | violation |
| no_sync | —none— | MISSING SYNCHRONIZER | violation |
| | **Data Synchronizer** | | |
| bus_two_dff | Two or more in-phase flip-flops. | BUS SYNC | unknown |
| bus_two_dff_phase | Two out-of-phase flip-flops. | BUS SYNC | unknown |
| bus_four_latch | Four or more cascaded latches. | BUS SYNC | unknown |
| bus_dff_sync_gated_clk | Two flip-flops with gated clock. | BUS DFF GATED SYNC | violation |
| multi_bits | —none— | MULTIPLE BITS | unknown |
| | **Signal and Data Synchronizer** | | |
| dmux | MUX enable. | DMUX | unknown |
| multi_sync_mux_select | Multiple MUXs. | MULTIPLE SYNCHRONIZERS AT DMUX | unknown |
| memory_sync | 2D array. | MEMORY SYNC | evaluation |
| custom_sync | Custom. | CUSTOM | evaluation |
| | **Potential Problem** | | |
| combo_logic | Combinational logic on a synchronization path. | COMBO LOGIC | violation |
| reconvergence | Reconvergent CDC signals. | RECONVERGENCE | violation |
| redundant | CDC signal drives multiple synchronizers. | REDUNDANT | violation |
| fanin_different_clks | Fanin logic from multiple clock domains. | FANIN LOGIC FROM DIFFERENT CLOCKS | violation |
| blackbox | Black box in fan-out of a synchronized signal. | BLACKBOX | evaluation |

# ASYNC_RESET

ASYNC RESET : Signal feeds to the asynchronous reset port of the receiving register.




Synchronization using an asynchronous reset is a standard method of synchronizing a 1-bit signal.

### Crossing Type

1-bit signal

### Default Severity

evaluation

### Promoted Checker

none

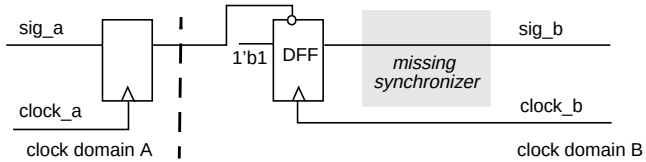
### Examples

1. To turn severity to violation:

```
// 0in set_cdc_report –scheme async_reset –severity violation
```

## ASYNC_RESET_NO_SYNC

ASYNC RESET NO SYNC : Signal feeds to the asynchronous reset port of the receiving register has no synchronizer.




Synchronization using an asynchronous reset is a standard method of synchronizing a 1-bit signal but the receiving register output must drive a control signal synchronizer before driving receive domain logic.arst_sync.scr

### Crossing Type

1-bit signal

### Default Severity

violation

### Promoted Checker

none

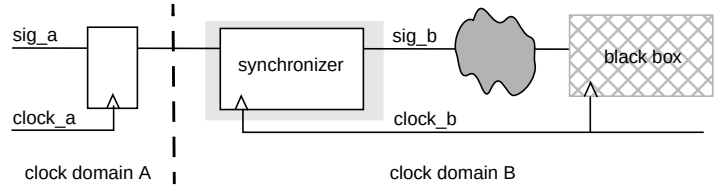
### Examples

1. To turn reporting off:

```
// 0in set_cdc_report –scheme async_reset_no_sync –severity off
```

# BLACKBOX

BLACKBOX : Blackbox Crossing.




### Crossing Type

control signal or data bus

### Default Severity

evaluation

### Promoted Checker

none

### Examples

1. To turn severity to violation:

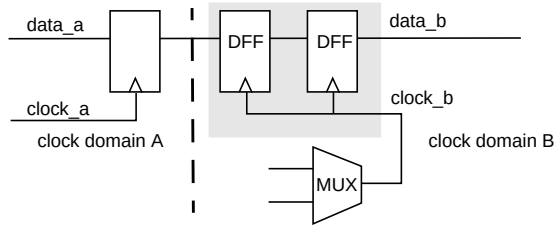
```
// 0in set_cdc_report –scheme blackbox –severity violation
```

2. To set module blackbox as a black box:

```
// 0in set_black_box blackbox_module
```

## BUS_DFF_SYNC_GATED_CLK

BUS DFF GATED SYNC : Multiple-bits signals synchronized with DFF synchronizer with gated clock.




Synchronization using two flip-flops is a standard method of synchronizing a 1-bit signal.

### Crossing Type

multiple-bit data bus

### Default Severity

violation

### Promoted Checker

none

### Examples

1. To turn severity to unknown:

```
/* 0in set_cdc_report –scheme bus_dff_sync_gated_clk
    –severity unknown */
```

2. To turn off reporting for a particular CDC signal:

```
/* 0in set_cdc_report –scheme bus_dff_sync_gated_clk –severity off
    –from in1 –to r1  */
```

### Potential Problems

1. CDC analysis does not promote a checker for this scheme. If you switch severity to unknown, you should implement a transfer protocol checking scheme using one or more checkers.

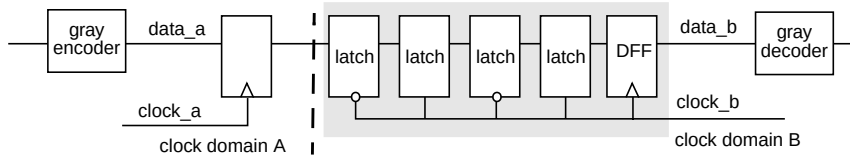
Notes

1. If you use a DFF synchronization scheme that has more than two flip-flops, you can get CDC analysis to recognize the scheme by specifying a set_cdc_synchronizer directive (see “set_cdc_synchronizer” on page 54). For example, the following global CDC directive identifies 4 DFF synchronizers as valid dff_sync_gated_clk schemes for CDC paths from the tx_clk domain to the rx_clk domain:

```
/* 0in set_cdc_synchronizer dff –level 4
       –tx_clock tx_clk –rx_clock rx_clk */
```

## BUS_FOUR_LATCH

BUS SYNC : Multiple-bit signal synchronized by latch synchronizer.




Synchronization using four latches is a standard method of synchronizing a gray-coded data bus. Gray coding (i.e., hamming distance 1) assures that only one bit of data changes in any cycle, so a four-latch synchronizer—typically used for 1-bit signal crossings—is appropriate if the data changes only at the frequency of the receiving domain.

### Crossing Type

multiple-bit data bus

### Default Severity

unknown

### Promoted Checker

cdc_hammimg_distance_one

### Examples

1. To turn severity to violation:

```
// 0in set_cdc_report –scheme bus_four_latch –severity violation
```

2. To turn off promotion:

```
// 0in set_cdc_report –scheme bus_four_latch –promotion off
```

3. Example promoted transfer protocol checker for four_latch synchronization scheme:

```
// Synchronized multi-bit variable q
/* 0in cdc_hamming1
        –tx_clock clk3 –tx_var r1
        –name cdc_data_0  –module test
        –inferred  */
```

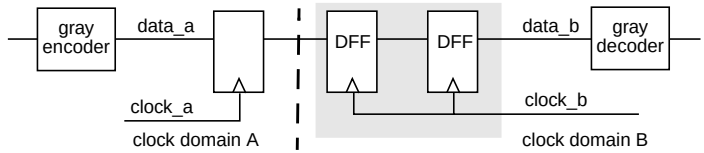
Potential Problems

1. Four-latch synchronization of a data bus assumes the bus is gray coded. Any data buses that cross clock domains that are not gray coded should be identified as requiring complete data synchronization. Any of these crossings synchronized by four-latch synchronizers should be flagged as violations. To set these schemes to violations, use the set_cdc_report directive.

```
/* 0in set_cdc_report –scheme bus_four_latch
       –tx_clock clk_a –severity violation */
```

## BUS_TWO_DFF

BUS SYNC : Multiple-bit signal synchronized by DFF synchronizer.




Synchronization using 2 flip-flops is a standard method of synchronizing a gray-coded data bus. Gray coding (i.e., hamming distance 1) assures that only one bit of data changes in any cycle, so a 2DFF synchronizer—typically used for 1-bit signal crossings—is appropriate if the data changes only at the frequency of the receiving domain.

### Crossing Type

multiple-bit data bus

### Default Severity

unknown

### Promoted Checker

cdc_hammimg_distance_one

### Examples

1. To turn severity to violation:

```
// 0in set_cdc_report –scheme bus_two_dff –severity violation
```

2. To turn off promotion:

```
// 0in set_cdc_report –scheme bus_two_dff –promotion off
```

3. Example promoted transfer protocol checker for two_dff synchronization scheme:

```
// Synchronized multi-bit variable U1.U1.dsyncr
/* 0in cdc_hamming1
    –tx_clock clk1  –tx_var U1.U1.dinr
    –name cdc_data_1 –module dut –inferred  */
```

### Potential Problems

1. 2DFF synchronization of a data bus assumes the bus is gray coded. Any data buses that cross clock domains that are not gray coded should be identified as requiring complete data synchronization. Any of these crossings synchronized by 2DFF synchronizers should be flagged as violations. To set these schemes to violations, use the set_cdc_report directive.

```
/* 0in set_cdc_report –scheme bus_two_dff
       –tx_clock clk_a –severity violation */
```

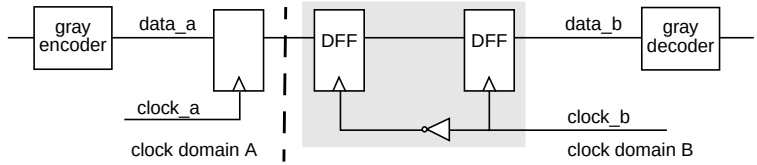
### Notes

1. If you use a DFF synchronization scheme that has more than two flip-flops, you can get CDC analysis to recognize the scheme by specifying a set_cdc_synchronizer directive (see “set_cdc_synchronizer” on page 54). For example, the following global CDC directive identifies 4 DFF synchronizers as valid two_dff schemes for CDC paths from the tx_clk domain to the rx_clk domain:

```
/* 0in set_cdc_synchronizer dff –level 4
       –tx_clock tx_clk –rx_clock rx_clk */
```

## BUS_TWO_DFF_PHASE

BUS SYNC : Multiple-bit signal synchronized by DFF synchronizer.




Synchronization using two opposite polarity flip-flops is a bad scheme in most cases and should be caught as a static violation. However, some synchronization schemes are tolerant of this type of synchronizer, for example, cases in which the receive clock domain is clocked with the same clock as the transmit domain, but with the opposite phase. When used to synchronize a data bus, the bus should be gray-coded (i.e., have hamming distance 1), which assures that only one bit of data changes in any cycle, and the data should change only at the frequency of the receiving domain.

### Crossing Type

multiple-bit data bus

### Default Severity

violation

### Promoted Checker

none (CDC cannot determine the proper coefficients for the promoted checker).

### Examples

1. To turn severity to unknown:

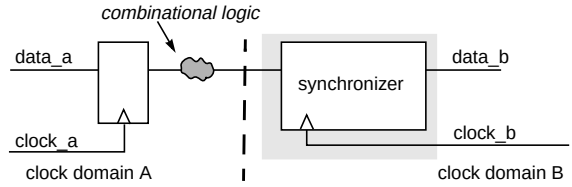
```
// 0in set_cdc_report –scheme bus_two_dff_phase –severity unknown
```

### Potential Problems

1. Synchronization of a data bus using two opposite polarity DFFs assumes the bus is gray coded.
2. CDC analysis does not promote a checker for this scheme. If you switch severity to unknown, you should implement a transfer protocol checking scheme using multiple checkers (including a gray-code checker).

# COMBO_LOGIC

COMBO LOGIC : Combinational logic before synchronizer.




Combinational logic in a synchronization path is a significant problem for synchronized signals, because the data used to determine an acceptable failure rate on synchronization paths assumes a single transition on a CDC signal during each clock period. But if combinational logic is placed on the path, this assumption is false because of glitch propagation. As a result, the error rate rises significantly. To address this problem, you should remove all combinational logic from the logic path.

For some cases, this combinational logic will be constant during normal operation, so glitches cannot be generated. For example, the combinational logic might consist of test MUXes or configuration logic. To remove the violation, identify the signal feeding that logic as a constant value using `set_constant`.

### Crossing Type

control signal or data bus

### Default Severity

violation

### Promoted Checker

none

### Examples

1. To specify that the MUX select in the combinational logic feeding a CDC signal is constant 1’b0 (so the crossing can be checked for proper synchronization):

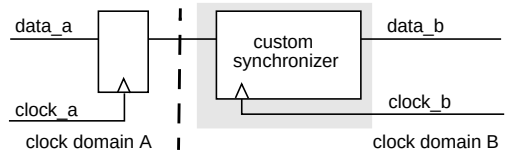
   ```
   // 0in set_constant test_sel 1'b0
   ```

2. To turn severity for the CDC signal en to unknown:

```
/* 0in set_cdc_report –scheme combo_logic
        –severity unknown –from dut.U0.en
```

# CUSTOM_SYNC

CUSTOM : Custom CDC Scheme.




CDC analysis recognizes custom synchronizer logic (for both control signals and data buses) and reports each occurrence as a custom CDC scheme. A custom synchronizer must be defined as a separate module and you must specify each custom synchronizer module using the set_cdc_synchronizer CDC directive (page 54).

### Crossing Type

control signal or data bus

### Default Severity

evaluation

### Promoted Checker

none

### Examples

1. To specify the cust_sync module with clock pin clk as a custom synchronizer:

```
// 0in set_cdc_synchronizer custom –module cust_sync –clock clk
```

2. Suppose the cust_2sync module acts as two synchronizers: one with clk1, in1, out1 and the other with clk2, in2, out2. You can identify the synchronizer using the following directives:

```
/* 0in set_cdc_synchronizer custom –module cust_2sync
       –clock clk1 –pins in1 out1  */
/* 0in set_cdc_synchronizer custom –module cust_2sync
       –clock clk2 –pins in2 out2  */
```

3. To turn severity to unknown:

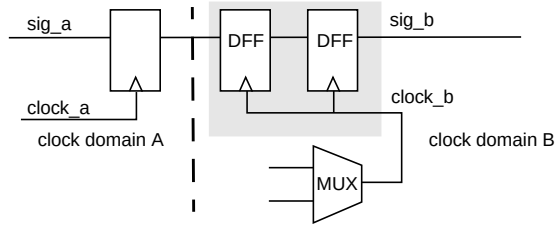
```
// 0in set_cdc_report –scheme custom_sync –severity unknown
```

### Potential Problems

1. CDC analysis does not promote a checker for this scheme. If you switch severity to unknown, you should implement a transfer protocol checking scheme using one or more checkers.

# DFF_SYNC_GATED_CLK

DFF GATED SYNC : DFF synchronizer with gated clock.




Synchronization using two flip-flops is a standard method of synchronizing a 1-bit signal.

### Crossing Type

1-bit signal

### Default Severity

violation

### Promoted Checker

none

### Examples

1. To turn severity to unknown:

```
// 0in set_cdc_report –scheme dff_sync_gated_clk –severity unknown
```

2. To turn off reporting for a particular CDC signal:

```
/* 0in set_cdc_report –scheme dff_sync_gated_clk –severity off
    –from in1 –to r1  */
```

### Potential Problems

1. CDC analysis does not promote a checker for this scheme. If you switch severity to unknown, you should implement a transfer protocol checking scheme using one or more checkers.

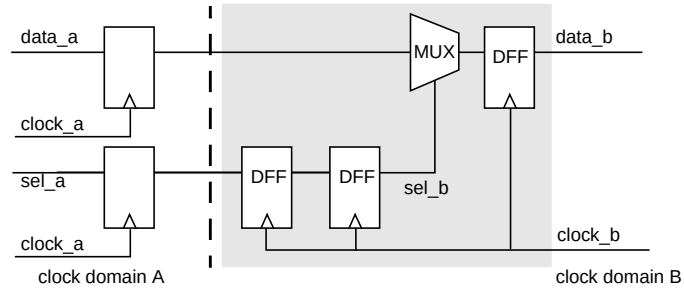
Notes

1. If you use a DFF synchronization scheme that has more than two flip-flops, you can get CDC analysis to recognize the scheme by specifying a set_cdc_synchronizer directive (see “set_cdc_synchronizer” on page 54). For example, the following global CDC directive identifies 4 DFF synchronizers as valid dff_sync_gated_clk schemes for CDC paths from the tx_clk domain to the rx_clk domain:

```
/* 0in set_cdc_synchronizer dff –level 4
       –tx_clock tx_clk –rx_clock rx_clk */
```

# DMUX

DMUX : DMUX synchronization.




Synchronization using a DMUX scheme is a standard method of synchronizing a data bus (or a control signal).

### Crossing Type

control signal or data bus

### Default Severity

unknown

### Promoted Checker

`cdc_dsel`

### Examples

1. To turn severity to evaluation:

```
// 0in set_cdc_report –scheme dmux –severity evaluation
```

2. To turn off promotion:

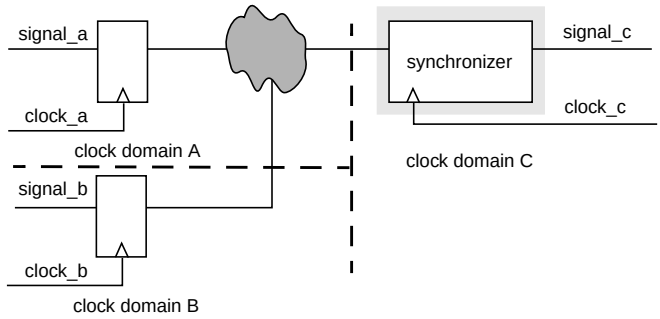
```
// 0in set_cdc_report –scheme dmux –promotion off
```

3. Example promoted transfer protocol checker for `dmux` synchronization scheme:

```
/* 0in cdc_dsel
    –tx_data isyncdbus.data_reg –tx_clock clk1
    –rx_clock clk2 –tx_data_select 1'b0
    –rx_data_select (isyncdbus.stage3_q ^ isyncdbus.isync.out)
    –tx_min 'P_clk1_clk2_tx_min
    –name cdc_dsel_1 –module test –inferred */
```

## FANIN_DIFFERENT_CLKS

FANIN LOGIC FROM DIFFERENT CLOCKS : Fanin logic from multiple clock domains.




Fanin logic for the input to a synchronizer must be synchronized to a single transmit clock domain for CDC analysis to properly identify the synchronization scheme. CDC analysis reports a FANIN LOGIC FROM DIFFERENT CLOCKS scheme if it finds two unsynchronized signals from different clock domains in the fanin logic of the input to a synchronizer

For some cases, the signals from one domain will be constant during normal operation. For example, the fanin logic from one of the domains might consist of test MUXes or configuration logic. To remove the violation, identify the signals from that logic as constants using `set_constant` directives.

### Crossing Type

control signal or data bus

### Default Severity

violation

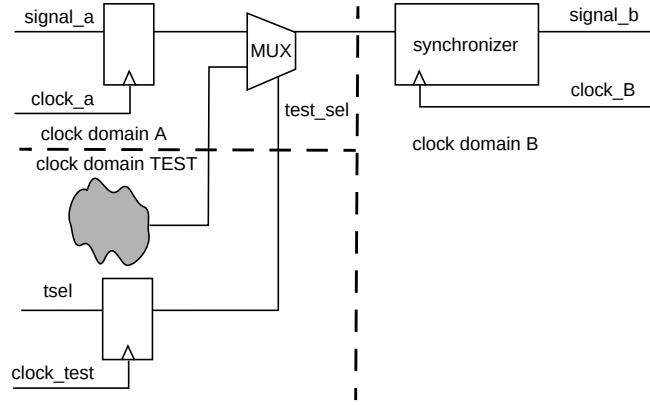
### Promoted Checker

none

Examples

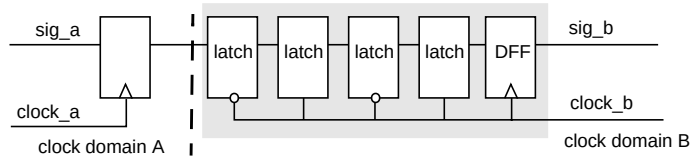
1. The following example sets the test_sel signal from the TEST clock domain to the constant 1’b0, to remove the fanin from multiple clock domains violation:

```
// 0in set_constant test_sel 1'b0
```



# FOUR_LATCH

FOUR LATCH SYNCHRONIZER : Single-bit signal synchronized by latch synchronizer.




Synchronization using four flip-flops is a standard method of synchronizing a 1-bit signal.

### Crossing Type

1-bit signal

### Default Severity

evaluation

### Promoted Checker

cdc_sync

### Examples

1. To turn severity to unknown:

   ```
   // 0in set_cdc_report –scheme four_latch –severity unknown
   ```

2. To turn off promotion:

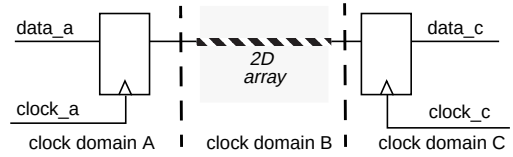
   ```
   // 0in set_cdc_report –scheme four_latch –promotion off
   ```

3. Example promoted transfer protocol checker for four_latch synchronization scheme:

   ```
   /* 0in cdc_sync
       –tx_var mtr.fifo_rd –tx_clock pci_clk
       –tx_min ‘P_pci_clk_cpu_clk_tx_min
       –name cdc_sync_0 –module cpu_top
       –inferred */
   ```

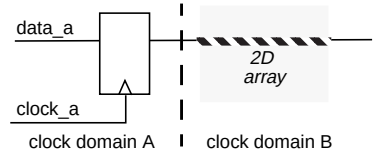
# MEMORY_SYNC

MEMORY SYNC : Memory Synchronization.

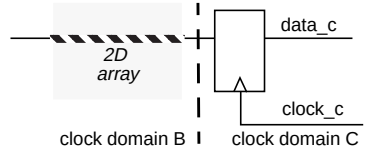



CDC synchronization using a 2D array (for example, a memory element used as a FIFO) is a standard method of synchronizing CDC signals or data buses. CDC analysis detects CDC crossings to and from 2D arrays, but the analysis cannot verify proper synchronization configuration and does not promote a transfer protocol checker to verify proper synchronization operation.

Both write:




and read:




memory synchronization schemes are reported.

### Crossing Type

control signal or data bus

### Default Severity

evaluation

### Promoted Checker

none

Examples

1. To turn severity to unknown:

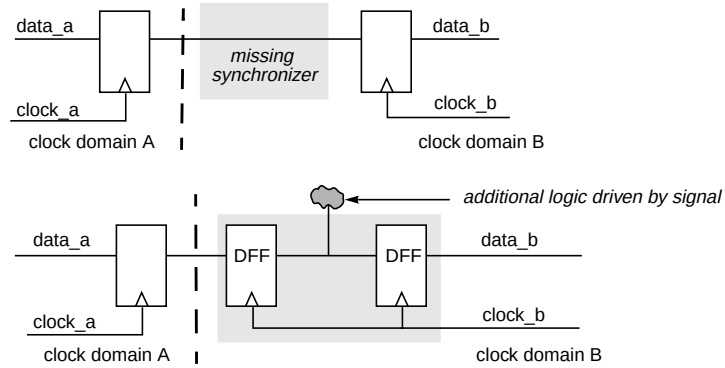
   ```
   // 0in set_cdc_report –scheme memory_sync –severity unknown
   ```

2. To turn off reporting:

   ```
   // 0in set_cdc_report –scheme memory_sync –severity off
   ```

# MULTI_BITS

MULTIPLE BITS : Multiple-bit signal across clock domain boundary.




An unsynchronized CDC data bus in most cases should be a static violation. Similarly, a CDC data bus synchronized using a 2DFF synchronizer should not drive any logic from the wire connecting the two flip-flops.

However, some data buses from test or configuration logic might be constant or ad hoc synchronous with the receive clock domain. Similarly, test or configuration logic driven by the wire connecting the two flip-flops of a 2DFF synchronizer might also be constant or ad hoc synchronous with the receive clock domain. The severity of these crossing schemes are typically set to unknown.

### Crossing Type

multi-bit data bus

### Default Severity

unknown

### Promoted Checker

`cdc_sample`

### Examples

1. To turn reporting off for the `status_test` signal in module `dut`:

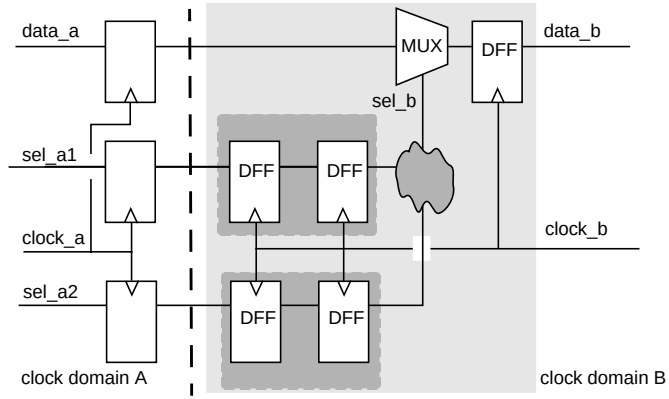
```
/* 0in set_cdc_report –scheme multi_bits –from status_test
      –severity off –module dut */
```

2. Example promoted transfer protocol checker for multi_bits synchronization:

```
/* 0in cdc_sample
       -tx_var u1.q -rx_var u2.q
       -tx_clock clk1 -rx_clock clk2
       -name cdc_data_0 -module dut -inferred */
```

# MULTI_SYNC_MUX_SELECT

MULTIPLE SYNCHRONIZERS AT DMUX : Mux select fanin contains more than one synchronizer.




In most cases, if a MUX select is used to synchronize a signal or data bus, the MUX select fanin should not have more than one synchronizer. However, some synchronization schemes are tolerant of this type of synchronization scheme.

### Crossing Type

control signal or data bus

### Default Severity

unknown

### Promoted Checker

none

### Examples

1. To turn severity to evaluation:

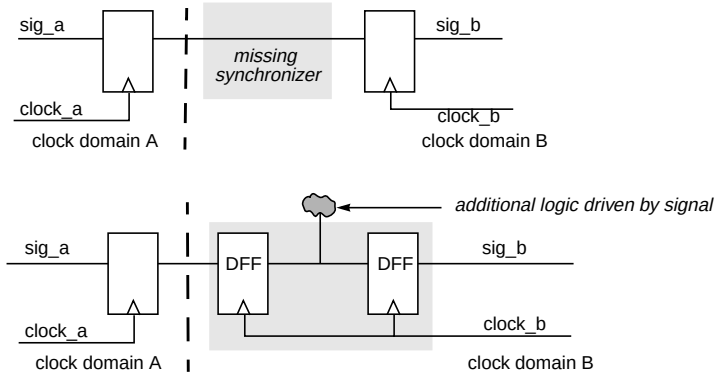
```
/* 0in set_cdc_report –scheme multi_sync_mux_select
      –severity evaluation */
```

2. To turn off promotion:

```
// 0in set_cdc_report –scheme multi_sync_mux_select –promotion off
```

## NO_SYNC

MISSING SYNCHRONIZER : Single-bit signal does not have proper synchronizer.




An unsynchronized CDC signal in most cases should be a static violation. Similarly, a CDC signal synchronized using a 2DFF synchronizer should not drive any logic from the wire connecting the two flip-flops.

However, some signals from test or configuration logic might be constant or ad hoc synchronous with the receive clock domain. Similarly, test or configuration logic driven by the wire connecting the two flip-flops of a 2DFF synchronizer might also be constant or ad hoc synchronous with the receive clock domain. The severity of these crossing schemes are typically set to unknown.

### Crossing Type

1-bit signal

### Default Severity

violation

### Promoted Checker

`cdc_sample`

### Examples

1. To turn severity to unknown for the write_test signal in module dut:

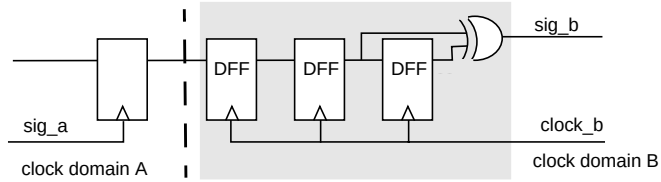
```
/* 0in set_cdc_report –scheme no_sync –from write_test
      –severity unknown –module dut */
```

2. Example promoted transfer protocol checker for no_sync synchronization:

```
/* 0in cdc_sample
        –tx_var u1.q –rx_var u2.q
        –tx_clock clk1 –rx_clock clk2
        –name cdc_data_0 –module dut –inferred */
```

## PULSE_SYNC

PULSE SYNC : Pulse Synchronization.




Pulse synchronization is a standard method of synchronizing a 1-bit signal.

### Crossing Type

1-bit signal

### Default Severity

evaluation

### Promoted Checker

`cdc_sync`

### Examples

1. To turn severity to unknown:

   ```
   // 0in set_cdc_report –scheme pulse_sync –severity unknown
   ```

2. To turn off promotion:

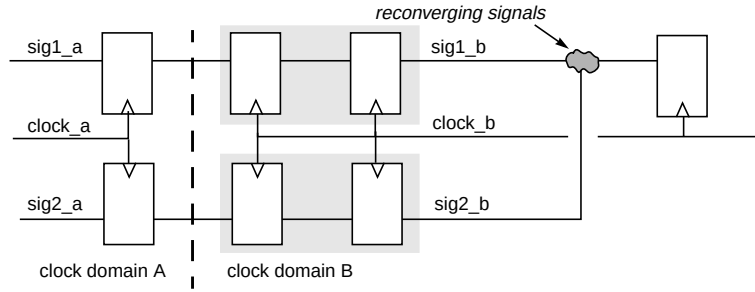
   ```
   // 0in set_cdc_report –scheme pulse_sync –promotion off
   ```

3. Example promoted transfer protocol checker for two_dff synchronization scheme:

   ```
   /* 0in cdc_sync
       –tx_var isyncabus.stage1_q –tx_clock clk1
       –tx_min ‘P_clk1_clk2_tx_min –name cdc_sync_0
       –module test –inferred  */
   ```

# RECONVERGENCE

RECONVERGENCE : Reconvergence of synchronizers.




Reconvergence occurs when the fanin of a flip-flop includes two separately synchronized CDC signals from a different clock domain. Reconvergence of two signals might result in synchronization problems.

When multiple bits reconverge, their relative timing is unpredictable. Logic that receives these signals should account for potential cycle skew, for example by using a Hamming encoding scheme for signals in the receiving domain. For signals that have large guard times between changes, this encoding happens naturally. If it is not feasible to either design receiving logic that accounts for the potential cycle skew or encode the signals, then you should group the signals and transfer them as a group using a synchronized MUX enable (as used for multiple-bit data signals).

### Crossing Type

multiple control signals or data buses

### Default Severity

violation

### Promoted Checker

`cdc_hamming_distance_one`

### Examples

1. To disable reporting of all reconvergence points originating in the `tx_clk` domain:

```
/* 0in set_cdc_report –scheme reconvergence
      –severity off –tx_clock tx_clk  */
```

2. To disable reporting of all reconvergence points originating from the cluster of synchronizers driving stat1, stat2 and stat3:

```
/* 0in set_cdc_report –scheme reconvergence
       –severity off –from_signals stat1 stat2 stat3  */
```

3. Example promoted transfer protocol checker for reconvergence scheme:

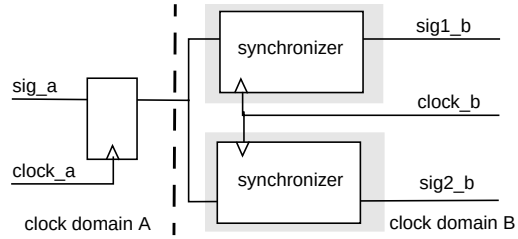
```
// Reconverge on variables bb1, bb0
/* 0in cdc_hamming1
    –tx_clock tx_clk –tx_var ( { out2a, out2b } )
    –name cdc_data_0 –module dut –inferred  */
```

4. To change the reconvergence depth to 4:

```
// 0in set_cdc_reconvergence_depth 4
```

# REDUNDANT

REDUNDANT : Redundant synchronization.




Redundant synchronizers are multiple synchronizers in the same clock domain that synchronize the same CDC signal. If these separately synchronized signals are in the fan-in of the same flip-flop, a reconvergence problem might exist. But, even if they are not, redundant synchronizers create extra logic that can be eliminated by splitting the signals after they are synchronized.

### Crossing Type

multiple control signals or data buses

### Default Severity

violation

### Promoted Checker

`cdc_sync`

### Examples

1. To disable reporting of redundant synchronizers on `*_en` signals from the `tx_clk` domain:

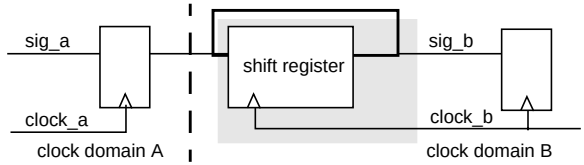
```
/* 0in set_cdc_report –scheme redundant
       –severity off –from *_en –tx_clock tx_clk  */
```

2. Example promoted transfer protocol checker for `redundant` scheme:

```
/* 0in cdc_sync
    –tx_var samp1 –tx_clock clk1
    –tx_min ‘P_clk1_clk2_tx_min
    –name cdc_sync_0 –module dut –inferred */
```

## SHIFT_REG

SHIFT REG : Shift-register synchronization.




Synchronization using a shift register is a standard method of synchronizing a 1-bit signal. It is equivalent to synchronization with two or more in-phase flip-flops.

### Crossing Type

1-bit signal

### Default Severity

evaluation

### Promoted Checker

`cdc_sync`

### Examples

1. To turn severity to unknown:

   ```
   // 0in set_cdc_report –scheme shift_reg –severity unknown
   ```

2. To turn off promotion:

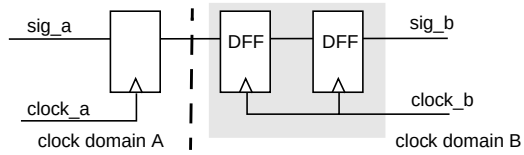
   ```
   // 0in set_cdc_report –scheme shift_reg –promotion off
   ```

3. Example promoted transfer protocol checker for `shift_reg` synchronization scheme:

   ```
   /* 0in cdc_sync
       –tx_var u0.q –tx_clock clk1
       –tx_min 'P_clk1_clk2_tx_min
       –name cdc_sync_0 –module dut –inferred */
   ```

# TWO_DFF

TWO DFF SYNCHRONIZER : Single-bit signal synchronized by DFF synchronizer.




Synchronization using two flip-flops is a standard method of synchronizing a 1-bit signal.

## Crossing Type

1-bit signal

## Default Severity

evaluation

## Promoted Checker

`cdc_sync`

## Examples

1. To turn severity to unknown:

   ```
   // 0in set_cdc_report –scheme two_dff –severity unknown
   ```

2. To turn off promotion:

   ```
   // 0in set_cdc_report –scheme two_dff –promotion off
   ```

3. Example promoted transfer protocol checker for two_dff synchronization scheme:

   ```
   /* 0in cdc_sync
       –tx_var mtr.fifo_rd –tx_clock pci_clk
       –tx_min ‘P_pci_clk_cpu_clk_tx_min
       –name cdc_sync_0 –module cpu_top
       –inferred */
   ```

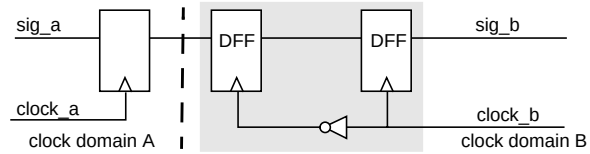
Notes

1. If you use a DFF synchronization scheme that has more than two flip-flops, you can get CDC analysis to recognize the scheme by specifying a set_cdc_synchronizer directive (see “set_cdc_synchronizer” on page 54). For example, the following global CDC directive identifies 4 DFF synchronizers as valid two_dff schemes for CDC paths from the tx_clk domain to the rx_clk domain:

```
/* 0in set_cdc_synchronizer dff –level 4
       –tx_clock tx_clk –rx_clock rx_clk */
```

# TWO_DFF_PHASE

TWO DFF SYNCHRONIZER OPPOSITE POLARITY : Single-bit signal synchronized by DFF of opposite clock polarity.




Synchronization using two opposite polarity flip-flops is a bad scheme in most cases and should be caught as a static violation. However, some synchronization schemes are tolerant of this type of synchronizer, for example, cases in which the receive clock domain is clocked with the same clock as the transmit domain, but with the opposite phase.

### Crossing Type

1-bit signal

### Default Severity

violation

### Promoted Checker

none (CDC cannot determine the proper coefficients for the promoted checker).

### Examples

1. To turn severity to unknown:

```
// 0in set_cdc_report –scheme two_dff_phase –severity unknown
```

### Potential Problems

1. CDC analysis does not promote a checker for this scheme. If you switch severity to unknown, you should implement a transfer protocol checking scheme using one or more checkers.

# 3 CDC SCHEME DATA SHEETS

# CHAPTER 4: CDC ANALYSIS

The CDC compiler checks the design, infers clock trees, identifies signals that cross clock domain boundaries and detects a variety of synchronization patterns (schemes). After running the CDC compiler, you have two ways of analyzing the results:

- ❏ Run 0-In View and view the CDC results.
- ❏ Examine the CDC reports.

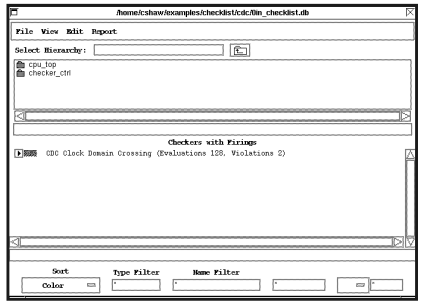
CDC analysis of many large designs can be performed in steps by focusing CDC analysis on different levels of the design hierarchy. You can configure the CDC compiler to generate control files that automate this analysis.

The CDC compiler also suggests CDC checkers for possible inclusion in the design instrumentation. Such promoted assertions can be included in simulation and formal verification.

# RUNNING 0-IN VIEW

You can run 0-In View and load the viewer database generated by the CDC compiler in a single invocation:

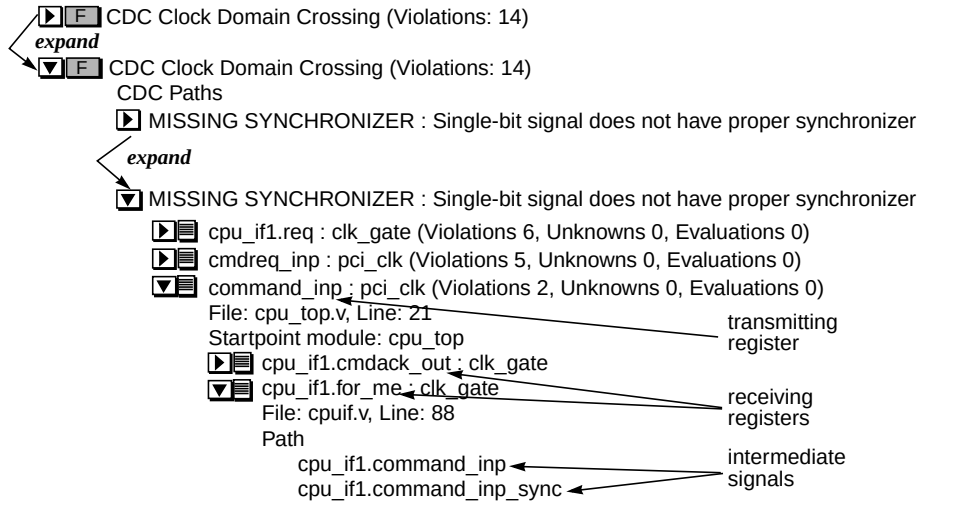
*shell prompt*> `0in_view 0in_cdc.db`




## CDC CHECK FIRINGS

When a viewer database is first loaded, the CDC Firings view is shown. Each check that had at least one firing (i.e., violation) is listed here. Figure 27 shows the details of a sample CDC firing entry.

**Figure 27: Sample CDC check firing entry**



The information in each CDC firing item includes:

- ❏ Violation — Cause of the problem.
- ❏ Transmitting register — transmitting register with its clock.
- ❏ File and line (transmit) — location of the transmitting register in the source code.
- ❏ Startpoint module — source module for the violation.
- ❏ Receiving registers — transmitting registers with their clocks.
- ❏ File and line (receive) — location of the receiving register in the source code.
- ❏ Path — intermediate signals along the clock domain crossing path from the transmitter to the receiving register.

### CDC Paths

The CDC Paths menu (at the bottom of the 0in_view window) reverses the direction of the displayed CDC paths:

- ❏ by Startpoint — lists the transmit register, the receive register and the way points from the transmit register to the receive register.
- ❏ by Endpoint — lists the receive register, the transmit register and the way points from the receive register back to the transmit register.

## CDC CHECK ENTRIES

The CDC Firings view shows information relevant to CDC firings (violations). Run View —>All Checkers to view all CDC information—including information about unknowns and evaluations. Figure 28 shows the details of a sample CDC check entry.

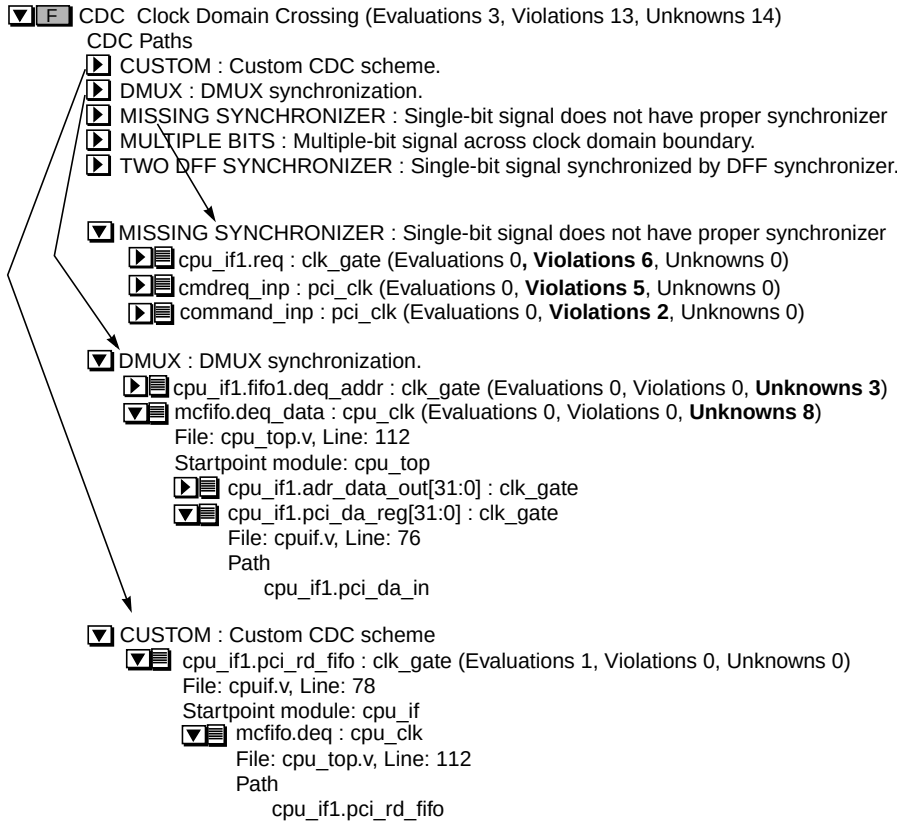
The information in the CDC check entry is grouped by synchronization scheme. Each scheme lists items identifying the paths detected with that scheme. Each item shows a transmitting register and lists the registers that receive data from that register. For each receive register, way points on the path from the transmitting register are listed.

CDC entries show the following severity statistics:

- ❏ Evaluations — properly synchronized crossings detected. The source code browser icon (▤) is green to indicate proper synchronization.

- Violations — synchronization problem detected. The source code browser icon ( ) is red to indicate a problem.
- Unknowns — potential synchronization problem detected. The source code browser icon ( ) is yellow to indicate a possible problem.

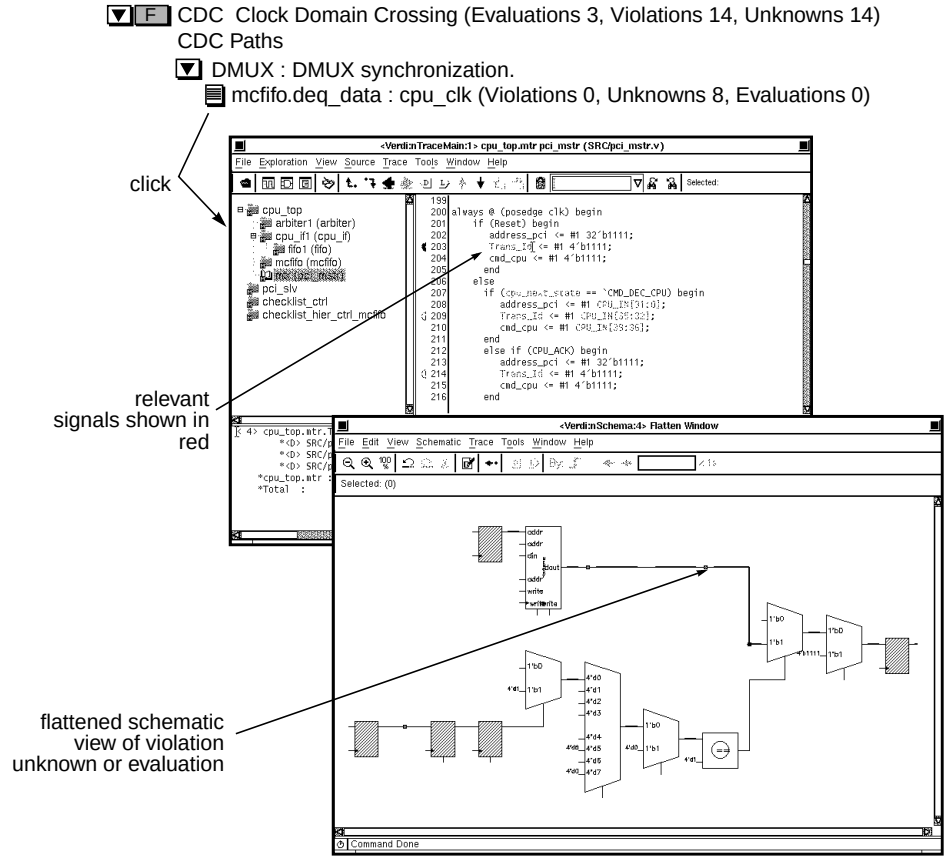
**Figure 28: Sample CDC check entry**



# VERDI/DEBUSSY DEBUG ENVIRONMENT INTEGRATION

If you select Debussy or Verdi as the source browser, the Debussy/Verdi application is used as a debug environment. The initial click of a source browser button ( ) starts up the application with the corresponding source code for the violation (red button), unknown (yellow button) or evaluation (green button). Relevant signals appear in red letters. A flattened schematic view showing the associated signals and design elements is also displayed (Figure 29). Each subsequent click on another source browser button resets the running Debussy/Verdi application with the source code and schematic views of the violation/unknown.

**Figure 29: Debussy/Verdi debug environment integration**



# ANALYZING REPORTS

The 0in shell creates a log (`0in.log`) and a detailed log (`0in_detail.log`) of the 0in session. In addition, the CDC compiler generates a general design report, a CDC design report and a CDC report.

## CDC DESIGN REPORT

The clock domain crossing design report (`0in_cdc_design.rpt`) shows detailed information about the clocks and clock groups in the design. It also shows summary information about design elements. The report has the following tables:

- ❏ Clock Group Summary.
- ❏ User-Specified Clock Groups.
- ❏ Inferred Clock Groups.
- ❏ General Design Information.
- ❏ Detailed Design Information.
- ❏ Port Domain Information.

### Clock Group Summary

The clock group summary table (Figure 36) shows the total counts for each type of clock group.

**Figure 30: Clock group summary**

```
Clock Group Summary for 'cpu_top'
================================
Total Number of Clock Groups              : 5
Number of User-Defined Clock Groups       : 2
Number of Inferred Clock Groups           : 3
```

Reports the number of clock groups.

## User-Specified Clock Groups

The user-specified clock groups table (Figure 31) shows the clock groups created using `set_clock` directives.

**Figure 31: User-specified clock groups**

```
User-Specified Clock Groups
=======================
Group     0 (14 Register Bits)
-----------
arb_clk
      arbiter1.clk
cpu_clk
      mcfifo.deq_clk
```

Reports the clocks that comprise the user-specified clock groups.

## Inferred Clock Groups

The inferred clock groups table (Figure 32) shows the remaining clock groups inferred by the compiler.

**Figure 32: Inferred clock groups**

```
Inferred Clock Groups
=====================
Group     0 (798 Register Bits)
-----------
cpu_clk
  pci2cpu_1bsync.clk
  cpu_if1.clk
    cpu_if1.fifo1.clk
  mcfifo.deq_clk

Group     1 (277 Register Bits)
-----------
pci_clk
  mcfifo.enq_clk
  mtr.clk

Group     2 (9 Register Bits)
-----------
arb_clk
  arbiter1.clk
```

Reports the clocks that comprise the clock groups inferred by the compiler.

## General Design Information

The general design information table (Figure 33) shows the total counts for each basic design element.

**Figure 33: General design information**

```
General Design Information
==========================
Number of blackboxes          = 1
Number of Registers           = 1084
Number of Latches             = 5
Number of RAMs                = 3
```

Reports the number each basic type of design element.

## Detailed Design Information

The detailed design information table (Figure 34) lists the basic design elements (except the registers).

**Figure 34: Detailed design information**

```
Detailed Design Information
===========================
Black Boxes:
------------------
    mtr.ram

Latches:
------------
    mtr.C_BE
    mtr.Req

RAMs:
---------
   mcfifo.fifo
   mtr.data_in_cpu
   cpu_if1.fifo1.fifo
```

Reports the names of basic design elements.

## Port Domain Information

The port domains table (see Figure 35) shows the clock-domains identified for the design's ports.

**Figure 35: Port Domain Information**

Port Domain Information

| Port | Direction | Constraint | Clock-Domain | Type |
|---|---|---|---|---|
| cpu_clk | input | Clock | —— | Inferred |
| pci_clk | input | Clock | —— | Inferred |
| arb_clk | input | Clock | —— | Inferred |
| rst | input | Reset | {pci_clk} | User |
| reqa | input | | {arb_clk} | Inferred |
| reqb | input | | {arb_clk} | Inferred |
| cmdack_inp | input | | —— | Inferred |
| transid_inp | input | | {cpu_clk} | Inferred |
| adr_data_inp | input | | {cpu_clk} | Inferred |
| AD_in | input | | —— | Inferred |
| Trdy | input | | {pci_clk} | Inferred |
| Gnt | input | | {pci_clk} | Inferred |
| Devsel | input | | {pci_clk} | Inferred |
| cmdreq_inp | input | | {pci_clk} | User |
| command_inp | input | | {cpu_clk pci_clk} | Inferred |
| gnta | output | | {arb_clk} | Inferred |
| gntb | output | | {arb_clk} | Inferred |

Shows information about the clock domains of the design's ports.

| | |
|---|---|
| Port | Port name. |
| Direction | Direction: input/output/inout |
| Constraint | Identifies clock and reset ports. |
| Clock-Domain | Clock domain of the port. {*clock*} indicates the primary clock for the domain. {*clock1* \| *clock2* \|...} indicates the clock domain is ambiguous. The relevant primary clocks are listed. async indicates the port is marked as an asynchronous port using the set_cdc_port_domain directive. |
| Type | Method used to determine the clock domain. User indicates the clock domain is assigned by the set_cdc_clock directive or the port is marked asynchronous (using set_cdc_port_domain). Inferred indicates checklist analysis identified the domain (or potential domains). |

## CLOCK DOMAIN CROSSING REPORT

The clock domain crossing report (`0in_cdc.rpt`) shows detailed information about the CDC signals and their schemes. The report has the following tables:

- ❏ CDC Summary.
- ❏ Violations.
- ❏ Unknowns.
- ❏ Evaluations.
- ❏ Memory Synchronization.
- ❏ Blackbox Crossing.
- ❏ Custom Synchronization Modules.
- ❏ Asynchronous Reset Detection.
- ❏ Transmitting Signals.

### CDC Summary

The CDC summary table (Figure 36) shows the counts for each type of violation, unknown and evaluation.

**Figure 36: CDC summary**

```
CDC Summary
=====================================================================
Violations (16)
---------------------------------------------------------------------
Single-bit signal does not have proper synchronizer: 16

Unknowns (13)
---------------------------------------------------------------------
DMUX synchronization: 3
Multiple-bit signal across clock domain boundary: 10

Evaluations (2)
---------------------------------------------------------------------
Single-bit signal synchronized by DFF synchronizer: 2
```

Reports the numbers of violations, unknowns and evaluations by category.

## CDC Violations

The CDC violations table (Figure 37) shows the paths that result in the CDC violations.

**Figure 37: CDC violations**

```
Violations
=========================================================================
Single-bit signal does not have proper synchronizer. (no_sync)
-------------------------------------------------------------------------------
pci_clk : start : cmdreq_inp (/examples/cdc/SRC/cpu_top.v : 19)
        clk_gate : end : cpu_if1.cmd_reg (/examples/cdc/SRC/cpuif.v : 61)
                via : cpu_if1.cmdreq_inp
        clk_gate : end : cpu_if1.transid_reg (/examples/cdc/SRC/cpuif.v : 62)
                via : cpu_if1.cmdreq_inp
        clk_gate : end : cpu_if1.adr_data_reg (/examples/cdc/SRC/cpuif.v : 63)
                via : cpu_if1.cmdreq_inp
        clk_gate : end : cpu_if1.cmdack_out (/examples/cdc/SRC/cpuif.v : 46)
                via : cpu_if1.cmdreq_inp
        clk_gate : end : cpu_if1.current_cpustate (/examples/cdc/SRC/cpuif.v : 81)
                via : cpu_if1.cmdreq_inp
pci_clk : start : command_inp (/examples/cdc/SRC/cpu_top.v : 21)
        clk_gate : end : cpu_if1.for_me (/examples/cdc/SRC/cpuif.v : 88)
                via : cpu_if1.command_inp
        clk_gate : end : cpu_if1.cmdack_out (/examples/cdc/SRC/cpuif.v : 46)
                via : cpu_if1.command_inp
cpu_clk : start : mcfifo.deq_addr (/examples/cdc/SRC/mcfifo.v : 23)
        clk_gate : end : cpu_if1.req (/examples/cdc/SRC/cpuif.v : 90)
                via : mcfifo.deq_data
                via : cpu_if1.pci_da_in
```

Reports details of each CDC violation.

| | |
|---|---|
| *clock* : start : *register* | Identifies the transmitting register and clock. |
| *clock* : end : *register* | Identifies the receiving register and clock |
| via : *signal*<br>via : *signal*<br>. . .<br>via : *signal* | Identifies the path from the transmitting register to the receiving register.. |

## CDC Unknowns

The CDC unknowns table (Figure 38) shows the paths that result in the CDC unknowns.

**Figure 38: CDC unknowns**

```
Unknowns
================================================================
DMUX synchronization. (dmux)
----------------------------------------------------------------
clk_gate : start : cpu_if1.fifo1.deq_addr (/examples/cdc/SRC/fifo.v : 25)
        pci_clk : end : mtr.address_pci (/examples/cdc/SRC/pci_mstr.v : 47)
                via : cpu_if1.fifo1.deq_data
                via : cpu_if1.pci_da_out
                via : mtr.CPU_IN
        pci_clk : end : mtr.Trans_Id (/examples/cdc/SRC/pci_mstr.v : 38)
                via : cpu_if1.fifo1.deq_data
                via : cpu_if1.pci_da_out
                via : mtr.CPU_IN
        pci_clk : end : mtr.cmd_cpu (/examples/cdc/SRC/pci_mstr.v : 37)
                via : cpu_if1.fifo1.deq_data
                via : cpu_if1.pci_da_out
                via : mtr.CPU_IN

Multiple-bit signal across clock domain boundary. (multi_bits)
--------------------------------------------------------------------------------
pci_clk : start : command_inp (/examples/cdc/SRC/cpu_top.v : 21)
        clk_gate : end : cpu_if1.cmd_reg (/examples/cdc/SRC/cpuif.v : 61)
                via : cpu_if1.command_inp
        clk_gate : end : cpu_if1.current_cpustate (/examples/cdc/SRC/cpuif.v : 81)
                via : cpu_if1.command_inp

cpu_clk : start : mcfifo.deq_addr (/examples/cdc/SRC/mcfifo.v : 23)
        clk_gate : end : cpu_if1.current_pcistate (/examples/cdc/SRC/cpuif.v : 82)
                via : mcfifo.deq_data
                via : cpu_if1.pci_da_in
        clk_gate : end : cpu_if1.adr_data_out[31:0] (/examples/cdc/SRC/cpuif.v : 43)
                via : mcfifo.deq_data
                via : cpu_if1.pci_da_in
```

Reports details of each CDC unknown.

| | |
|---|---|
| *clock* : start : *register* | Identifies the transmitting register and clock. |
| *clock* : end : *register* | Identifies the receiving register and clock |
| via : *signal*<br>via : *signal*<br>. . .<br>via : *signal* | Identifies the path from the transmitting register to the receiving register.. |

## CDC Evaluations

The CDC evaluations table (Figure 39) shows the paths that result in the CDC evaluations.

**Figure 39: CDC evaluations**

```
Evaluations
================================================================
Single-bit signal synchronized by DFF synchronizer. (two_dff)
------------------------------------------------------------------------------
clk_gate : start : cpu_if1.pci_req (/examples/cdc/SRC/cpuif.v : 74)
        pci_clk : end : mtr.CPU_REQ_temp (/examples/cdc/SRC/pci_mstr.v : 107)
                via : cpu_if1.pci_req
                via : mtr.CPU_REQ

pci_clk : start : mtr.fifo_rd (/examples/cdc/SRC/pci_mstr.v : 33)
        clk_gate : end : cpu_if1.pop_temp (/examples/cdc/SRC/cpuif.v : 103)
                via : mtr.fifo_rd
                via : cpu_if1.pci_pop

Custom CDC Scheme: custSynch1_2 (custom)
------------------------------------------------------------------
clk_gate : start : cpu_if1.pci_rd_fifo (/examples/cdc/SRC/cpuif.v : 78)
        cpu_clk : end : mcfifo.deq (/examples//cdc/SRC/cpu_top.v : 112)
                via : cpu_if1.pci_rd_fifo
```

Reports details of each CDC unknown.

| | |
|---|---|
| *clock* : start : *register* | Identifies the transmitting register and clock. |
| *clock* : end : *register* | Identifies the receiving register and clock |
| via : *signal*<br>via : *signal*<br>. . .<br>via : *signal* | Identifies the path from the transmitting register to the receiving register.. |

## Memory Synchronization

The memory synchronization table (Figure 40) shows the CDC paths that start or end at 2D arrays.

**Figure 40: Memory synchronization**

```
Memory Synchronization. (memory_sync)
------------------------------------------------------------------
cpu_clk : start : cpu_if1.fifo1.deq_addr (/examples/cdc/SRC/fifo.v : 25)
        pci_clk : end : mtr.CPU_IN (/examples/cdc/SRC/pci_mstr.v : 48)
                via : cpu_if1.fifo1.deq_data
                via : cpu_if1.pci_da_out
```

Reports details of each clock domain crossing to or from a memory element.

| | |
|---|---|
| *clock* : start : *register* | Identifies the transmitting register and clock. |
| *clock* : end : *register* | Identifies the receiving register and clock |
| via : *signal*<br>via : *signal*<br>. . .<br>via : *signal* | Identifies the path from the transmitting register to the receiving register.. |

## Blackbox Crossing

The blackbox crossing table (Figure 41) shows the CDC paths that end at black boxes (created using the `set_black_box` directive).

**Figure 41: Blackbox crossing**

```
Blackbox Crossing. (blackbox)
-------------------------------------------------------------------
pci_clk : start : pci2cpu_1bsync (/examples/cdc/SRC/cpu_top.v : 59)
        cpu_clk : end : cpu_if1.pci_ack (/examples/cdc/SRC/cpuif.v : 71)
                via : pci2cpu_1bsync.out
        cpu_clk : end : cpu_if1.pci_ack (/examples/cdc/SRC/cpuif.v : 78)
                via : pci2cpu_1bsync.out
cpu_clk : start : mtr.cpu_curr_state (/examples/cdc/SRC/pci_mstr.v : 43)
        pci_clk : end : pci2cpu_1bsync.in (/examples/cdc/SRC/cpu_top.v : 59)
                via : mtr.CPU_ACK
```

Reports details of each clock domain crossing to a black box.

| | |
|---|---|
| *clock* : start : *register* | Identifies the transmitting register and clock. |
| *clock* : end : *register* | Identifies the receiving register and clock |
| via : *signal*<br>via : *signal*<br>. . .<br>via : *signal* | Identifies the path from the transmitting register to the receiving register.. |

## Custom Synchronization Modules

The custom synchronization modules table (Figure 42) lists the modules designated as custom synchronizers (using `set_cdc_synchronizer custom`).

**Figure 42: Custom synchronization modules**

```
Custom Synchronization Modules
==============================
custSynch1_2
custSynch1_3
custSynch2_3
```

Lists the modules designated as custom synchronizers..

## Asynchronous Reset Detection

The asynchronous reset detection table (Figure 43) shows details about CDC signals used as asynchronous resets.

**Figure 43: Asynchronous reset detection**

```
Asynchronous Reset Detection ( top-level module )
=========================================
Asynchronous Reset Synchronizers Detected
-----------------------------------------------------------------
rstn (clk_bd)
rstd (clk_dg)

Asynchronous Reset with Missing Synchronizer
-----------------------------------------------------------------------
bd2.R3 (wrong clock : clk2)
bd2.R2 (wrong reset polarity : negedge U0.f2)
bd4.R3 (wrong clock : clk2)
bd4.R2 (wrong reset polarity : negedge U0.f2)
dg2.R1 (wrong reset polarity : negedge U0.f2)
dg2.R3 (wrong clock : clk2, wrong reset polarity : negedge U0.f2)
dg3.R1 (wrong reset polarity : negedge U0.f2)
```

Shows detailed information about CDC asynchronous reset signals.

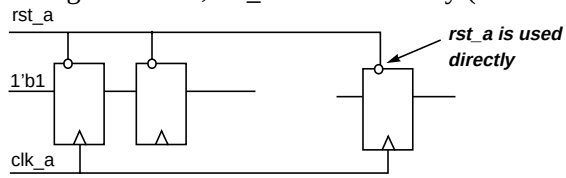
| | |
|---|---|
| Asynchronous Reset Synchronizers Detected | Identifies the CDC reset signals that are synchronized. |
| Asynchronous Reset with Missing Synchronizer | Identifies the CDC reset signals that are not synchronized properly and indicates the cause of the potential problem. |

## Asynchronous Reset with Missing Synchronizer

Asynchronous reset signals that are not synchronized properly are listed in the Asynchronous Reset with Missing Synchronizer section. Such signals either have no synchronizer or are not properly synchronized:

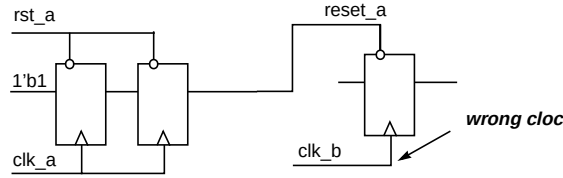
- no synchronizer

  Asynchronous reset detection reports no synchronizer if a reset signal is used to reset flops in a multiple clock design without using a synchronizer . In the following subcircuit, rst_a is used directly (without a synchronizer):



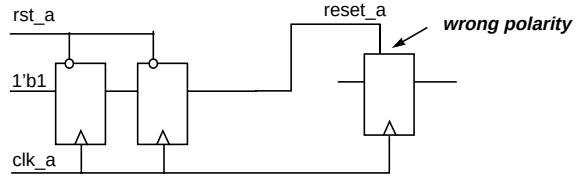

- wrong clock

  Asynchronous reset detection reports wrong clock if an asynchronous reset signal is used to reset logic in another clock domain. In the following subcircuit, the reset_a signal is used incorrectly to reset logic in a clock domain other than the domain of clk_a.




- wrong reset polarity

  Asynchronous reset detection reports a wrong reset polarity violation if an active high asynchronous reset is used as an active low reset or if an active low asynchronous reset is used as an active high reset (i.e., the reset has the wrong polarity). In the following subcircuit, the active high reset_a signal is used as an active low reset



## Transmitting Signals

The transmitting signals table (Figure 44) lists the sources of signals that cross clock domains.

**Figure 44: Transmitting signals**

All Transmitting Signals (9)

| Name | Width | Location |
|---|---|---|
| cpu_if1.pci_req | 1 | /examples/cdc/SRC/cpuif.v:74 |
| mtr.fifo_rd | 1 | /examples/cdc/SRC/pci_mstr.v:3 |
| cmdreq_inp | 4 | /examples/cdc/SRC/cpu_top.v:21 |
| cpu_if1.pci_rd_fifo | 1 | /examples/cdc/SRC/cpu_top.v:21 |
| cpu_if1.cmdreq_out | 1 | /examples/cdc/SRC/cpuif.v:45 |
| cpu_if1.req | 1 | /examples/cdc/SRC/cpuif.v:90 |
| mcfifo.deq_addr | 5 | /examples/cdc/SRC/mcfifo.v:23 |
| cpu_if1.fifo1.deq_addr | 5 | /examples/cdc/SRC/fifo.v:25 |

Lists the sources of signals that cross clock domains..

| | |
|---|---|
| Name | Signal name. |
| Width | Number of bits. |
| Location | Source code location. |

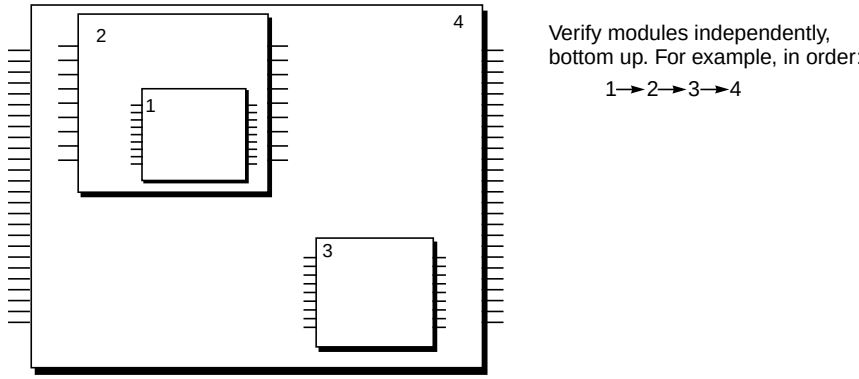
# HIERARCHICAL CDC VERIFICATION

Hierarchical CDC verification is a methodology for partitioning the CDC verification process. Once various submodules are verified, they can be considered as properly synchronized and can be left out of the analysis of their parent modules. Similarly, a submodule can be assumed to be properly synchronized in order to verify (and debug) a higher-level module; the submodule can be verified and debugged later.

The common reason to use a hierarchical CDC methodology is to verify large designs, which can run out of memory or consume unacceptable run times.

Figure 45 shows an example of a hierarchical CDC verification process. Four modules are independently verified and debugged using a bottom-up approach.

**Figure 45: Hierarchical CDC verification**




## VERIFYING THE PARENT MODULE

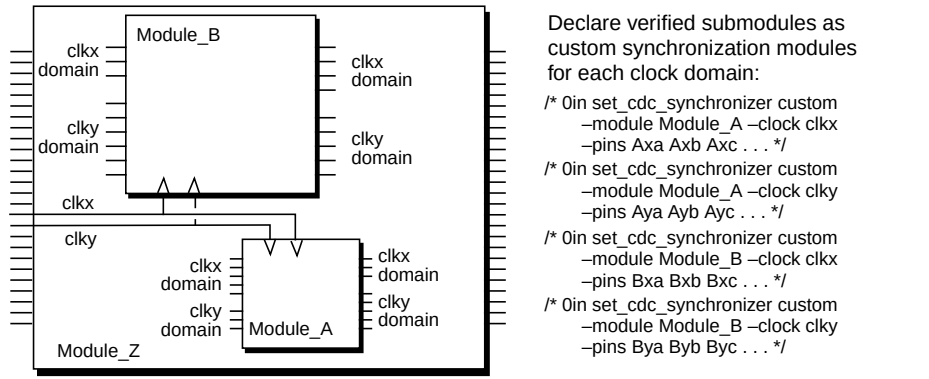
When CDC checking a parent module, you identify the submodules that are pre-verified or need to be skipped. To do this, you can declare the logic in these submodules to be properly synchronized (i.e., the modules are considered custom synchronizers) and specify the clock domains of their pins.

set_cdc_synchronizer custom

The set_cdc_synchronizer directive sets special properties of certain synchronizers (see “set_cdc_synchronizer” on page 54). When you specify the custom argument, the directive identifies a module to be considered a custom synchronizer.

When performing hierarchical CDC verification, the top module at each step is selected to partition the CDC verification process. In particular, each of these modules is clocked by multiple clocks (otherwise, it would not include CDC signals). When you use set_cdc_synchronizer with the custom option to declare a multi-clocked custom synchronizer, you must specify the corresponding clock signal to identify the clock domain to which the custom synchronizer applies. You also must identify which of the module’s pins correspond to that clock domain (Figure 46). In particular, each module to be skipped has one set_cdc_synchronizer directive for each of the module’s clocks.

**Figure 46: Verifying a hierarchical module**




## 0in_cdc_hier_ctrl_*module*.v

Specifying the set_cdc_synchronizer directives for the pre-verified submodules can be a significant task—if done manually. The CDC compiler provides a semi-automatic method for simplifying the process. The –hier option to the CDC compiler creates a 0in_cdc_hier_ctrl_*module*.v file (Figure 47), where *module* is the name of the target design specified with the –d *module* argument. If *module* is not the top-level module in the file list, an additional file is created for the top-level module. You then specify this *CDC hierarchy control file* (as a CDC control file) when running the CDC compiler on the parent module (along with the CDC

hierarchy control files for the other pre-verified submodules). The CDC compiler will not spend time analyzing these submodules. For example, assume mod_A and mod_B are pre-verified submodules of mod_Z. Then the following steps run the CDC compiler on the hierarchy at the mod_Z level.

```
shell prompt> cp 0in_cdc_hier_ctrl_mod_A.v cdc_hier_ctrl_mod_A.v
shell prompt> cp 0in_cdc_hier_ctrl_mod_B.v cdc_hier_ctrl_mod_B.v
shell prompt> 0in -cmd cdc filelist -d mod_Z \
-ctrl cdc_hier_ctrl_mod_A.v \
-ctrl cdc_hier_ctrl_mod_B.v
```

**Figure 47: 0in_cdc_hier_ctrl_*module*.v example**

```
module cdc_hier_ctrl_mcfifo;
    // Ports belong to multiple clocks domains
    /* xx0in set_cdc_synchronizer custom
        -module mcfifo -pins reset
        -clock deq_clk enq_clk
    */
    /* 0in set_cdc_synchronizer custom
        -module mcfifo -clock enq_clk
        -pins
            enq
            enq_data
    */
    /* 0in set_cdc_synchronizer custom
        -module mcfifo -clock deq_clk
        -pins
            deq
            deq_data
    */
endmodule
```

### Pins with Ambiguous Clock Domains

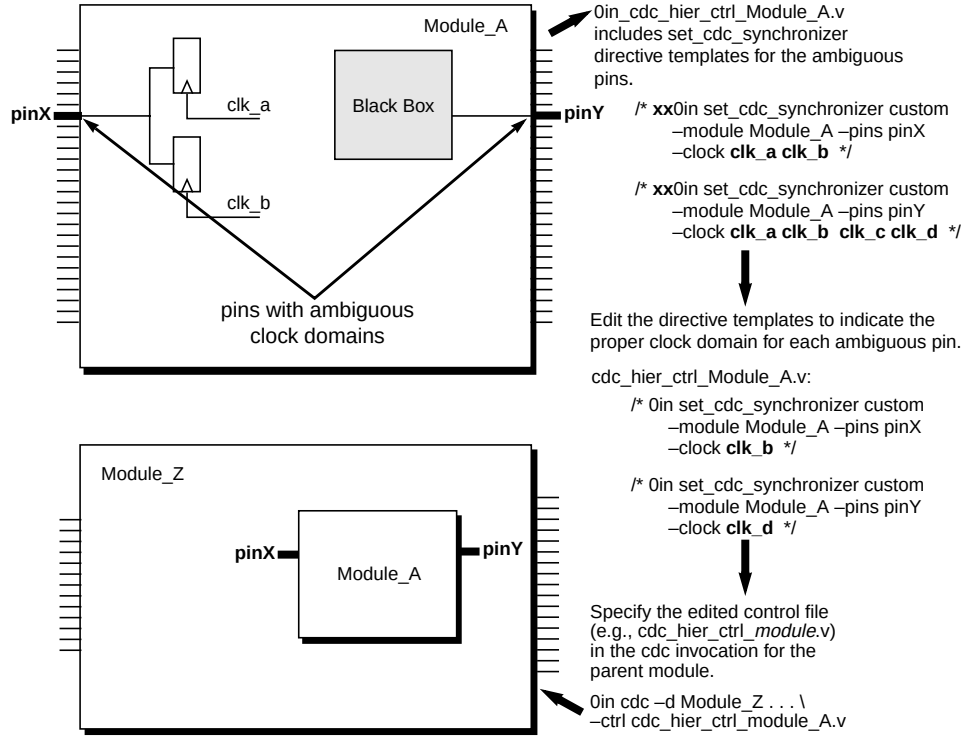
Using the above procedure, hierarchical CDC verification is automated—almost. Manual intervention is needed in certain situations. The CDC compiler cannot determine the clock domain of a pin if the pin is connected to a black box or directly to logic clocked by multiple clocks (Figure 48).

In the CDC hierarchy control file, each pin with ambiguous clock domain is identified with a set_cdc_synchronizer custom directive template of the following form:

```
// Ports belong to multiple clocks domains
/* xx0in set_cdc_synchronizer custom -module current_module
    -pins pin_with_ambiguous_clock_domain
    -clock  possible_clock possible_clock . . .  */
```

You must edit the CDC hierarchy control file and resolve the ambiguous clock domain declarations for these pins.

**Figure 48: Handling pins with ambiguous clock domains**



# CDC PROMOTION

The CDC checkers are modules that ensure that data crossing clock domain boundaries are transmitted and received correctly. The CDC checkers are implemented the same as other CheckerWare checkers: They monitor the design during assertions in simulation. Their checks can be targets for formal verification.They are instantiated directly by using checker directives.

The CDC compiler generates a file named 0in_cdc_ctrl.v containing CDC checker directives. For each signal and data bus that crosses clock domain boundaries, CDC analysis generates a checker directive template for the signal or data bus. You can modify the directive templates to create checker directives configured for your particular designs. The CDC checkers verify the associated transfer protocols when the target design is operating.You also can manually place CDC checkers in a target design.

## DATA PREPARATION

Using information from the CDC report, determine which cross-domain control signals and data buses need synchronization monitoring. Each such signal or bus needs a separate CDC checker instantiated. Select the checker type for the method used to synchronize the signal or bus. cdc_sync, cdc_sample, cdc_dsel, cdc_handshake_data, cdc_hamming_distance_one or cdc_fifo.

To instantiate a CDC checker, use CheckerWare checker directive syntax. You can place a CDC checker directive in the appropriate HDL module or you can place the directive in a checker control file (and use the –module option). For example, the following directive placed in a module instantiates a cdc_sync checker:

```
/*0in cdc_sync –tx_var tx_var –tx_clock tx_clk –tx_reset tx_rst
       –tx_areset 1'b0 –tx_min 4 */
```

## CDC Checker Types

The *CheckerWare Data Book* has data sheets for the CDC checker types. The following summaries show the directive syntaxes for these checker types.

[<] {**cdc_dsel** | dsel}
[–tx_data *tx_data_variable*]
[–tx_min *tx_min_constant* | –tx_min_check off]
[–tx_data_stable off] [–rx_data_stable off]
[–tx_data_select *tx_dsel_signal*]
[–tx_clock *tx_clock*] [–tx_reset *tx_reset*]
[–rx_data_select *rx_dsel_signal*]
[–rx_clock *rx_clock*] [–rx_reset *rx_reset*]
[*standard_option...*]

Ensures that a signal between two clock domains, where the transmitter's data select signal drives the select input of a data multiplexor in the receiver, is held stable enough for the signal to be sampled reliably by the receiver and ensures that the data remains stable while the data select signal asserts.

[<] {**cdc_fifo** | cdcf}
–enq *enq_signal* –deq *deq_signal* –depth *depth_constant*
[–wr_ptr *write_pointer_variable* | –wr_ptr_check off]
[–rd_ptr *read_pointer_variable* | –rd_ptr_check off]
[–enqueue off] [–dequeue off]
[–tx_clock *tx_clock*] [–tx_reset *tx_reset*]
[–rx_clock *rx_clock*] [–rx_reset *rx_reset*]
[*standard_option...*]

Ensures that the write and read pointers of an asynchronous FIFO change by hamming distance one, and ensures that the FIFO does not overflow or underflow.

[<] {**cdc_hamming_distance_one** | cdc_hamming1}
[–tx_var *tx_variable*] [–tx_min *tx_min_constant*]
[–hamming_one off] [–tx_clock *tx_clock*]
[–tx_reset *tx_reset*] [*standard_option...*]

Ensures that the data crossing from transmit clock to receive clock domain changes by only one hamming distance and that it remains stable for a specified tx_min clock cycles.

[<] {**cdc_handshake_data** | cdc_hsd}
[–tx_data *tx_data_variable*]
[–tx_min *tx_min_constant*] [–rx_min *rx_min_constant*]
[–turnaround_min *turnaround_min_constant*]
[–turnaround_max *turnaround_max_constant*]
[–cdc_handshake off] [–data_stable off]
[–tx_valid_assert_until_tx_done_assert off]
[–tx_done_assert_until_tx_valid_deassert off]
[–tx_valid_deassert_until_tx_done_deassert off]
[–rx_valid_assert_until_rx_done_assert off]
[–rx_done_assert_until_rx_valid_deassert off]
[–tx_valid *tx_valid_signal*] [–tx_done *tx_done_signal*]
[–rx_valid *rx_valid_signal*] [–rx_done *rx_done_signal*]
[–tx_clock *tx_clock*] [–tx_reset *tx_reset*]
[–rx_clock *rx_clock*] [–rx_reset *rx_reset*] [*standard_option...*]

Ensures that the handshaking protocol between transmitter and receiver is correctly followed, and ensures that the transmit data remains stable in data transfer window.

[<] {**cdc_sample** | sample}
–tx_var *tx_variable* –rx_var *rx_variable*
[–tx_clock *tx_clock*] [–tx_reset *tx_reset*]
[–rx_clock *rx_clock*] [–rx_reset *rx_reset*]
[–setup off] [–hold off] [–constraint] [*standard_option...*]

Ensures that data between two clock domains remain stable in the transmit domain for one transmit clock cycle before, and one transmit clock cycle after, it is sampled in the receive domain.

[<] {**cdc_sync** | sync}
[–tx_var *tx_variable*] [–stable off] [–tx_min *tx_min_constant*]
[–tx_clock *tx_clock*] [–tx_reset *tx_reset*] [*standard_option...*]

Ensures that a clock domain crossing signal from a faster clock to a slower clock is held stable long enough for the signal to be sampled reliably by the receiver.

# RUNNING ASSERTIONS IN SIMULATION

Run assertions in simulation as described in the *Assertions in Simulation User Guide*—a detailed description of assertions in simulation is beyond the scope of this manual. The following example illustrates the steps for running simulation with 0-In assertions (using the Verilog-XL simulator).

1. Compile assertion logic.

   ```
   shell prompt> more check.scr
   ccl –f verilog_run.f –d pci_slv_wrap –ctrl 0in_cdc_ctrl.v

   shell prompt> 0in check.scr
   ```

   The ccl command compiles assertion logic for the simulation.

2. Run simulation with 0-In assertions.

   ```
   shell prompt> more verilog_run.f
   pci_mstr.v pci_slv.v pci_slv_wrap tb_bus_to_pci.v

   shell prompt> verilog –f 0in_sim.arg –f verilog_run.f
   ```

   The Verilog-XL simulator runs the simulation with the assertions. Firings data are printed to the output, however, the quickest method for examining firings data is with the 0-In View graphical browser.

## Running 0-In View

Simulation with 0-In assertions generates a viewer database (named 0in_checksim.db by default). You can run 0-In View and load the database in a single invocation as shown below:

```
shell prompt> 0in_view –db 0in_checksim.db
```

You can expand the firing entries as shown in Figure 49. Details of a sample CDC checker firing are shown in Figure 50.

**Figure 49: 0-In View showing CDC checker firing entry**

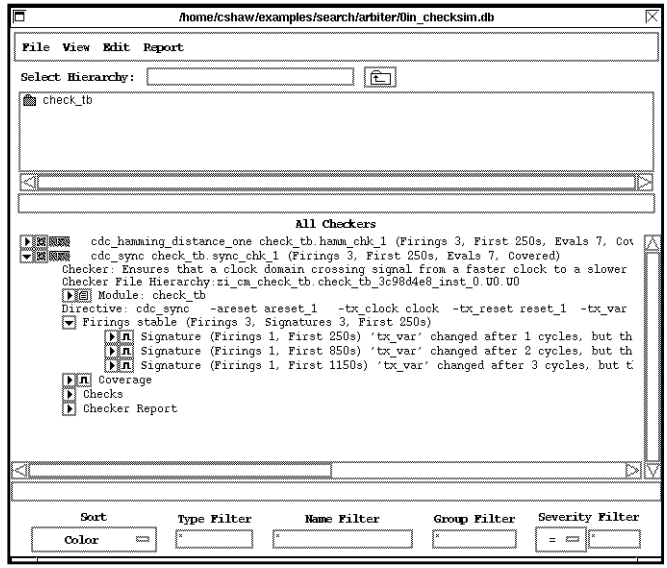



**Figure 50: Sample CDC checker firing entry**

cdc_sync check_tb.sync_chk_1 (Firings 3, First 250s, Evals 7, Covered)

Checker: Ensures that a clock domain crossing signal from a faster clock to a slower clock is held stable long enough for the signal to be sampled reliably by the receiver.

Checker File Hierarchy: zi_cm_check_tb.check_tb_3c98d4e8_inst_0.U0.U0

Module: check_tb

Directive: cdc_sync –areset areset_1 –tx_clock clock–tx_reset reset_1 –tx_var tx_var_1 –tx_min 4 –active active_1 –stable_fire stable_fire_1 –name sync_chk_1

Firings stable (Firings 3, Signatures 3, First 250s)

- Signature (Firings 1, First 250s) 'tx_var' changed after 1 cycles, but the specified minimum time to remain constant is 4 cycles.
- Signature (Firings 1, First 850s) 'tx_var' changed after 2 cycles, but the specified minimum time to remain constant is 4 cycles.
- Signature (Firings 1, First 1150s) 'tx_var' changed after 3 cycles, but the specified minimum time to remain constant is 4 cycles.
  - Time 1150s
    Message:'tx_var' changed after 3 cycles, but the specified minimum time to remain constant is 4 cycles.

Coverage

Checks

Checker Report

# 4 CDC ANALYSIS

# CHAPTER 5: CDC-FX

Signals that cross clock domain “boundaries” must be properly synchronized and they must obey their transmit protocols. But, properly synchronized CDC signals that obey protocol rules do not guarantee valid functionality. If any CDC signal does not hold steady during the setup and hold time of its receiving register, the register can become *metastable*—its output can settle at random to a value that is different from the RTL simulated value. In effect, data values that cross clock domains can be advanced or delayed randomly relative to RTL simulation.

If the receiving logic is not specifically designed to tolerate these *metastability effects*, functional errors can occur. Unfortunately, standard simulation cannot accurately model metastability in a design. An extension to standard functional verification is needed to model the effects of metastability in a design.

The 0-In CDC-FX product does just that. CDC-FX runs standard simulation with metastability injected into the circuit. In effect, CDC analysis and CDC-FX metastability injection together provide a complete CDC verification methodology.

# METASTABILITY EFFECTS

A clock domain is a part of a design that operates with a single clock. A clock domain crossing (CDC) signal is a signal originating in a clock domain that crosses the boundary with another domain (whose clock is asynchronous to the original clock) and is then sampled by a register in that asynchronous clock domain.

When the active edge of a CDC signal's transmit clock is too close to the active edge of the receive register's clock, a phenomenon called *metastability* can occur. The register's output settles to an unpredictable value. Metastability can occur if the clocks are asynchronous or if they are synchronous but have unpredictable or drifting skews. Every type of bi-stable storage element is susceptible to metastability. Logic subject to metastability must be designed to "tolerate" its effects.

In hardware, the effects of metastability are unpredictable, but normal simulation provides predictable results. As a result, normal simulation does not accurately model the hardware implementation when metastability is present. To ensure a circuit design is immune to metastability effects, functional verification methods must incorporate technology beyond normal simulation.

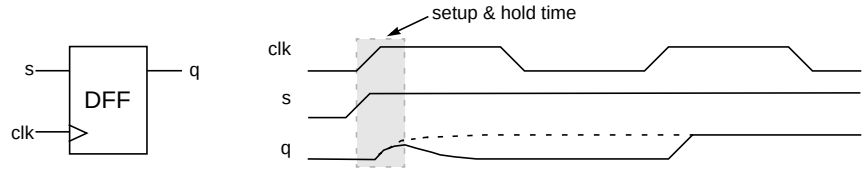
## METASTABLE REGISTERS

The next sections explain how registers in hardware exhibit metastability and how registers function in simulation when the conditions for metastability are present.

### Metastable Registers in Hardware

Consider the 1-bit CDC signal s that is sampled by the flip-flop DFF in Figure 51. Since s comes from a different clock domain, its value can change at any time with respect to DFF's clock (clk). If the value of s is not held constant at 0 or 1 during DFF's setup and hold time, the output (q) might assume an intermediate voltage for an indeterminate amount of time. Then, q "settles" randomly to either 0 or 1. The flip-flop is said to be *metastable* for that interval.

**Figure 51: Metastable flip-flop**



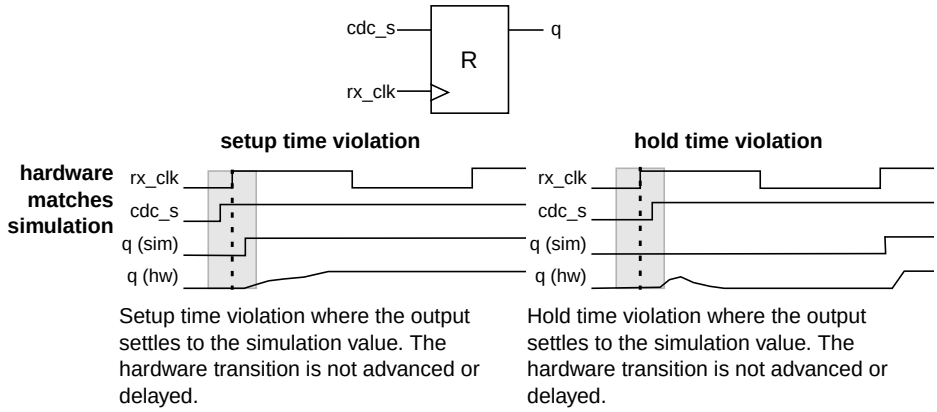

## Metastable Registers in Normal Simulation

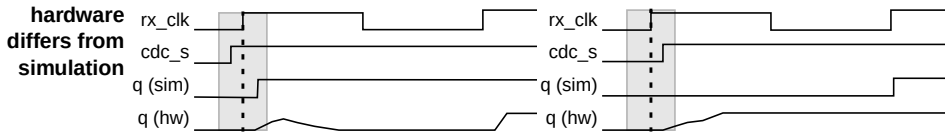
For cycles in which a single-bit flip-flop is metastable, its output settles randomly to 1 or 0. Alternately, normal simulation handles single-bit registers predictably:

- ❏ If the simulated input switches value before the simulated clock edge, the simulated output switches value at the clock edge.
- ❏ If the simulated input switches value after the simulated clock edge, the simulated output does not switch value.

So, for both setup time and hold time violations, two results are possible: the hardware output value *matches* the value predicted by simulation or the hardware value *differs* from the value predicted by simulation. The resulting four metastability scenarios are illustrated in Figure 52.

**Figure 52: Four metastability scenarios**






Setup time violation where the output settles to the complement of the simulation value. The hardware transition is **delayed** by one cycle.

Hold time violation where the output settles to the complement of the simulation value. The hardware transition is **advanced** by one cycle.

To summarize the comparison of hardware and normal simulation behaviors:

- ❏ When a setup time violation occurs, the hardware transition is *delayed* randomly by one cycle.
- ❏ When a hold time violation occurs, the hardware transition is *advanced* randomly by one cycle.

*Important!* Metastability models can be generalized for multibit registers by treating them as aggregate single-bit registers.

## Modeling Metastable Registers

Metastability in a register can be modeled by "injecting" metastability at selected cycles. Three methods of metastability injection methods are as follows:

- ❏ Invert all metastable outputs.

  All metastable bits are inverted in the current cycle. This method matches normal simulation for cycles with setup time violations. For cycles with hold time violations, it advances the transition to the current cycle.

- ❏ Do not invert metastable outputs.

  All metastable bits remain unchanged in the current cycle. This method matches normal simulation for cycles with hold time violations. For cycles with setup time violations, it delays the transition until the next cycle.

- ❏ Invert metastable outputs at random.

  Metastable bits are inverted at random in the current cycle. This method has a decreased chance of matching normal simulation or the other metastability injection methods.

Random metastability injection is the most robust method. It includes the behaviors of the other methods as possible behaviors. But it also simulates cases where metastable register outputs assume intermediate values for cycles with metastability. This simulated behavior is not reproduced in normal simulation or in the other simulation with metastability injection methods.

If the circuit's hardware implementation is prone to incorrect behavior because it is intolerant of metastability, simulation with random metastability injection has a high probability of detecting the problem. Normal simulation can uncover many such problems, but it cannot provide a high level of confidence that the hardware implementation will be completely tolerant of metastability effects.

# METASTABILITY INJECTION

Metastability-injected simulation is an extension to normal RTL simulation that injects random metastability into the circuit's behavior. CDC-FX metastability-injected simulation is similar to simulation with assertions. But, it uses special CDC-FX checkers to inject metastability into the circuit during simulation. These checkers also monitor the metastability logic and report coverage of metastability effects during simulation.

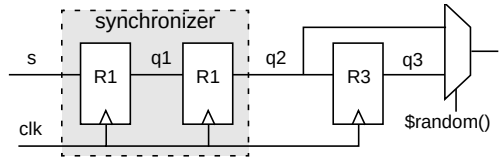
Various ways of injecting metastability during simulation are available.

## COMMON METHODS

### Pseudorandom Delay Insertion

A common method of modeling metastability in RTL simulation is to introduce pseudorandom delays in CDC signals at the synchronizers. For example, Figure 53 shows a two-register synchronizer with a MUX that selects a 3-cycle delayed value (instead of the normal 2-cycle delayed value) in a pseudorandom manner. End-to-end functional simulation with these types of pseudorandom delays helps to verify that the design works properly in the presence of CDC metastability.

**Figure 53: Synchronizer with pseudorandom 1-cycle delay on output**




This method has the following limitations:

- It is incomplete, because it only models one of the four possible metastability scenarios.
- It models metastability only across synchronized CDC signals.
- Results are difficult to predict, because metastability is introduced into the simulation at random.

- Synchronization violations are difficult to debug, because the offending metastability can occur any number of cycles before the cycle in which the simulation check first fails (and irrelevant metastability can occur along the way).

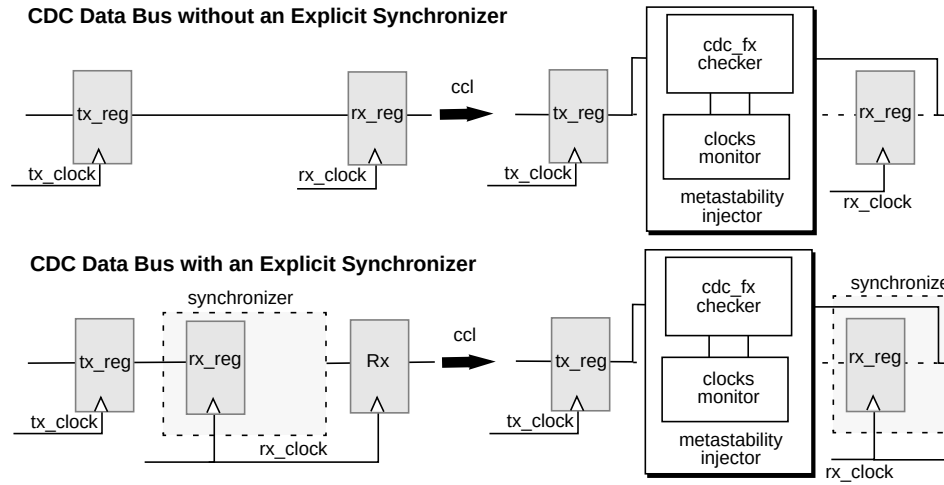
### Clock Jitter

Another method of modeling CDC metastability effects in RTL simulation is to jitter the clocks in simulation and see if end-to-end functional simulation still works when the clock phase relationships change. This method is good for verifying that the design works properly in the presence of clock jitter and clock skews. But, this method does not completely model CDC metastability effects.

## CDC-FX METASTABILITY-INJECTED SIMULATION

For metastability-injected simulation, circuitry for injecting metastability on a CDC signal or data bus is inserted between the transmitting register and the first register along the path to the receiver. For a path without a synchronizer, this is the receiving register itself. For a path with a synchronizer, this is the first register in the synchronizer (see Figure 54).

**Figure 54: Metastability injector for a CDC data bus**



# 5 CDC-FX

The `ccl` compiler generates the checker logic for running metastability-injected simulation. To do this, it adds metastability injection logic for each affected CDC signal or bus in two parts:

- CDC clocks monitor logic

  Glue logic used to extract load and timing conditions from the circuit.

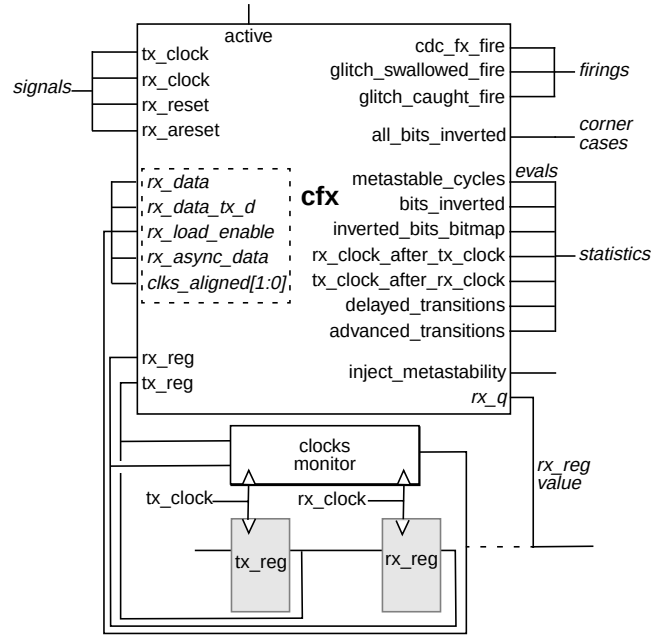
- `cdc_fx` checker

  Checker that monitors conditions that might cause metastability on a CDC bus and injects metastability at the receiver outputs.

The `cdc_fx` checker maintains statistics and corner-case counts for the metastability and CDC effects modeled by the transformed circuit. This checker has a default-off check (`cdc_fx`) that fires in cycles for which metastability might occur (for use on data buses that are presumed to be synchronized properly).

# CDC_FX CHECKER

Injects metastability at the output of a receive register.




**checker class:**
user
**default name**:
*rx_reg_name*
**checks**:
cdc_fx (Off)
**tx_clock:**
inferable from *tx_reg*
**rx_clock, rx_reset, rx_areset**
inferable from *rx_reg*

[<] {cdc_fx | cfx}
–rx_reg *rx_register_variable* [–tx_reg *tx_register_variable*]
[–tx_clock *tx_clock*] [–rx_clock *rx_clock*]
[–rx_reset *rx_reset*] [–rx_areset *rx_areset*]
[–cdc_fx] [–glitch_swallowed] [–glitch_caught]
[*standard_option*...*]

*except –clock, –reset and –areset options

---

### Required Arguments

–rx_reg *rx_register_variable*

Receiving register in the receive clock domain.

### Inferable Options

[–tx_reg *tx_register_variable* ]

Transmitting register in the transmit clock domain. Default: 0.

[–tx_clock *tx_clock*]

Transmit clock. If unspecified, it is inferred from *tx_register_variable*.

[–rx_clock *rx_clock*]

Receive clock. If unspecified, it is inferred from *rx_register_variable*.

[–rx_reset *rx_reset*]

Receive synchronous reset. If unspecified, it is inferred from *rx_register_variable*.

[–rx_areset *rx_areset*]

Receive asynchronous reset. If unspecified, it is inferred from *rx_register_variable*.

## Checks and Related Options

**Cdc_fx check**, default Off

[–cdc_fx]

Ensures the data output of the transmitting register is stable in the receiving register's metastability window. Typically, the data output of the transmit register *can* change value in the receive register's metastability window. This change can cause metastability of the receiving register's output, but the circuit is hopefully tolerant of this effect. So by default, the cdc_fx check is off.

However, an ad hoc synchronizer might presume the transmit logic prevents the transmitting register from changing value in the receiving register's metastability window—so the ad hoc synchronizer logic assumes the receiving register cannot become metastable. In this case, the cdc_fx check can be used to verify the transmit value is held stable whenever the transmit/receive clocks are aligned. Here, the cdc_fx check is a transmit protocol check for the ad hoc synchronizer.

This check fires on the transition of clks_aligned to 0.

**Firing message:** Data changed in metastability window.

**Glitch_caught check**, default Off

[–glitch_caught]

Ensures metastability injection does not cause a glitch at the rx_reg input to be reflected as a pulse at the rx_reg output unless the glitch is also caught in simulation (Figure 55). The check fires at the second active receive clock edge after the glitch. The check reports a bitmap showing those rx_reg bits that had a glitch caught.

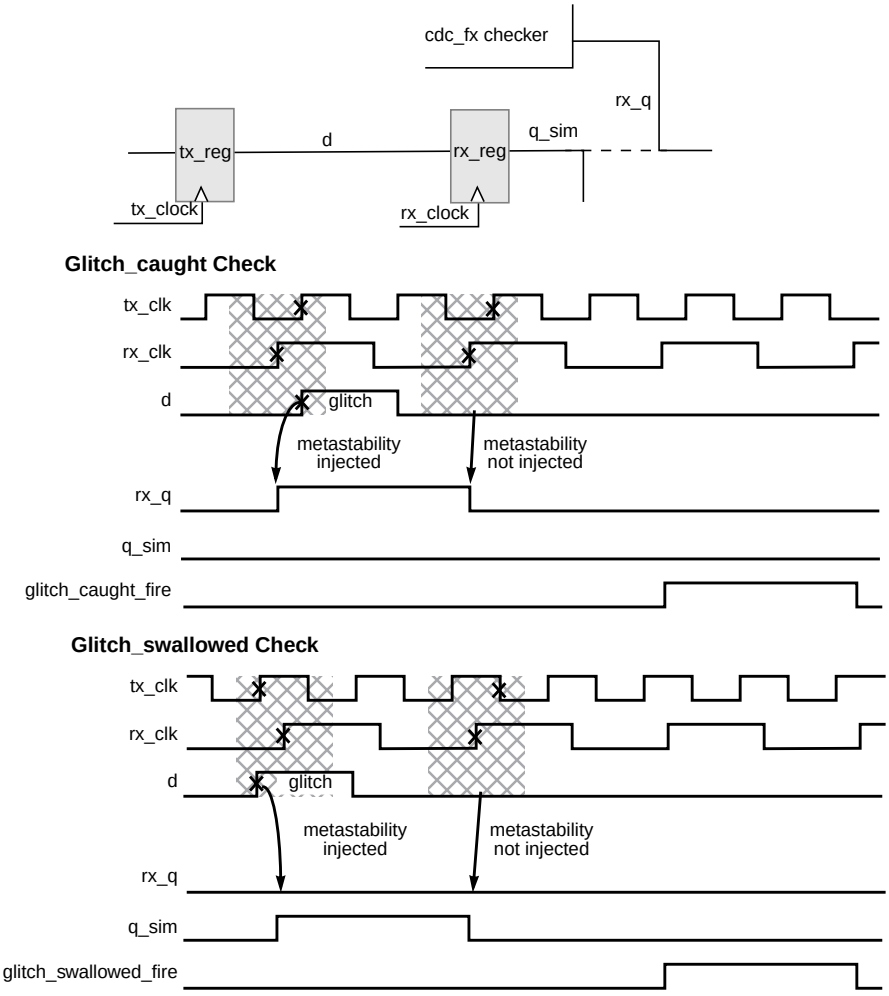
**Firing message:** Glitch detected at the receiver output. Glitched output bitmap: *glitch_caught_bitmap*.

**Glitch_swallowed check**, default Off

[–glitch_swallowed]

Ensures metastability injection does not cause a glitch at the rx_reg input to be swallowed (i.e., missed) at the rx_reg output unless the glitch is also swallowed in simulation (Figure 55).

**Figure 55: Transmit protocol checks for glitches**



The check fires at the second active receive clock edge after the glitch. The check reports a bitmap showing those rx_reg bits that had a glitch swallowed.

**Firing message:** Glitch swallowed by receiver output. Swallowed glitch bitmap: *glitch_swallowed_bitmap*.

### Corner Cases

- All Bits Metastable — non-zero if every output bit of the receive register has had metastability inserted.

### Statistics

- Metastable Cycles (Evals) — number of cycles of the receive clock in which the receiver is prone to metastability (i.e., clocks aligned and at least one bit transitioning).
- Bits Inverted — number of bits of the receive register output that had metastability inserted (i.e., numbers of 1's in the inverted bits bitmap).
- Inverted Bits Bitmap — bitmap of the bits that had metastability injected.
- Rx_clock After Tx_clock — number of cycles of the receive clock in which the receiver is prone to metastability and the receive clock edge goes active after the transmit clock edge.
- Tx_clock After Rx_clock — number of cycles of the receive clock in which the receiver is prone to metastability and the transmit clock edge goes active after the receive clock edge.
- Delayed Transitions — number of times metastability injection delayed a transition until the next cycle.
- Advanced Transitions — number of times metastability injection advanced a transition to the current cycle.

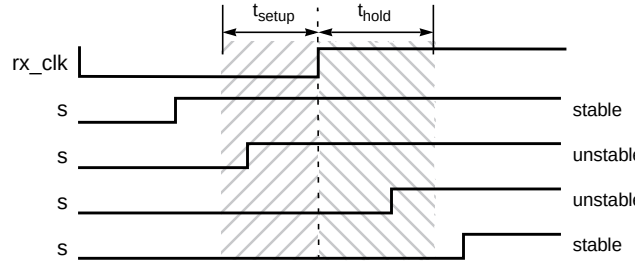
# SPECIFYING METASTABILITY WINDOWS

*Metastability windows* indicate the transmit/receive clock edge relations that determine when metastability can occur. They are specified to ccl using set_cdc_fx_metastability_window directives.

## SETUP AND HOLD PERIODS

Figure 56 shows *setup* and *hold time* periods around a register's clock edge along with sample waveforms for an input to the register (s). The input signal is *stable* if is it is held at a constant 0 or 1 value during the setup and hold periods. The signal is *unstable* if it changes during these periods. If the signal input to a register is unstable, the register can become metastable.

**Figure 56: Setup and hold times for a register input**




## METASTABILITY WINDOWS

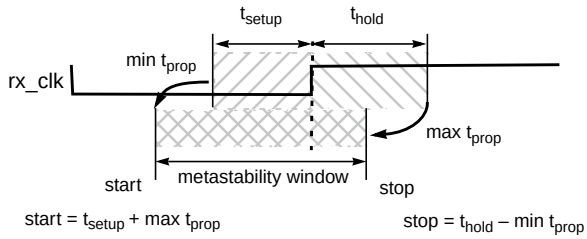
The setup and hold times determine receive clock cycles for which a register can become metastable—based on the active edge of the receive clock and the value of the signal *at the input port of the register.* In hardware, however, this input port switches value after the output port driving the signal (in the transmit clock domain) switches and the new value propagates to the input port (in the receive clock domain). This propagation delay ($t_{prop}$) has physical bounds:

$$\min t_{prop} \leq t_{prop} \leq \max t_{prop}$$

Accounting for propagation delay identifies a *metastability window* relative to each active edge of the receive clock, as shown in Figure 57. A metastability window defined this way assumes the worst case events:

- ❏ CDC signal propagates as slowly as possible (max $t_{prop}$) and violates the setup time ($t_{setup}$).
- ❏ CDC signal propagates as quickly as possible (min $t_{prop}$) and violates the hold time ($t_{hold}$).

**Figure 57: Metastability window**

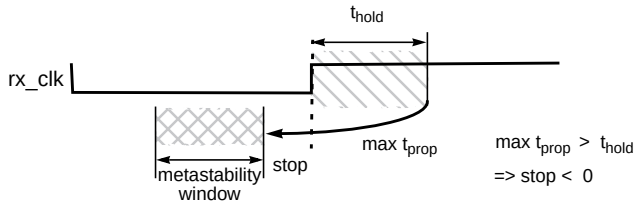



The start value is the number of time units *before* the active edge of the receive clock that the metastability window "opens". The stop value is the number of time units *after* the active edge of the receive clock that the metastability window "closes".

## Negative Stop Value

For many technologies, the minimum propagation delay of a CDC signal might exceed the hold time for the receive register input. In these cases, the stop value can be less than zero (see Figure 58).

**Figure 58: Negative stop time**



As described below, the metastability window for a receive clock will determine for which receive clock cycles the active edges of the transmit and receive clock are “aligned”. These are cycles in which the receive registers are prone to metastability.

## Metastability Windows Usage

Note the following information about metastability windows:

- Except for the default case, the metastability windows are not set automatically by software. You set up metastability windows based on your knowledge of the hardware technology and characteristics of the design—by specifying their `start` and `stop` values. However, you do not need to specify setup/hold times or propagation delay ranges.
- Each pair of transmit/receive clocks has its own metastability window (either specified or default). In particular, a receive clock might have multiple windows.
- If the duration of a metastability window is sufficiently large, every active edge of the corresponding transmit clock falls inside the window. In this case, simulation with metastability injectors randomly inserts metastability effects *every clock cycle* the register changes value.
- A common metastability verification methodology is to start with large metastability windows (for example, the default case). Then, successively narrow the windows and focus analysis on select CDC paths. This way, you can analyze the worst case scenario (so no bugs might be missed). Then, after eliminating “false” violations, you can identify real problems caused by metastability intolerance.
- Metastability windows are used for metastability-injected simulation. They have no counterpart in hardware. In hardware, a changing CDC signal either does or does not result in the receive register going metastable and the hardware value either does or does not agree with the value used in plain simulation.

## clks_aligned[1:0]

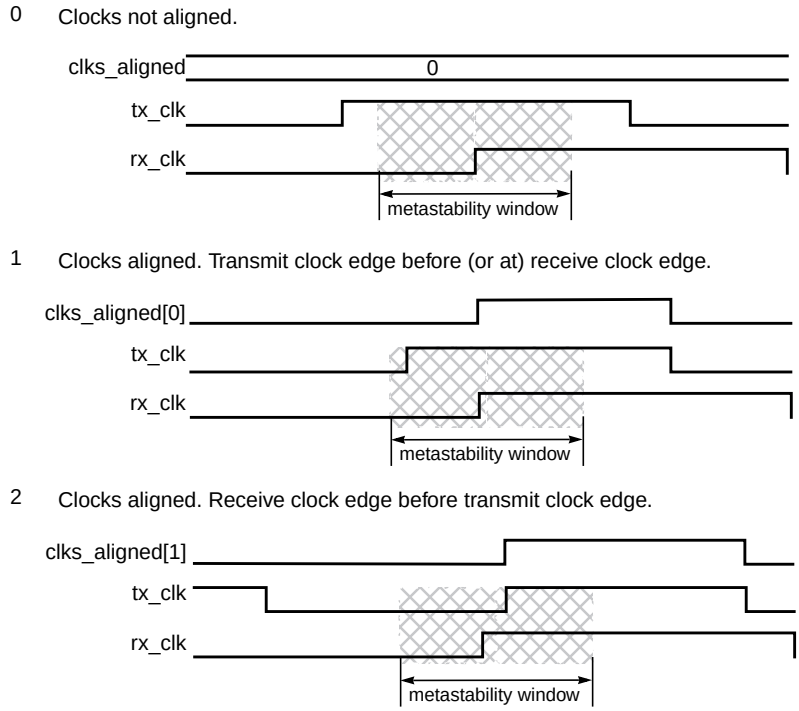
When the assertion compiler instantiates a `cdc_fx` checker, it also creates clocks monitor logic that determines when the transmit clock is within the metastability window of the receive clock (for that transmit clock). Whenever this occurs, the receive register is prone to metastability if its input value also changes in that receive clock cycle. The `clks_aligned` output from the clocks monitor is used to pass this information to the `cdc_fx` checker.

The clks_aligned output is a 2-bit value that is 0 throughout “normal” cycles. When the clocks monitor detects an active transmit clock edge in the corresponding metastability window of the receive clock edge, one of the clks_aligned bits asserts a pulse that begins at the second active clock edge and stops at the first subsequent inactive clock edge (Figure 59).

Note that:

- If tx_clk is before rx_clk, clks_aligned[0] asserts the pulse.
- If tx_clk is after rx_clk, clks_aligned[1] asserts the pulse.

**Figure 59: clks_aligned input to the cdc_fx checker**



### set_cdc_fx_metastability_window

Specifies a metastability window for one or more receive register clocks. This global directive is used by ccl. A set_cdc_fx_metastability_window directive has the following syntax:

```
set_cdc_fx_metastability_window
    –start start_time_value –stop stop_time_value
    [–tx_clock tx_clock –rx_clock rx_clock]
```

The –start and –stop values specify the metastability using directional time units:

- The –start value is the number of time units before the active receive clock edge that the associated metastability window opens.
- The function of the –stop value depends on its sign:
  - *stop_time_value* > 0

    The metastability window closes *stop_time_value* time units after the active receive clock edge.
  - *stop_time_value* < 0

    The metastability window closes –*stop_time_value* time units before the active receive clock edge.

If the directive specifies –tx_clock and –rx_clock, the directive applies to cdc_fx checkers with these clock pairs. If the directive omits –tx_clock and –rx_clock, the directive applies to the remaining cdc_fx checkers. It is an error to specify more than one directive without the –tx_clock and –rx_clock arguments.

If no set_cdc_fx_metastability_window directives are specified, the special default case described in the next section is followed. However, if at least one set_cdc_fx_metastability_window directive is specified, the "default" metastability window configuration has a duration set to 0. In this case, cdc_fx checkers not covered by explicit set_cdc_fx_metastability_window directives never inject metastability. Their cdc_fx checks never fire.

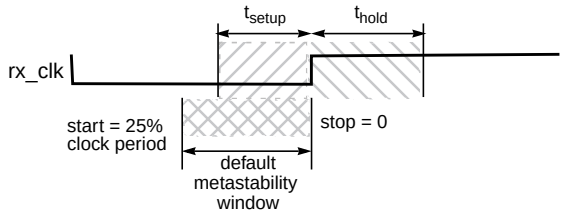
**Note:** The –tx_clock and –rx_clock arguments do not allow wildcards.

## Special Default Case

If a set_cdc_fx_metastability_window directive is not specified for a CDC crossing, the (default) metastability window is set as follows (Figure 60):

- start time is 25% of the receive clock cycle before the receive clock edge
- stop time is the receive clock edge

**Figure 60: Default metastability window**




For example, if rx_clk has period 100 time units, then the default metastability window is the same as the window set by:

```
/* 0in set_cdc_fx_metastability_window
    -start 25 -stop 0
    -tx_clock tx_clk -rx_clock -rx_clk */
```

# RUNNING CDC -FX

The metastability injectors (cdc_fx checkers and metastability detection logic) are instantiated from cdc_fx checker directives. You could specify these directives manually, but CDC paths can be numerous and this process is prone to user error. Instead, you can use a built-in feature of the cdc command to automate the process of preparing the design data for ccl. Since cdc analyzes CDC paths anyway, it can readily construct the cdc_fx directives for them. Therefore, if you include the –fx option to cdc, it generates a CDC-FX control file (0in_cdc_fx_sim_ctrl.v) that contains cdc_fx directives that you can use with the ccl compiler (Figure 61).

**Figure 61: Cdc prepares data for simulation with metastability**

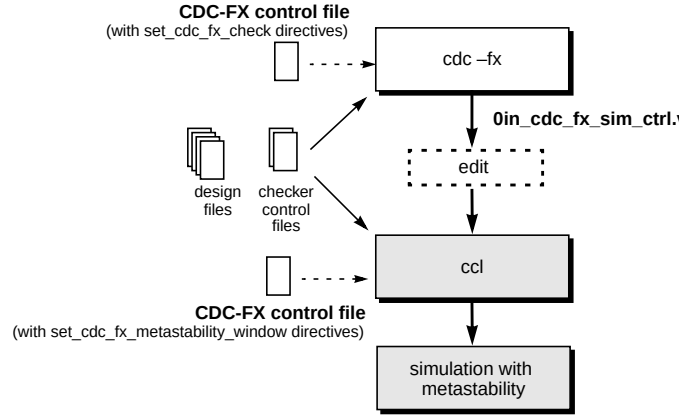



Figure 61 also shows that part of the data preparation for CDC-FX analysis is to set up an optional CDC-FX control file, which contains the set_cdc_fx_metastability_window global directives used to set the metastability windows for the cdc_fx checkers.

## set_cdc_fx_check

Turns on specified checks in corresponding cdc_fx checker directives promoted by cdc –fx. This global directive has the following syntax:

```
set_cdc_fx_check
    {–fx | –glitch_found | –glitch_swallowed}...
    [–tx_clock tx_clock] [–rx_clock rx_clock] [–scheme sync_scheme]
```

By default, cdc_fx checker directives promoted from ccl –fx are configured as follows:

- cdc_fx checkers on single-bit registers have no checks turned on.
- cdc_fx checkers on multiple-bit registers have only the cdc_fx checks turned on.

Use the set_cdc_fx_check directive to force cdc –fx to turn on cdc_fx checkers' checks.

*Important!* The ccl compiler instantiates cdc_fx and cover checkers even if none of their checks are turned on. Other checkers with no checks on are excluded unless you specify the –scov option to ccl.

You must specify at least one check to turn on (–fx, –glitch_found, –glitch_swallowed) . The –tx_clock option restricts the directive to those cdc_fx checkers with the specified transmit clock. The –rx_clock option restricts the directive to those cdc_fx checkers with the specified receive clock. The –scheme option restricts the directive to those cdc_fx checkers connected to synchronizers of the specified type. This directive does not support wildcards.

## 0in_cdc_fx_sim_ctrl.v

0-In CDC analyzes CDC information. For each CDC signal and data bus, it formulates a checker directive that instantiates a cdc_fx checker and generates metastability detection logic for modeling CDC metastability effects on the signal/bus during simulation with assertions. These cdc_fx directives are saved in the zin_cdc_fx_sim_ctrl module in the 0in_cdc_fx_sim_ctrl.v file (Figure 62).

You can edit this file to remove unnecessary directives and make changes that might apply to your design. Then, you can specify the edited file as a checker control file when you invoke ccl, for example:

```
shell prompt> cp 0in_cdc_fx_sim_ctrl.v cdc_fx_sim_ctrl.v
shell prompt> vi cdc_fx_sim_ctrl.v
shell prompt> 0in –cmd ccl –d design –ctrl cdc_fx_sim_ctrl.v ...
```

**Figure 62: Sample 0in_cdc_fx_sim_ctrl.v file**

```
//------------------------------------------------------------------
// CDC-FX Simulation File
//------------------------------------------------------------------
//Summary
//-------
//Count :            TX Clk              RX Clk
//-----------------------------------------------------------------
//   3 :           cpu_clk             pci_clk
//   2 :           pci_clk             cpu_clk

module zin_cdc_fx_sim_ctrl;

//cpu_clk --> pci_clk
/* 0in cdc_fx
    -tx_reg cpu_if1.pci_req -rx_reg mtr.CPU_REQ_temp
    -tx_clock cpu_clk -rx_clock pci_clk
    -module cpu_top
    */
//pci_clk --> cpu_clk
/* 0in cdc_fx
    -tx_reg mtr.fifo_rd -rx_reg cpu_if1.pop_temp
    -tx_clock pci_clk -rx_clock cpu_clk
    -module cpu_top
    */
//pci_clk --> cpu_clk
/* 0in cdc_fx
    -tx_reg mtr.cpu_curr_state -rx_reg pci2cpu_1bsync.r1
    -tx_clock pci_clk -rx_clock cpu_clk
    -module cpu_top
    */
//cpu_clk --> pci_clk
/* 0in cdc_fx
    -tx_reg cpu_if1.fifo1.deq_addr -rx_reg mtr.address_pci
    -tx_clock cpu_clk -rx_clock pci_clk
    -cdc_fx -module cpu_top
    */
//cpu_clk --> pci_clk
/* 0in cdc_fx
    -tx_reg cpu_if1.fifo1.deq_addr -rx_reg mtr.Trans_Id
    -tx_clock cpu_clk -rx_clock pci_clk
    -cdc_fx -module cpu_top
    */

endmodule
```

## RUNNING METASTABILITY-INJECTED SIMULATION

To prepare the design data for enabling metastability injection in simulation with assertions, you do the following:

1. Run `cdc –fx` on the design and generate `0in_cdc_fx_sim_ctrl.v` containing the `cdc_fx` checker directives:

   *shell prompt*> `0in cdc –fx .....`
2. Edit `0in_cdc_fx_sim_ctrl.v`. Remove or modify directives as desired. Add directives for additional checkers if desired.
3. Add CDC-FX global directives to a checker control file, if necessary.

These control files are all you need. To enable metastability injection in simulation with assertions, include the edited `0in_cdc_fx_sim_ctrl.v` file as a `–ctrl` argument in the `ccl` invocation (with a CDC-FX control file if needed):

*shell prompt*> `0in ccl –d dut –ctrl 0in_cdc_fx_sim_ctrl.v –ctrl cdc_fx_ctrl.v ...`

Then, run simulation with assertions as normal:

*shell prompt*> *simulator* `–f 0in_sim.arg ...`

## DESIGN STYLE SUPPORT

The checklist utility creates a `cdc_fx` directive for every CDC control signal and CDC data bus except:

- ❏ CDC paths that use custom synchronizers.
- ❏ Individual signals from multibit registers.
- ❏ CDC signals driven by two or more transmitting registers in different clock domains.
- ❏ Memory-synchronized CDC signals (i.e., signals for which the receive or transmit register is a 2D array).

You can however, manually specify `cdc_fx` directives for these cases.

## SUPPRESSING METASTABILITY INJECTION

You can suppress metastability injection during simulation for individual CDC signals or all CDC signals. The cdc_fx checkers maintain statistics but metastability is not injected into simulation.

### +0in_suppress_fx_name+*name*

To suppress metastability injection for individual signals, specify them with arguments of the following form:

```
+0in_suppress_fx_name{+name}...
```

Here, *name* is the name of the signal. Wildcards can be used in the signal names. For example:

```
+0in_suppress_fx_name+tx4_*+tx_data
```

### +0in_disable_fx_force

To suppress metastability injection for all individual signals, specify the following argument:

```
+0in_disable_fx_force
```

## CDC-FX PLI FUNCTIONS

Testbenches also can include PLI function calls that dynamically control the operation of the metastability injectors.

### $0in_always_invert_metastable_signals () ;

Inverts signals when they are metastable.

### $0in_never_invert_metastable_signals () ;

Leaves the cdc_fx checkers active, but disables metastability injection.

### $0in_randomly_invert_metastable_signals () ;

Randomly injects metastability into CDC signals when they are metastable. This is the default behavior.

# ANALYZING SIMULATION RESULTS

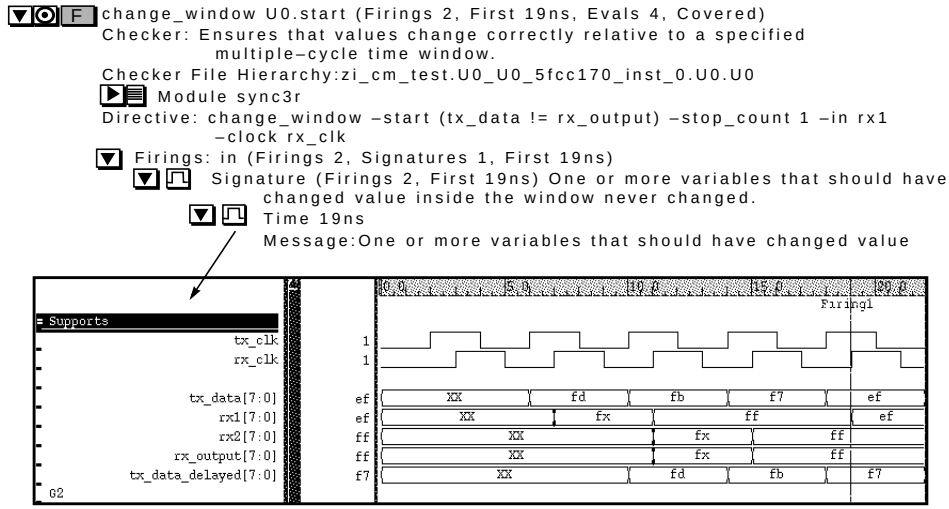
Metastability injectors in simulation uncover problems that arise from CDC metastability effects. These problems cannot be detected with basic simulation. Therefore, you should use the following methodology:

1. Start with a “clean” design.
   - End-to-end tests run completely, without errors.
   - If the design is instrumented with assertions, plain simulation with assertions does not violate any assertions.
2. Run metastability-injected simulation.
3. Use the 0-In View database viewer to analyze the results.
   - End-to-end test errors indicate receive logic is not tolerant of CDC metastability effects.
   - Assertion failures indicate receive logic is not tolerant of CDC metastability effects.
   - `cdc_fx` checker coverage shows how completely each metastability injector covers the range of metastability effects during simulation.
4. If needed, suppress some checkers and rerun metastability-injected simulation.
5. Debug any `cdc_fx` checker firings. The `cdc_fx` checker has three transfer protocol checks that by default are turned off.
   - The `cdc_fx` check ensures that the data input to the receive register does not change in a cycle for which the transmit/receive clock edges align (i.e., metastability is not possible for the corresponding register). The generated `cdc_fx` directives for CDC data buses automatically include the `–cdc_fx` option turning on this check.
   - The `glitch_caught` check ensures that metastability injection does not catch a glitch if it is not detected by simulation.
   - The `glitch_swallowed` check ensures that metastability injection does not swallow a glitch if it is detected by simulation.

## ASSERTION VIOLATIONS

If metastability-injected simulation produces assertion violations (i.e., check firings), you can analyze them the same as you do firings from basic simulation with assertions (Figure 63). Use the 0-In View viewer to examine details of the check firings and to display their waveforms. These violations are caused by design defects in the fanouts of the receiving registers that make the circuit intolerant of random delays in the transmission of their driving CDC signals.

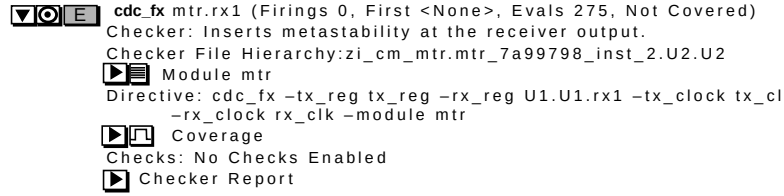
**Figure 63: Viewer entry and waveform for an assertion violation**



## CDC_FX CHECKER

The cdc_fx checker entries in the viewer database provide a variety of information about the metastability injectors during simulation (Figure 64).

**Figure 64: Viewer entry for a cdc_fx checker**




The checker report is included in entries in both the static database from ccl (0in_check.db) and the dynamic database from simulation (0in_checksim.db):

```
Checker Report
    Module                  : mtr
    Check cdc_fx            : Off <Default Off>
    –rx_q                   : U1.U1.rx1
    –name                   : rx1
    Checker Class           : Usr
    Checker Internal
```

The –rx_q field identifies the register output from the original circuit that is replaced in the new circuit—remodeled with the metastability injection logic—with the rx_q output from the checker.

**Note:** Even though the cdc_fx checker's formal attribute is , it cannot be a target for formal analysis. The csl compiler excludes the checker and its formal attribute cannot be changed. CDC-FX formal verification uses a different process to model metastability.

## Coverage

Each cdc_fx checker maintains coverage information about its performance during simulation. The checker entry in the simulation database (0in_checksim.db) has this information

Coverage
Statistics

| | |
|---|---|
| 47: | Metastable Cycles |
| 4a6e52b773b2: | Inverted Bits Bitmap |
| 26: | Bits Inverted |
| 500: | Tx_clock After Rx_clock |
| 0: | Rx_clock After Tx_clock |
| 0: | Delayed Transitions |
| 233: | Advanced Transitions |

Corner Cases

| | |
|---|---|
| 0: | All Bits Metastable |

See page 140 for details on the coverage values. The checker has one corner case: All Bits Metastable. If this is zero, some bits of the CDC bus are not inverted in simulation. Additional test cases might be needed to fully test the CDC metastability effects for the circuit.

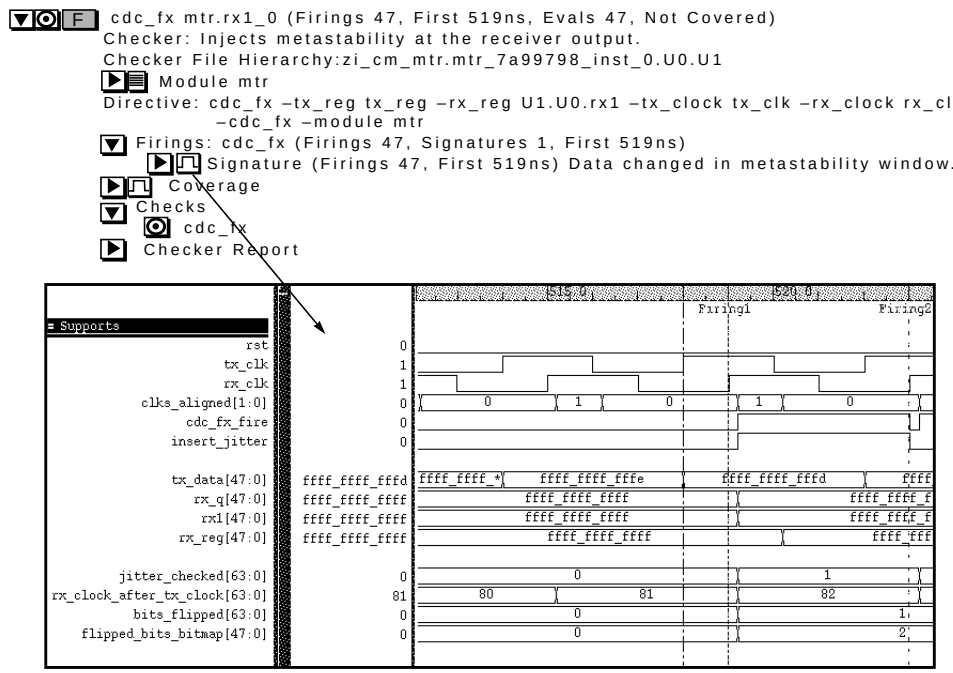
## cdc_fx Check Firings

The cdc_fx checker's cdc_fx check fires when the active edge of the transmit clock is in the metastability window of the receive clock. These cycles are candidates for metastability injection. The Metastable Cycles evaluation statistic increments each cycle this happens. Normally, this is not problematic. The logic in the fanout of the receive register is expected to tolerate this situation. Therefore, the cdc_fx check is turned off by default. The viewer entry shows:

```
Checks: No Checks Enabled
```

Some design styles have transmitting circuitry that presumes data is stable during cycles when both clock edges occur in the metastability window. No metastability can occur and the receiving logic does not need to account for CDC metastability effects. In such cases, you can use the cdc_fx check to verify the transmit data remains stable when metastability can occur (Figure 65).

**Figure 65: Viewer entry and waveform for a cdc_fx check violation**




**Note:** The ccl compiler discards checkers whose checks are all disabled. The cdc_fx and coverage checkers are exceptions.

## CDC_FX CHECKER PINS

The cdc_fx checker waveforms show additional signals that are helpful for your analysis, including the following:

- inject_metastability
- rx_q

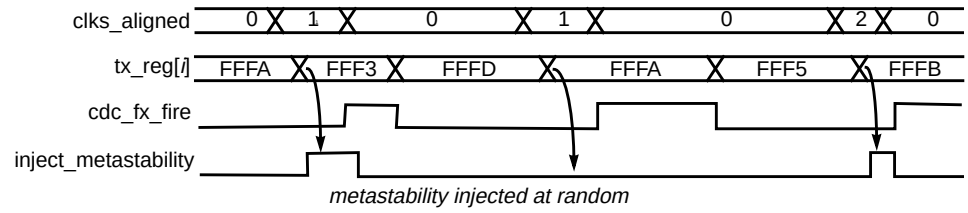
### inject_metastability

Unless the $0in_always_invert_metastable_signals or $0in_never_invert_metastable_signals PLI calls are in effect, the default behavior of the cdc_fx checker is to randomly inject metastability when it can occur. So, the cdc_fx checker has a hierarchy of events that all must occur before it goes ahead and injects metastability (Table 9). When this happens, the checker asserts its inject_metastability output (Figure 66).

**Table 9: Hierarchy of events for metastability injection**

| Level | Condition | Result |
|---|---|---|
| 1 | Clocks align | clks_aligned goes > 0 |
| 2 | Transmit clock remains asserted | clks_aligned stays > 0 |
| 3 | Transmit register output changes | cdc_fx_fire asserts (if on) |
| 4 | Checker randomly selects the output bits for metastability injection. | inject_metastability asserts |

**Figure 66: inject_metastability output from the cdc_fx checker**



### rx_q

The rx_q output from the cdc_fx checker is the replacement for the rx_reg output from the original logic. When ccl compiles the design for simulation, each cdc_fx checker is connected so the output from the transmit register is routed to the cdc_fx checker and the rx_q output from the checker drives the load from the original receive register.

For most cycles, the transmit data is latched by the checker and passed through to the rx_q output when the receive clock activates. This mimics the behavior of the original circuit under normal simulation.

But, when the checker determines it should inject metastability, randomly selected bits of the rx_q output are inverted. The rx_q output reflects a corrupted value unrelated to the original transmission. This condition emulates data transmission metastability from crossing clock domains. The circuit must be immune to these effects.

# 5 CDC-FX

# INDEX


### Symbols

$0in_always_invert_metastable_signals 151
$0in_never_invert_metastable_signals 151
$0in_randomly_invert_metastable_signals 152
+0in_disable_fx_force 151
+0in_suppress_fx_name 151
+define+ 38
+incdir+ 38
+libext 38
+skip_modules 38
+skip_syscalls 38

### Numerics

0in.log 42
0in_cdc_fx_sim_ctrl.v 148
0in_checklist.db 42
0in_checklist_cdc.rpt 42
0in_checklist_cdc_ctrl.v 42
0in_checklist_hier_ctrl_.v 119
0in_checklist_loop.rpt 42
0in_detail.log 42

### A

ad hoc synchronizer 21
analyzing simulation results 153
assertion 28
assertion violations 154
assertion-based verification 28
assigning ports to clock domains 51
async_reset 65
async_reset_no_sync 66
asynchronous clocks 15
asynchronous inputs 15
asynchronous reset detection table 115

### B

blackbox 67
blackbox crossing table 114
bus_dff_sync_gated_clk 68
bus_four_latch 70
bus_two_dff 72
bus_two_dff_phase 74

### C

CDC check entry 103
CDC checker types 123
CDC checkers in simulation 30
CDC checking a parent module 118
CDC clocks 49
CDC design report 106
CDC evaluations table 113
CDC firings 102
cdc input files 39
CDC jitter 17
cdc output files 41
CDC promotion 58
CDC protocol checker 29
CDC reporting 58
CDC scheme 43
CDC scheme types 43
CDC schemes 63
CDC summary table 110
CDC unknowns table 112
CDC violations table 111
–cdc_fx 138, 139
cdc_fx checker 137, 155
  pins 158
CDC-FX 135
CDC-FX PLI functions 151
changing value 12
check firings 156
checker 29
checker control files 40
checker promotion 27, 44
checkers
  cdc_fx 137
checklist CDC hierarchy control file 119
checklist command invocation 37

# INDEX


classifying clock domain crossings 55
clks_aligned 143
–clock 55
clock domain 130
clock domain crossing 130
clock domain crossing report 110
clock domains 15
clock group 15
clock group summary table 106
clock groups 49
clock jitter 135
clocks 49
combo_logic 75
constant 12
–control 56
control files 40
control signal synchronizers 18
control signals 56
corner cases
  All Bits Flipped 140
counterexample 31
coverage 156
–ctrl 37
custom 54
custom synchronization modules table 115
custom_sync 77

**D**
–data 57
data bus synchronizer 19
data buses 57
data preparation 122
Debussy debug environment 105
default metastability window 146
design example 31
design style support 150
detailed design information table 108
dff 54
dff_sync_gated_clk 79
directive conventions 12
dmux 81

**E**
environment variables 37
evaluation 44, 58
exclude_hier 61

**F**
fanin_different_clks 82
fence 17
formal constraint 31
formal target 31
formal tools 31
formal verification 31
four_latch 84
–from 60
–from_group 60
functions 151

**G**
general design information table 108
global directives
  set_black_box 61
  set_cdc_clock 50
  set_cdc_port_domain 51
  set_cdc_reconvergence_depth 62
  set_cdc_report 59
  set_cdc_signal 56
  set_cdc_synchronizer 54
  set_constant 52
global reconvergence 24
–gray_coded 57
gray-coded variables 57
groups 49

**H**
hierarchical CDC verification 118
hold time 141

**I**
identifying black boxes 61
identifying custom synchronizers 53
inferred clock groups table 107
inject_metastability 158
integration
  with Debussy/Verdi 105

**J**
jitter 17, 135

**L**
local reconvergence 23

**M**
mean-time-between-failure 16
memory synchronization table 113
memory_sync 85
metastability 16

metastability effects 130
metastability injection 132, 134
metastability window 145
metastability windows 141
metastable 16, 130
metastable registers 130
  in hardware 130
  in normal simulation 131
modeling metastability 132
MTBF 16
multi_bits 87
multi_sync_mux_select 89
multibit reconvergence 24
multicycle reconvergence 23

**N**

negative stop value 142
netlist analysis 27
no synchronizer 116
no_sync 90

**O**

out-of-band signals 17
–oy 38

**P**

–pins 55
pins 158
PLI functions 151
port domain information table 109
promoted checks 59
–promotion 59
promotion 27, 44, 58
protocol checkers 29
pseudorandom delay insertion 134
pulse_sync 92

**Q**

–qv 38
–qy 38

**R**

ratioed synchronous clocks 21
recommended methodology 37
reconvergence 23, 57, 93
reconvergence depth 62
redundant 95
reporting 58
reporting off 44
RSC 21
running 0-In Check 125
running 0-In View 102, 125
running CDC 36
running cdc –fx 147
running simulation with CDC-FX checkers 150
–rx_clock 60, 138, 145
rx_q 159
–rx_reg 137
–rx_reset 138

**S**

–scheme 59
schemes
  async_reset 65
  async_reset_no_sync 66
  blackbox 67
  bus_dff_sync_gated_clk 68
  bus_four_latch 70
  bus_two_dff 72
  bus_two_dff_phase 74
  combo_logic 75
  custom_sync 77
  dff_sync_gated_clk 79
  dmux 81
  fanin_different_clks 82
  four_latch 84
  memory_sync 85
  multi_bits 87
  multi_sync_mux_select 89
  no_sync 90
  pulse_sync 92
  reconvergence 93
  redundant 95
  shift_reg 96
  two_dff 97
  two_dff_phase 99
set_black_box 61
set_cdc_clock 50
set_cdc_fx_check 148
set_cdc_fx_metastability_window 145
set_cdc_port_domain 51
set_cdc_reconvergence_depth 62
set_cdc_report 59
set_cdc_signal 56
set_cdc_synchronizer 54
set_constant 52
set_reset 61
setting port values to constants 52

# INDEX

setup time 141
–severity 59
severity 43, 58
severity levels 58
shift_reg 96
simulation 30
specifying design intent 29
–split 57
splitting buses 57
–stable 58
stable CDC signals 58
–start 145
statistics
  Bits Jittered 140
  Jittered Bits Bitmap 140
  Metastable Cycles (Evals) 140
  Rx_clock After Tx_clock 140
  Tx_clock After Rx_clock 140
–stop 145
structured synchronizers 18
suppressing metastability injection 151
synchronization scheme 17

synchronizer 17

**T**

target design 40
–through 60
–to 60
transfer protocol 17
transfer protocol checkers 29
transmitting signals table 117
two_dff 97
two_dff_phase 99
–tx_clock 60, 138, 145
–tx_reg 137

**U**

UNIX environmant variables 37
unknown 44, 58
user-specified clock groups table 107

**V**

value 12
variable 12
Verdi debug environment 105
violation 43, 58

**W**

waveforms 154
–wd 38
working directory 39
wrong clock 116
wrong polarity violation 116
wrong reset polarity 116

# INDEX

# End-User License Agreement

**IMPORTANT - USE OF THIS SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE SOFTWARE.**

**This license is a legal "Agreement" concerning the use of Software between you, the end user, either individually or as an authorized representative of the company acquiring the license, and Mentor Graphics Corporation and Mentor Graphics (Ireland) Limited acting directly or through their subsidiaries or authorized distributors (collectively "Mentor Graphics"). USE OF SOFTWARE INDICATES YOUR COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. If you do not agree to these terms and conditions, promptly return, or, if received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.**

## END-USER LICENSE AGREEMENT

1. **GRANT OF LICENSE.** The software programs you are installing, downloading, or have acquired with this Agreement, including any updates, modifications, revisions, copies, documentation and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Mentor Graphics grants to you, subject to payment of appropriate license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form; (b) for your internal business purposes; and (c) on the computer hardware or at the site for which an applicable license fee is paid, or as authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or service plan purchased, apply to the following and are subject to change: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be communicated and technically implemented through the use of authorization codes or similar devices); (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. Current standard policies and programs are available upon request.

2. **ESD SOFTWARE.** If you purchased a license to use embedded software development ("ESD") Software, Mentor Graphics grants to you a nontransferable, nonexclusive license to reproduce and distribute executable files created using ESD compilers, including the ESD run-time libraries distributed with ESD C and C++ compiler Software that are linked into a composite program as an integral part of your compiled computer program, provided that you distribute these files only in conjunction with your compiled computer program. Mentor Graphics does NOT grant you any right to duplicate or incorporate copies of Mentor Graphics' real-time operating systems or other ESD Software, except those explicitly granted in this section, into your products without first signing a separate agreement with Mentor Graphics for such purpose.

3. **BETA CODE.** Portions or all of certain Software may contain code for experimental testing and evaluation ("Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to you a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. This grant and your use of the Beta Code shall not be construed as marketing or offering to sell a license to the Beta Code, which Mentor Graphics may choose not to release commercially in any form. If Mentor Graphics authorizes you to use the Beta Code, you agree to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. You will contact Mentor Graphics periodically during your use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of your evaluation and testing, you will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements. You agree that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on your feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this subsection shall survive termination or expiration of this Agreement.

4. **RESTRICTIONS ON USE.** You may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. You shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. You shall not make Software available in any form to any

person other than employees and contractors, excluding Mentor Graphics' competitors, whose job performance requires access. You shall take appropriate action to protect the confidentiality of Software and ensure that any person permitted access to Software does not disclose it or use it except as permitted by this Agreement. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, you shall not reverse-assemble, reverse-compile, reverse-engineer or in any way derive from Software any source code. You may not sublicense, assign or otherwise transfer Software, this Agreement or the rights under it, whether by operation of law or otherwise ("attempted transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable transfer charges. Any attempted transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and licenses granted under this Agreement.

The terms of this Agreement, including without limitation, the licensing and assignment provisions shall be binding upon your heirs, successors in interest and assigns. The provisions of this section 4 shall survive the termination or expiration of this Agreement.

**5. LIMITED WARRANTY.**

5.1. Mentor Graphics warrants that during the warranty period Software, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Software will meet your requirements or that operation of Software will be uninterrupted or error free. The warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. You must notify Mentor Graphics in writing of any nonconformity within the warranty period. This warranty shall not be valid if Software has been subject to misuse, unauthorized modification or installation. MENTOR GRAPHICS' ENTIRE LIABILITY AND YOUR EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF SOFTWARE TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF SOFTWARE THAT DOES NOT MEET THIS LIMITED WARRANTY, PROVIDED YOU HAVE OTHERWISE COMPLIED WITH THIS AGREEMENT. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) SOFTWARE WHICH IS LICENSED TO YOU FOR A LIMITED TERM OR LICENSED AT NO COST; OR (C) EXPERIMENTAL BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."

5.2. THE WARRANTIES SET FORTH IN THIS SECTION 5 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO SOFTWARE OR OTHER MATERIAL PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

6. **LIMITATION OF LIABILITY.** EXCEPT WHERE THIS EXCLUSION OR RESTRICTION OF LIABILITY WOULD BE VOID OR INEFFECTIVE UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT PAID BY YOU FOR THE SOFTWARE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER.

7. **LIFE ENDANGERING ACTIVITIES.** NEITHER MENTOR GRAPHICS NOR ITS LICENSORS SHALL BE LIABLE FOR ANY DAMAGES RESULTING FROM OR IN CONNECTION WITH THE USE OF SOFTWARE IN ANY APPLICATION WHERE THE FAILURE OR INACCURACY OF THE SOFTWARE MIGHT RESULT IN DEATH OR PERSONAL INJURY.

8. **INDEMNIFICATION.** YOU AGREE TO INDEMNIFY AND HOLD HARMLESS MENTOR GRAPHICS AND ITS LICENSORS FROM ANY CLAIMS, LOSS, COST, DAMAGE, EXPENSE, OR LIABILITY, INCLUDING ATTORNEYS' FEES, ARISING OUT OF OR IN CONNECTION WITH YOUR USE OF SOFTWARE AS DESCRIBED IN SECTION 7.

**9. INFRINGEMENT.**

9.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against you alleging that Software infringes a patent or copyright or misappropriates a trade secret in the United States, Canada, Japan, or

member state of the European Patent Office. Mentor Graphics will pay any costs and damages finally awarded against you that are attributable to the infringement action. You understand and agree that as conditions to Mentor Graphics' obligations under this section you must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to defend or settle the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.

9.2. If an infringement claim is made, Mentor Graphics may, at its option and expense: (a) replace or modify Software so that it becomes noninfringing; (b) procure for you the right to continue using Software; or (c) require the return of Software and refund to you any license fee paid, less a reasonable allowance for use.

9.3. Mentor Graphics has no liability to you if infringement is based upon: (a) the combination of Software with any product not furnished by Mentor Graphics; (b) the modification of Software other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of Software as part of an infringing process; (e) a product that you make, use or sell; (f) any Beta Code contained in Software; (g) any Software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; or (h) infringement by you that is deemed willful. In the case of (h) you shall reimburse Mentor Graphics for its attorney fees and other costs related to the action upon a final judgment.

9.4. THIS SECTION 9 STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS AND YOUR SOLE AND EXCLUSIVE REMEDY WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY SOFTWARE LICENSED UNDER THIS AGREEMENT.

10. **TERM.** This Agreement remains effective until expiration or termination. This Agreement will immediately terminate upon notice if you exceed the scope of license granted or otherwise fail to comply with the provisions of Sections 1, 2, or 4. For any other material breach under this Agreement, Mentor Graphics may terminate this Agreement upon 30 days written notice if you are in material breach and fail to cure such breach within the 30-day notice period. If Software was provided for limited term use, this Agreement will automatically expire at the end of the authorized term. Upon any termination or expiration, you agree to cease all use of Software and return it to Mentor Graphics or certify deletion and destruction of Software, including all copies, to Mentor Graphics' reasonable satisfaction.

11. **EXPORT.** Software is subject to regulation by local laws and United States government agencies, which prohibit export or diversion of certain products, information about the products, and direct products of the products to certain countries and certain persons. You agree that you will not export any Software or direct product of Software in any manner without first obtaining all necessary approval from appropriate local and United States government agencies.

12. **RESTRICTED RIGHTS NOTICE.** Software was developed entirely at private expense and is commercial computer software provided with RESTRICTED RIGHTS. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement under which Software was obtained pursuant to DFARS 227.7202-3(a) or as set forth in subparagraphs (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable. Contractor/manufacturer is Mentor Graphics Corporation, 8005 SW Boeckman Road, Wilsonville, Oregon 97070-7777 USA.

13. **THIRD PARTY BENEFICIARY.** For any Software under this Agreement licensed by Mentor Graphics from Microsoft or other licensors, Microsoft or the applicable licensor is a third party beneficiary of this Agreement with the right to enforce the obligations set forth herein.

14. **AUDIT RIGHTS.** With reasonable prior notice, Mentor Graphics shall have the right to audit during your normal business hours all records and accounts as may contain information regarding your compliance with the terms of this Agreement. Mentor Graphics shall keep in confidence all information gained as a result of any audit. Mentor Graphics shall only use or disclose such information as necessary to enforce its rights under this Agreement.

15. **CONTROLLING LAW AND JURISDICTION.** THIS AGREEMENT SHALL BE GOVERNED BY AND CONSTRUED UNDER THE LAWS OF THE STATE OF OREGON, USA, IF YOU ARE LOCATED IN NORTH OR SOUTH AMERICA, AND THE LAWS OF IRELAND IF YOU ARE LOCATED OUTSIDE OF NORTH AND SOUTH AMERICA. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of Dublin, Ireland when the laws of Ireland apply, or Wilsonville, Oregon when the laws of Oregon apply. This section shall not restrict Mentor Graphics' right to bring an action against you in the jurisdiction where your place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.

16. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.

17. **PAYMENT TERMS AND MISCELLANEOUS.** You will pay amounts invoiced, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements, including but not limited to any purchase order terms and conditions, except valid license agreements related to the subject matter of this Agreement (which are physically signed by you and an authorized agent of Mentor Graphics) either referenced in the purchase order or otherwise governing this subject matter. This Agreement may only be modified in writing by authorized representatives of the parties. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse. The prevailing party in any legal action regarding the subject matter of this Agreement shall be entitled to recover, in addition to other relief, reasonable attorneys' fees and expenses.

Rev. 040401, Part Number 221417